



Intel® Arria® 10 SoC UEFI Boot Loader User Guide



Online Version



Send Feedback

UG-20100

ID: **683536**

Version: **2017.12.15**

Contents

1. Intel® Arria® 10 SoC UEFI Boot Loader User Guide.....	4
1.1. Acronyms and Definitions.....	4
1.2. Recommended System Requirements.....	7
1.2.1. Minimum Hardware Requirements.....	7
1.2.2. Minimum Software Requirements.....	7
1.3. Installation Folders.....	7
1.4. Boot Flow Overview.....	8
1.5. Getting Started.....	9
1.5.1. Compiling the Hardware Design.....	9
1.5.2. Generating the Boot Loader and Device Tree for UEFI Boot Loader.....	20
1.5.3. Building the UEFI Boot Loader.....	32
1.5.4. Creating an SD Card Image.....	38
1.5.5. Creating a QSPI Image.....	56
1.5.6. Booting the Board with SD/MMC.....	63
1.5.7. Booting the Board with QSPI.....	68
1.5.8. Early I/O Release.....	73
1.5.9. Booting Linux Using the UEFI Boot Loader.....	77
1.5.10. Debugging an Example Project.....	82
1.5.11. UEFI Boot Loader Customization.....	98
1.5.12. Enabling Checksum for the FPGA Image.....	104
1.5.13. NAND Bad Block Management.....	107
1.6. Enabling the UEFI DXE Phase and the UEFI Shell.....	109
1.7. Using the Network Feature Under the UEFI Shell.....	110
1.7.1. Obtain the IP Address from the DHCP Server.....	110
1.7.2. Use a Static IP Address.....	111
1.7.3. Ping Test.....	111
1.7.4. TFTP Test.....	112
1.8. Creating your First UEFI Application.....	113
1.9. Using Arm DS-5 Intel SoC FPGA Edition (For Windows Only).....	115
1.9.1. Importing the DS-5 Project.....	115
1.9.2. Building UEFI in Arm DS-5 Intel SoC FPGA Edition.....	118
1.9.3. Importing the Arm DS-5 Intel SoC FPGA Edition Debug Script.....	119
1.10. Pit Stop Utility Guide.....	119
1.10.1. Enabling or Disabling the Pit Stop Utility.....	120
1.10.2. Pit Stop Utility Commands and Use.....	121
1.11. Porting HWLIBs to UEFI Guidelines.....	124
1.11.1. Coding Standard.....	124
1.11.2. Data Types Conversion.....	126
1.11.3. Status Code Conversion.....	126
1.11.4. Header Conversion.....	127
1.11.5. Basic I/O Functions Conversion.....	127
1.12. Tera Term Installation.....	127
1.12.1. Making a Serial Connection.....	134
1.13. Minicom Installation.....	136
1.13.1. Making a Serial Connection.....	137
1.14. Win32DiskImager Tool Installation.....	138
1.14.1. Using Win32Disk Imager.....	141

1.15. TFTPd64 By Ph.Jounin Installation.....	141
1.15.1. Checking Your Machine or Laptop IPv4 Address.....	143
1.15.2. Enabling the TFTP Server through the Windows Firewall.....	143
1.15.3. Using the TFTP Server in Windows.....	143
1.16. Revision History of Intel Arria 10 SoC UEFI Boot Loader User Guide.....	147

1. Intel® Arria® 10 SoC UEFI Boot Loader User Guide

This document provides instructions on how to use the Unified Extensible Firmware Interface (UEFI) boot loader for Intel® Arria® 10 SoC as a second stage boot loader.

The instructions in this document summarize how to:

- Recompile the hardware design that comes with your Intel Arria 10 SoC Embedded Development Suite (EDS) 15.0.1.
- Generate and build both the UEFI boot loader and UEFI boot loader device tree.
- Program the UEFI boot loader image into the QSPI or SD/MMC card.
- Program the HPS to load an real-time operating system (RTOS) or bare-metal application directly from flash devices at board power-on.

1.1. Acronyms and Definitions

Table 1. UEFI Boot Loader User Guide Acronyms and Definitions

Acronym or Term	Definition
AXF	Object file format generated by Arm* compiler that contains both object code and debug information. File extension is .axf.
BSP	Board support package
CRC	Cyclical redundancy check
DDR4	DDR4 SDRAM; an abbreviation for double data rate fourth generation synchronous dynamic random-access memory.
DTB	Device tree blob. File extension is .dtb.
DTS	Device tree source. File extension is .dts.
DS-5	Arm Development Studio 5* (DS-5*) Intel SoC FPGA Edition; an end-to-end suite of tools for embedded C/C++ software development on any Intel SoC FPGA. The tool combines the features of the Arm DS-5 Intel SoC FPGA Edition with powerful FPGA debugging capabilities, providing unmatched visibility and control of your SoC FPGA. The SoC FPGA Embedded Development Suite (EDS) installs the Arm Development Studio 5 (DS-5) Intel SoC FPGA Edition.
ELF	Executable and linkable format; a common standard file format for executables, object code, shared libraries and core dumps. File extension is .elf.
FPGA	Field programmable gate array
FTDI	Future technology devices international; a Scottish, privately held semiconductor device company specializing in Universal Serial Bus (USB) technology.
<i>continued...</i>	

Acronym or Term	Definition
GHRD	Golden hardware reference design
GSRD	Golden system reference design
INF	EDK tool module information (INF) file
I/O	Input and output
JTAG	Joint Test Action Group; a software development and debug port
LED	light emitting diode
MKPIMAGE	The second-stage boot loader (SSBL) image tool; this tool creates a boot ROM compatible image of the Intel boot loader. The mkpimage tool generates the header and CRC checksum and inserts them into the output image with the SSBL program image and its exception vector.
OCRAM	on-chip RAM
PEI (UEFI)	UEFI pre-EFI initialization phase
RTOS	Real-time operating system
QSPI	Quad serial peripheral interface
Platform Designer	A system integration tool that is included as part of Intel Quartus® Prime software. Platform Designer captures system-level hardware designs at a high level of abstraction and also automates the task of defining and integrating custom HDL design blocks, commonly referred to as design modules, IP cores, or components.
Intel Quartus Prime	A programmable logic design software produced by Intel. You can analyze and synthesize HDL designs using Intel Quartus Prime.
RBF	Raw binary file. Contains configuration data for use outside of the Intel Quartus Prime software. A raw binary file contains the binary equivalent of a tabular text file (.tff). File extension is .rbf.
SoC FPGA EDS	SoC FPGA Embedded Development Suite is a comprehensive tool suite for embedded software development of Intel SoC devices. The tool suite contains development tools, utility programs, run-time software and application examples that enable firmware and application software development on the Intel SoC hardware platform.
SOF	SRAM object file with a .sof file extension. The compiler's assembler module or the makeprogfile command line utility generates this binary file. A .sof file contains the data for configuring all of the SRAM-based Intel devices supported by the Intel Quartus Prime software.
SPI	Serial peripheral interface
SRAM	Static random access memory
UART	Universal asynchronous receiver and transmitter
UEFI	The Unified Extensible Firmware Interface (UEFI) is a specification that defines a software interface between an operating system and platform firmware. UEFI replaces the Basic Input/Output System (BIOS) firmware interface, originally present in all IBM® PC-compatible personal computers. In practice, most UEFI firmware images provide
continued...	

Acronym or Term	Definition
	legacy support for BIOS services. UEFI can support remote diagnostics and repair of computers, even without another operating system.
USB	Universal serial bus

1.2. Recommended System Requirements

To load and execute the Intel Stratix® 10 SoC Unified Extensible Firmware Interface (UEFI) boot loader, your system must meet the following requirements.

1.2.1. Minimum Hardware Requirements

- Windows* PC or Linux* workstation with the following configuration:
 - Serial termination (Minicom for Linux and Tera Term for Windows)
 - Micro SD card slot or Micro SD card writer or SD capable writer with SD to micro SD converter
- Intel Arria 10 SoC development board revision A.1 or A.2. Alternatively, you can use Intel Arria 10 SoC development board revision B.1 with:
 - DDR4 memory card
 - Boot MicroSD flash daughter card (6XX-44320R) or Boot QSPI flash daughter card (6XX-44334R)

1.2.2. Minimum Software Requirements

Table 2. SoC FPGA Embedded Development Suite and Intel Quartus Prime Version

Intel Arria 10 SoC Development Board	SoC FPGA Embedded Development Suite	Git Tag of socfpga_udk2015 Branch
RevA.1 or A.2	v15.0.1 or 15.1	tags/rel_socfpga_arria10_soceds_15.0.1
RevB.1	v15.1.1	tags/rel_socfpga_arria10_soceds_15.1.1
RevC1	v16.0	tags/rel_socfpga_arria10_soceds_16.0

1.3. Installation Folders

The default installation folder for the Intel Quartus Prime Pro Edition Programmer and tools are:

Table 3. Quartus Installation Directory

Intel Quartus Prime Pro Edition Installation Directory	Description
c:\altera\15.1.1\qprogrammer on Windows ~/altera/15.1.1/qprogrammer on Linux	Intel Quartus Prime Pro Edition Programmer installation path
c:\altera\15.1.1\quartus on Windows ~/altera/15.1.1/quartus on Linux	Intel Quartus Prime Pro Edition installation path

The default installation folders for the SoC FPGA EDS are:

Table 4. SoC FPGA EDS Installation Directory

Intel Quartus Prime Pro Edition Installation Directory	Description
c:\altera\15.1.1\qprogrammer on Windows ~/altera/15.1.1/embedded on Linux	SoC FPGA EDS Programmer installation path
c:\altera\15.1.1\embedded\examples\hardware on Windows ~/altera/15.1.1/embedded\examples\hardware on Linux	SoC FPGA EDS hardware examples path

Throughout this document *<Quartus_Installation_Directory>* denotes the installation location for the Intel Quartus Prime Pro Edition tools.
<SoCEDs_Installation_Directory> denotes the installation location of the SoC FPGA EDS.

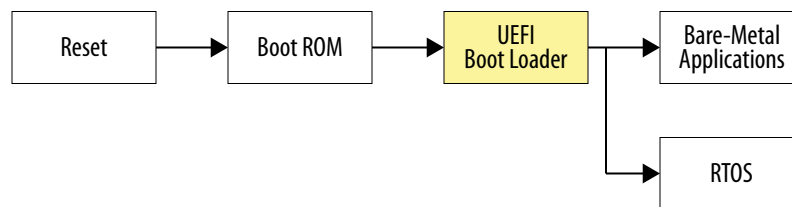
1.4. Boot Flow Overview

A typical UEFI boot flow runs entirely on the on-chip memory of the HPS and is a default selection for booting a bare-metal application and RTOS. UEFI replaces the MPL boot loader on Cyclone® V and Arria V devices.

UEFI boot loader tasks include:

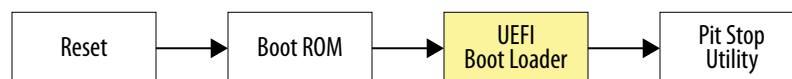
- Initializing DDR SDRAM memory
- Configuring low level hardware such as PLL, IOs, and pin-muxing
- Supporting basic hardware diagnostic features
- Fetching subsequent boot software such as the operating system package or kernel image.

Figure 1. Typical UEFI Boot Flow



Pre-DDR diagnostic boot flow boots into a diagnostic utility called Pit Stop. This utility executes from the on-chip memory of the HPS and debugs and performs diagnosis prior to booting the next stage. The **PcdEnablePitStopUtility** token in the *Arria10SoCPkg.dsc* file enables the Pit Stop utility.

Figure 2. Pre-DDR Diagnostic Boot Flow



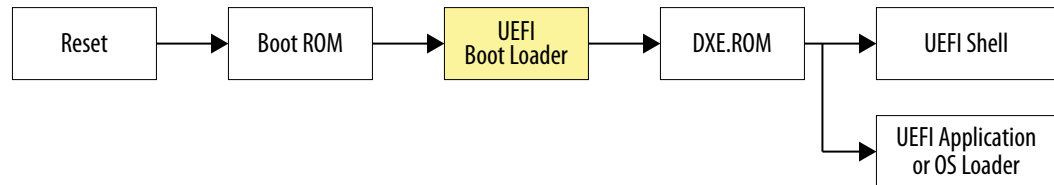
Post-DDR boot stage, also known as UEFI DXE phase, supports extended UEFI functionality. This boot flow allows you to access a broad range of pre-existing UEFI utilities already developed by the open source community.

Booting the DXE phase requires DDR SDRAM to be ready. In the post-DDR extended boot flow, you can boot into the UEFI shell to access features such as TFTP file transfer, scripting and running your UEFI applications. You can continue to boot after the UEFI shell by using commands such as:

- `runaxf` to boot to an ELF binary
- Linux Loader to continue booting in Linux

Note: SoC FPGA EDS 16.0 and above supports booting Linux.

Figure 3. Post-DDR Extended Boot Flow



1.5. Getting Started

1.5.1. Compiling the Hardware Design

Task Time: 30 Minutes

If you make any changes, you must re-compile the design that is packaged in the SoC FPGA EDS. Otherwise, you can directly work with the pre-compiled design. As an example, the following sections describe how to compile the hardware design packaged in the SoC FPGA EDS.

1.5.1.1. Open a Intel Quartus Prime Project

1. Open a Nios II command shell.
2. Make a copy of the GHRD into your working directory:

```

$ cd /cygdrive/<your_working_directory>

$ cp -r /cygdrive/c/altera/16.0/embedded/examples/hardware/
a10_soc_devkit_ghrd/ .

$ cd a10_soc_devkit_ghrd
  
```

Figure 4. Copying GHRD to Working Directory

```

~/cygdrive/c/tzyway/GSRD/a10_soc_devkit_ghrd
Altera Nios2 Command Shell [GCC 4]
Version 16.0, Build 211
-----
nios@nios01-M200: /cygdrive/c/altera/16.0
$ cd /cygdrive/c/tzyway/GSRD/
nios@nios01-M200: /cygdrive/c/tzyway/GSRD
$ cp -r /cygdrive/c/altera/16.0/embedded/examples/hardware/a10_soc_devkit_ghrd/ ./
nios@nios01-M200: /cygdrive/c/tzyway/GSRD
$ cd a10_soc_devkit_ghrd/
nios@nios01-M200: /cygdrive/c/tzyway/GSRD/a10_soc_devkit_ghrd
$ ls
Makefile          fpga_dp.sdc          ghrd_10as066n2_board_info.xml  ip
construct_subsys_dp.tcl  fpga_pcie.sdc        ghrd_10as066n2_top.v          jtag.sdc
construct_subsys_pcie.tcl ghrd_10as066n2       ghrd_reset.tcl                output_files
construct_subsys_sgml.tcl ghrd_10as066n2.dtb   ghrd_sc_script.tcl            readme.txt
create_ghrd_qsys.tcl    ghrd_10as066n2.dts   ghrd_timing.sdc               sd_fat.tar.gz
create_ghrd_quartus.tcl  ghrd_10as066n2.qsf   hps_a10_common_board_info.xml software
create_ghrd_top.tcl     ghrd_10as066n2.qsys  hps_a10_devkit_board_info.xml tgz
ctl_tapping.stp         ghrd_10as066n2.qsys  hps_isw_handoff               top_level_template.v.tsp
design_config.tcl       ghrd_10as066n2.sopcinfo hps_ignli.sdc
  
```

3. Delete the pre-built output, software handoff, GHRD and boot loader files. These files are regenerated when you re-compile the hardware design.

```

$ rm -rf output_files/ software/ hps_isw_handoff/ ghrd_10as066n2/ software/
bootloader sd_fat.tar.gz
  
```

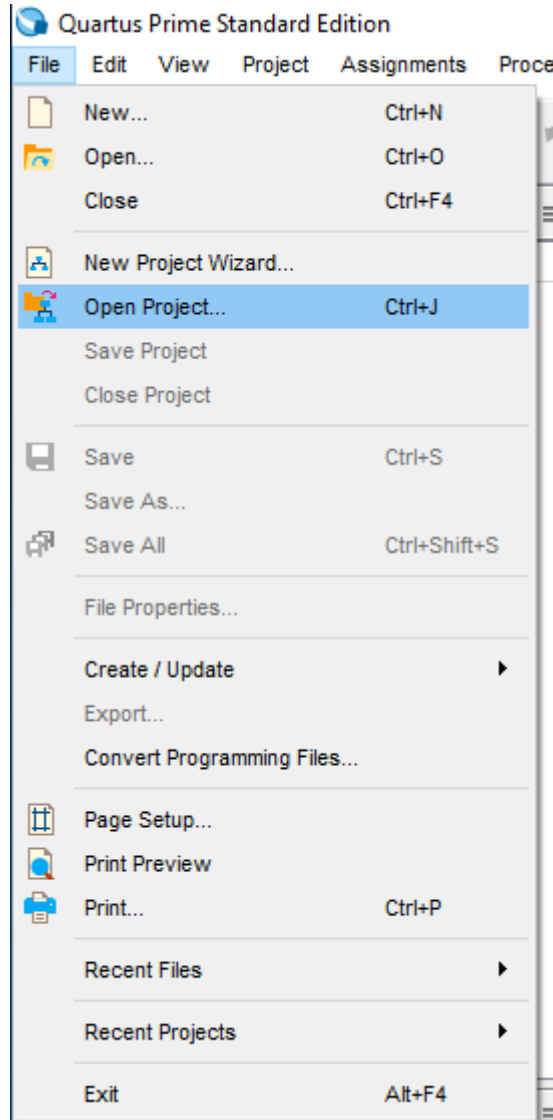
4. Launch Intel Quartus Prime.

```

$ quartus &
  
```

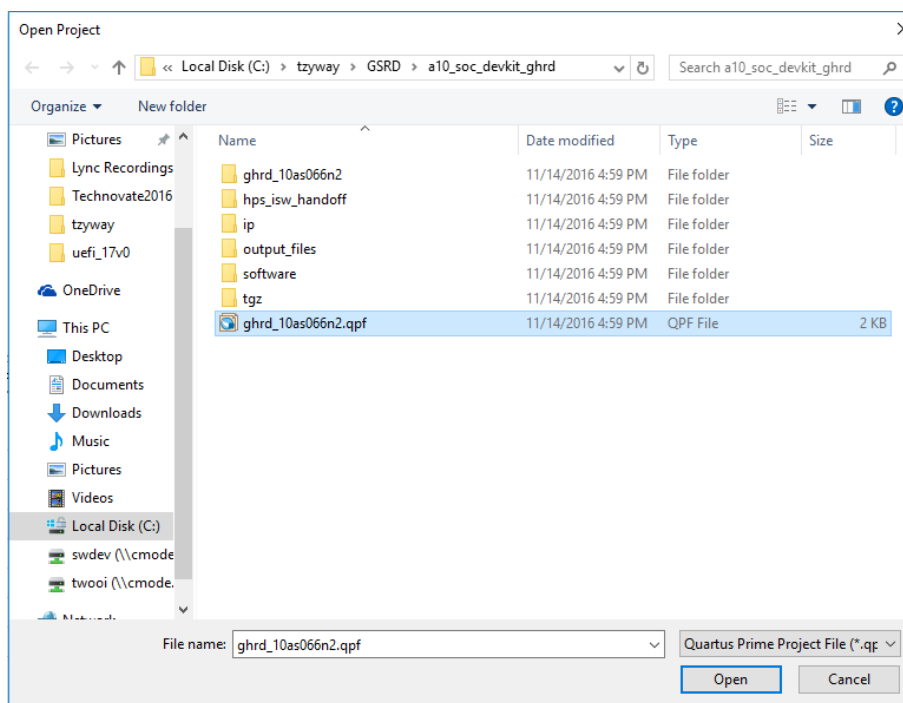
5. Open the hardware project in Intel Quartus Prime by selecting **File ► Open Project**.

Figure 5. Open Project File



6. Browse to the ~a10_soc_devkit_ghrd folder.
7. Select the ghrd_10as066n2.qpf project file and click **Open**.

Figure 6. Open Project File

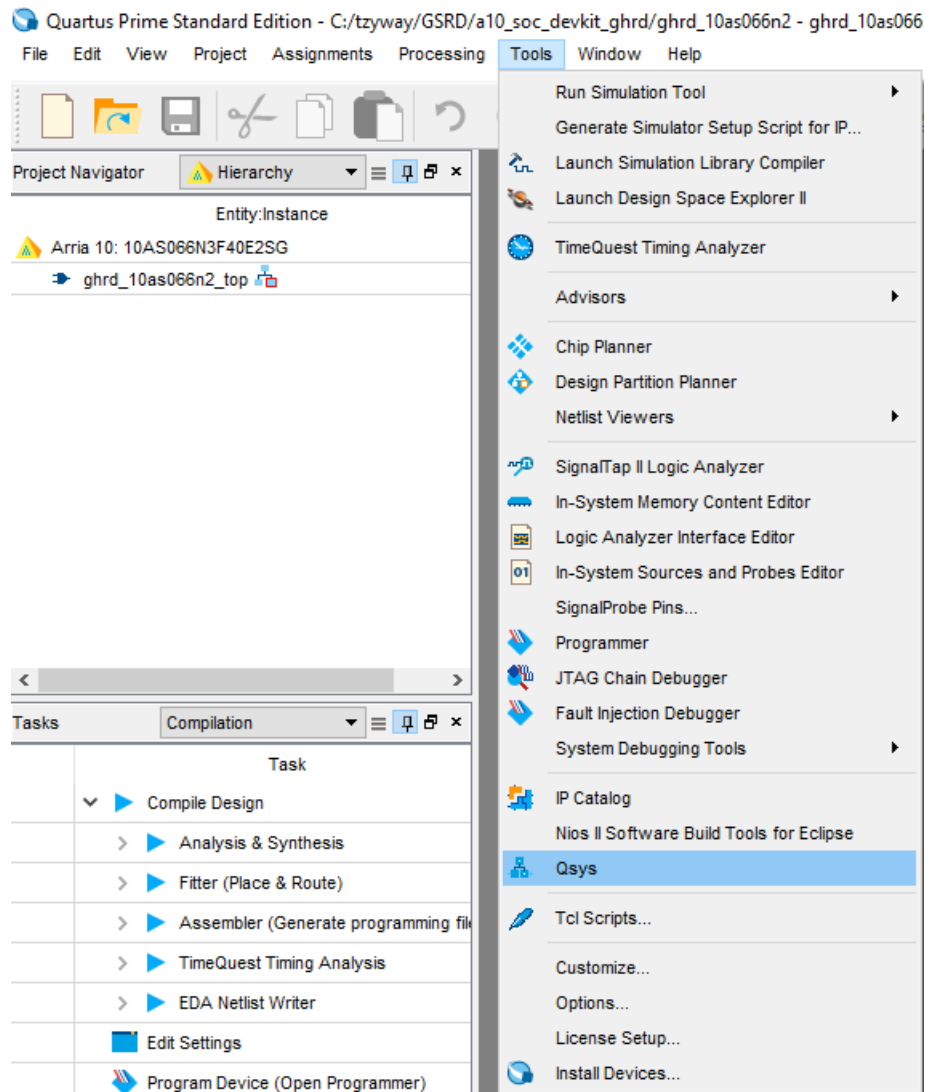


Intel Quartus Prime loads the project.

1.5.1.2. Generate the System in Platform Designer

1. In Intel Quartus Prime, select **Tools** ► **Qsys** from the menu bar to start the Platform Designer tool.

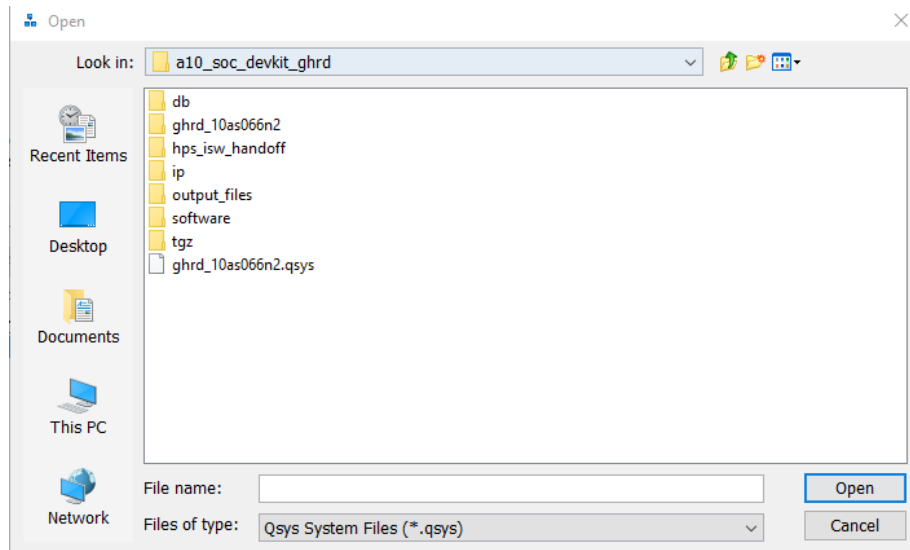
Figure 7. Opening the Platform Designer Tool



An **Open** file dialog box appears.

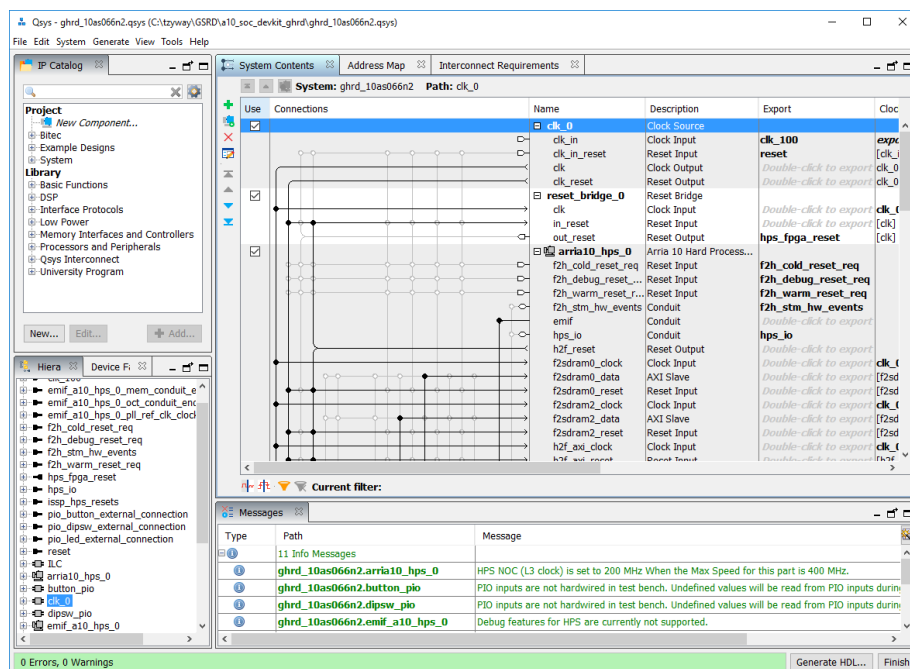
2. Select the `~/a10_soc_devkit_ghrd/ghrd_10as066n2.qsys` file and click **Open**.

Figure 8. Opening a Project from the Dialog Box



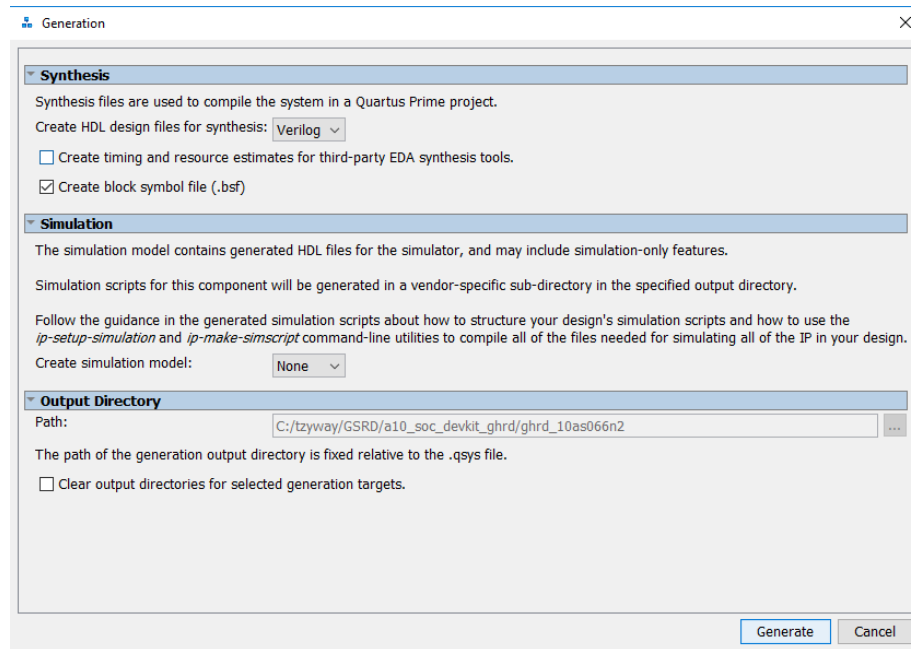
Platform Designer loads the project and displays the system.

Figure 9. Loading the Platform Designer Project



3. Click the **Generate HDL...** button on the bottom right corner of the window.
4. When the **Generation** dialog box appears, click the **Generate** button.

Figure 10. Generation Window



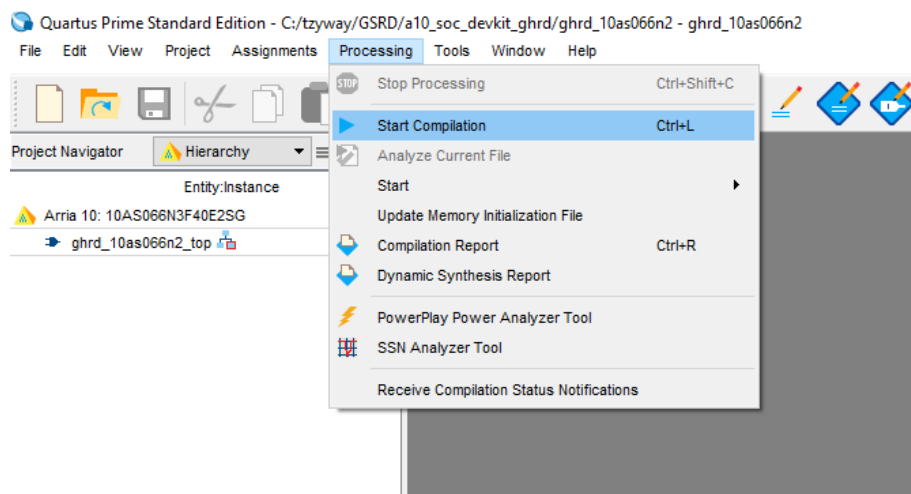
Platform Designer generates the system and a **Generate Completed** window appears.

5. Click **Close** to get back to the main Platform Designer window.
6. Click **Finish** in the lower right corner to close the Platform Designer window and return to the main Intel Quartus Prime window.

1.5.1.3. Compile the Design in Intel Quartus Prime

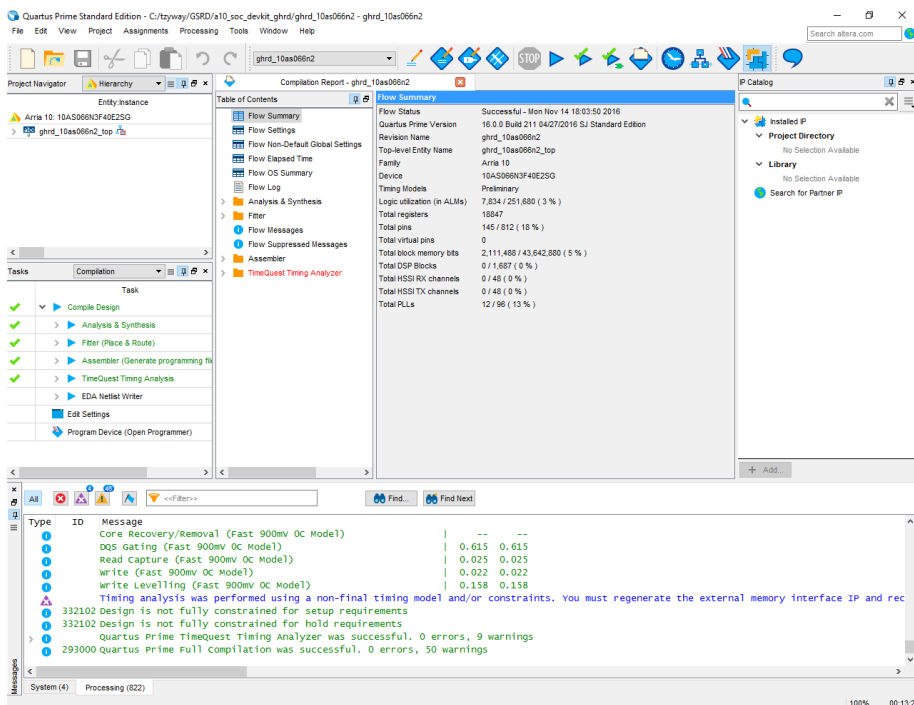
1. Compile the project in Intel Quartus Prime by selecting **Processing ► Start Compilation** from the main menu.

Figure 11. Starting the Design Compilation



On completion, Intel Quartus Prime displays a compilation status in the **Messages** window below.

Figure 12. Compilation Messages and Status Window



The compilation process generates these files:

Table 5. Compilation Output Files

File	Description
~/a10_soc_devkit_ghrd/output_files/ghrd_10as066n2.sof	FPGA configuration file
~/a10_soc_devkit_ghrd/hps_isw_handoff	Handoff folder containing XML files for generating the U-Boot device tree

1.5.1.4. Converting the Programming File

You must convert the SOF file produced by compiling the hardware design to the RBF format required by the UEFI boot loader. This section describes how to convert the SOF file into a single RBF file consisting of a combined CORE and PERIPH file. This section also describes how to convert the SOF file into two RBF files: a CORE file and separate a PERIPH file.

Note: You must generate two separate RBF files for SD card image generation. For QSPI image generation, you only need to generate the single RBF file that combines the CORE and PERIPH files.

1.5.1.4.1. Converting the SOF File into Two Split RBF Files

Splitting the SOF file creates two files:

- Peripheral file (PERIPH): Configures the I/O
 - Core file (CORE): configures the FPGA fabric
1. Open an SoC FPGA EDS command shell.
 2. Type the following command to convert the file:

```
$ quartus/bin/quartus_cpf -c --hps -o bitstream_compression=on
<your_working_directory>/a10_soc_devkit_ghrd/output_files/ghrd_10as066n2.sof
<your_working_directory>/a10_soc_devkit_ghrd/output_files/ghrd_10as066n2.rbf
```

Figure 13. SoC FPGA EDS File Conversion Command

```
/cydrive/c/tzyway/GSRD/a10_soc_devkit_ghrd/output_files
twooi@twooi-MOBL /cydrive/c/tzyway/GSRD/a10_soc_devkit_ghrd/output_files
$ quartus_cpf -c --hps -o bitstream_compression=on ./ghrd_10as066n2.sof ./ghrd_10as066n2.rbf
Info: *****
Info: Running Quartus Prime Convert programming file
Info: Version 16.0.0 Build 211 04/27/2016 SJ Standard Edition
Info: Copyright (C) 1991-2016 Altera Corporation. All rights reserved.
Info: Your use of Altera Corporation's design tools, logic functions
Info: and other software and tools, and its AMPP partner logic
Info: functions, and any output files from any of the foregoing
Info: (including device programming or simulation files), and any
Info: associated documentation or information are expressly subject
Info: to the terms and conditions of the Altera Program License
Info: Subscription Agreement, the Altera Quartus Prime License Agreement,
Info: the Altera MegaCore Function License Agreement, or other
Info: applicable license agreement, including, without limitation,
Info: that your use is for the sole purpose of programming logic
Info: devices manufactured by Altera and sold by Altera or its
Info: authorized distributors. Please refer to the applicable
Info: agreement for further details.
Info: Processing started: Tue Nov 15 01:21:13 2016
Info: Command: quartus_cpf -c --hps -o bitstream_compression=on ./ghrd_10as066n2.sof ./ghrd_10as066n2.rbf
Info: Quartus Prime Convert programming file was successful. 0 errors, 0 warnings
Info: Peak virtual memory: 1070 megabytes
Info: Processing ended: Tue Nov 15 01:21:28 2016
Info: Elapsed time: 00:00:15
Info: Total CPU time (on all processors): 00:00:15
twooi@twooi-MOBL /cydrive/c/tzyway/GSRD/a10_soc_devkit_ghrd/output_files
$
```

The command creates the following files in your *<your_working_directory>*/a10_soc_devkit_ghrd/output_files/ folder:

Table 6. Split RBF Files

File	Description
~ /a10_soc_devkit_ghrd/output_files/ghrd_10as066n2.periph.rbf	I/O configuration file
~ /a10_soc_devkit_ghrd/output_files/ghrd_10as066n2.core.rbf	FPGA fabric configuration file

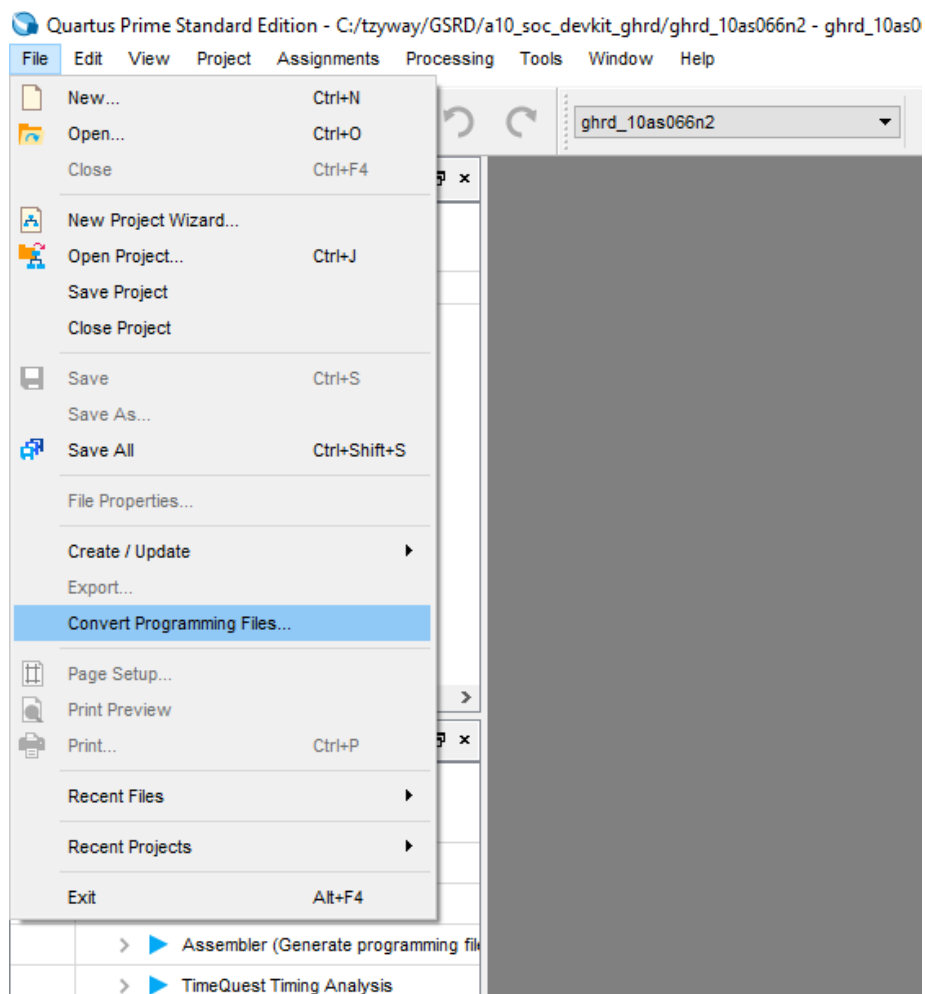
1.5.1.4.2. Converting the SOF File into a Single RBF File

Different methods exist to convert the SOF into a single RBF file. The following two sections describe how to use the Intel Quartus Prime Pro Edition Programmer command line tools or the GUI converter to convert the programming file.

1.5.1.4.3. Single RBF Conversion Using the GUI Converter

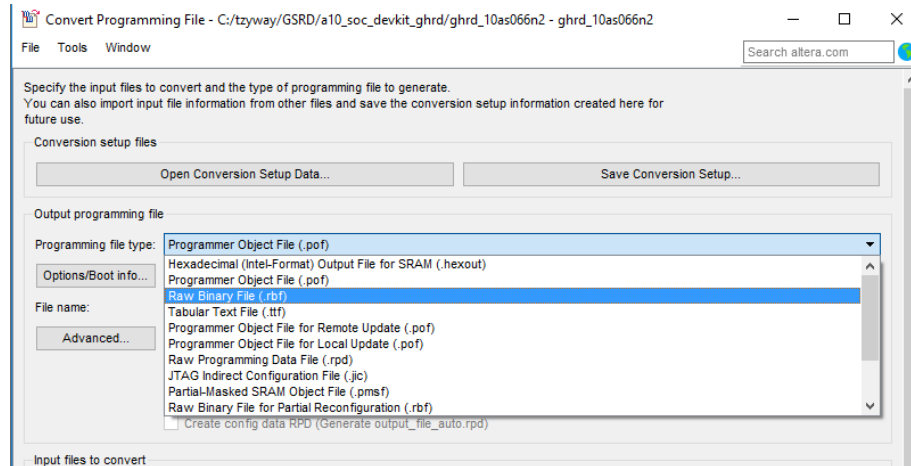
1. Launch Intel Quartus Prime Pro Edition.
2. From the Intel Quartus Prime Pro Edition menu, select **File ► Convert Programming Files....**

Figure 14. Convert Programming Files Selection



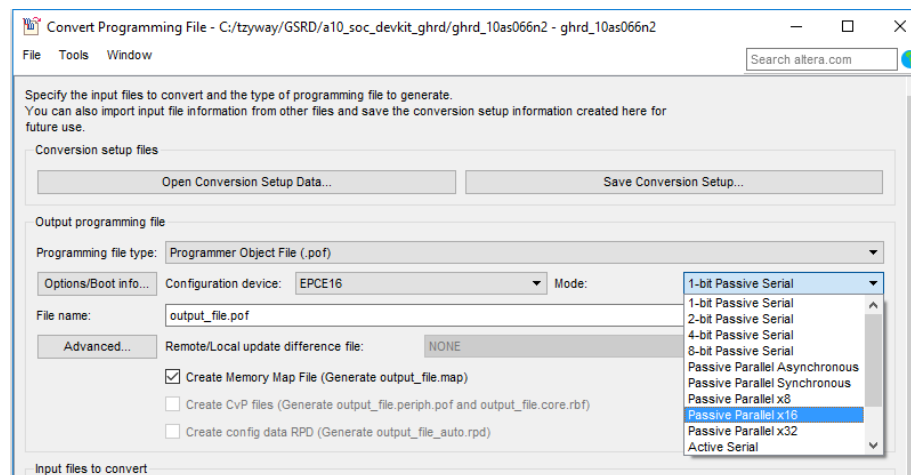
3. Select the **Programming file type** to be **Raw Binary File (.rbf)**.

Figure 15. Programming File Type Selection



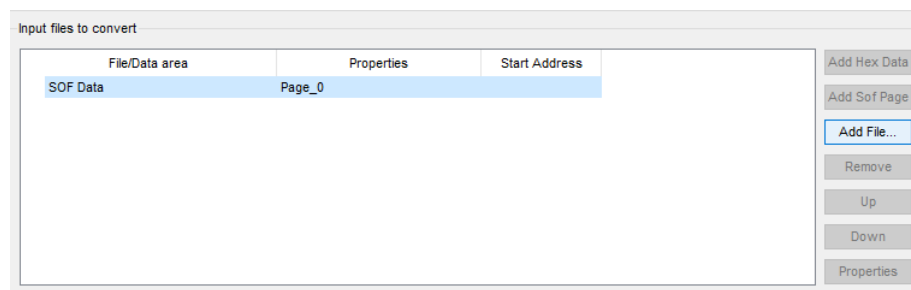
4. Select the **Mode** to be **Passive Parallel x8** or **Passive Parallel x16**.

Figure 16. Selecting Mode



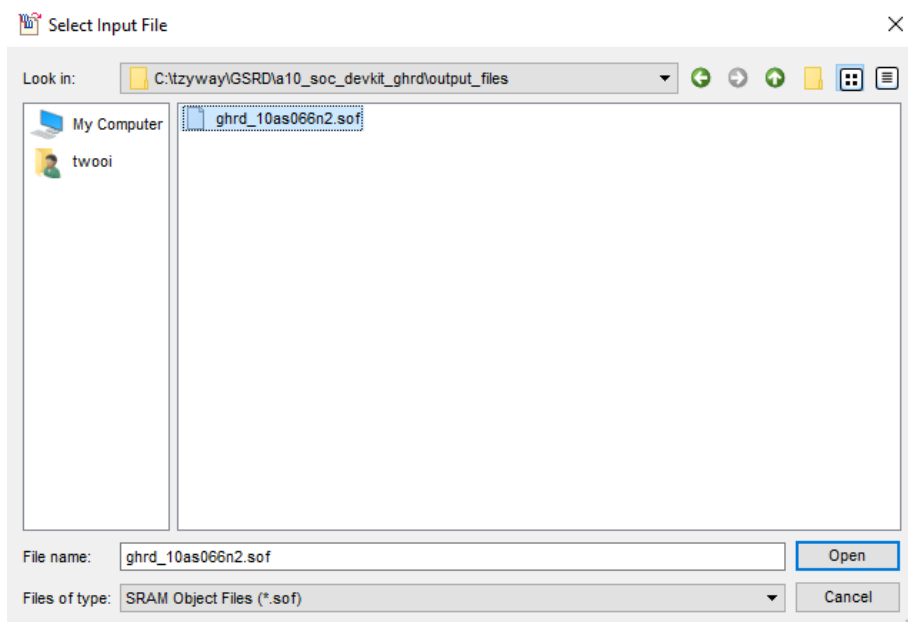
5. Click **SOF Data** and then click **Add File...**

Figure 17. Add SOF File



6. Browse to <your_working_directory>/a10_soc_devkit_ghrd/output_files/ghrd_10as066n2.sof file and click **Open**.

Figure 18. Open SOF File



7. In the Conversion setup files window type `ghrd_10as066n2.rbf` in the **File Name** prompt.
8. Click the **Close** button to exit the **Convert Programming File** window. These steps create a `ghrd_10as066n2.rbf` file in the `<your_working_directory>/a10_soc_devkit_ghrd/output_files/` folder.

1.5.1.4.4. Single RBF Conversion Using the Intel Quartus Prime Pro Edition Programmer Command Line Tools

1. Open an SoC FPGA EDS command shell.
2. Type the following command to convert the file:

```
$ quartus/bin/quartus_cpf -c -o bitstream_compression=on  
<your_working_directory>/a10_soc_devkit_ghrd/output_files/  
ghrd_10as066n2.sof  
<your_working_directory>/a10_soc_devkit_ghrd/output_files/ghrd_10as066n2.rbf
```

Figure 19. SoC FPGA EDS File Conversion Command

```

/cydrive/c/tzyway/GSRD/a10_soc_devkit_ghrd/output_files

twooi@twooi-MOBL /cydrive/c/tzyway/GSRD/a10_soc_devkit_ghrd/output_files
$ quartus_cpf -c -o bitstream_compression=on ./ghrd_10as066n2.sof ./ghrd_10as066n2.rbf
Info:
Info: *****
Info: Running Quartus Prime Convert_programming_file
Info: Version 16.0.0 Build 211 04/27/2016 SJ Standard Edition
Info: Copyright (C) 1991-2016 Altera Corporation. All rights reserved.
Info: Your use of Altera Corporation's design tools, logic functions
Info: and other software and tools, and its AMPP partner logic
Info: functions, and any output files from any of the foregoing
Info: (including device programming or simulation files), and any
Info: associated documentation or information are expressly subject
Info: to the terms and conditions of the Altera Program License
Info: Subscription Agreement, the Altera Quartus Prime License Agreement,
Info: the Altera MegaCore Function License Agreement, or other
Info: applicable license agreement, including, without limitation,
Info: that your use is for the sole purpose of programming logic
Info: devices manufactured by Altera and sold by Altera or its
Info: authorized distributors. Please refer to the applicable
Info: agreement for further details.
Info: Processing started: Tue Nov 15 01:35:26 2016
Info: Command: quartus_cpf -c -o bitstream_compression=on ./ghrd_10as066n2.sof ./ghrd_10as066n2.rbf
Info: Quartus Prime Convert_programming_file was successful. 0 errors, 0 warnings
Info: Peak virtual memory: 1056 megabytes
Info: Processing ended: Tue Nov 15 01:35:42 2016
Info: Elapsed time: 00:00:16
Info: Total CPU time (on all processors): 00:00:16

```

The command creates the following file: *<your_working_directory>/a10_soc_devkit_ghrd/output_files/ghrd_10as066n2.rbf*.

1.5.2. Generating the Boot Loader and Device Tree for UEFI Boot Loader

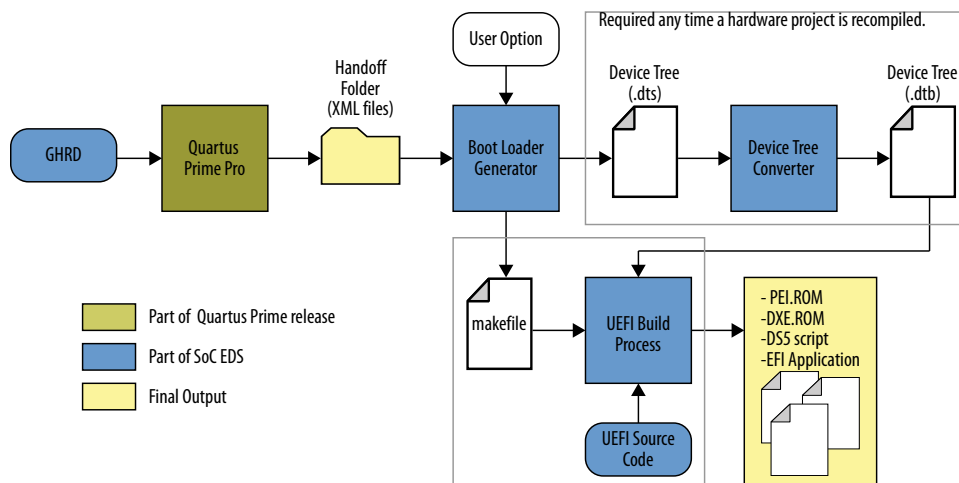
Task Time: 30 minutes

In this section, you generate a device tree in a Flattened Device Tree (FDT) binary format, DTB file. The UEFI build process uses this file to develop the hardware configuration.

Note: The boot loader and the kernel require different device trees for the Intel Arria 10 SoC. This section refers to the boot loader device tree.

The following figure represents an overview of the boot loader flow.

Figure 20. Boot Loader Flow



1.5.2.1. Generating the Boot Loader DTS File for 16.0 Release and Below

This section describes how to run the Boot Loader Generator (BSP Editor) in the Windows and Linux environments to generate the device tree.

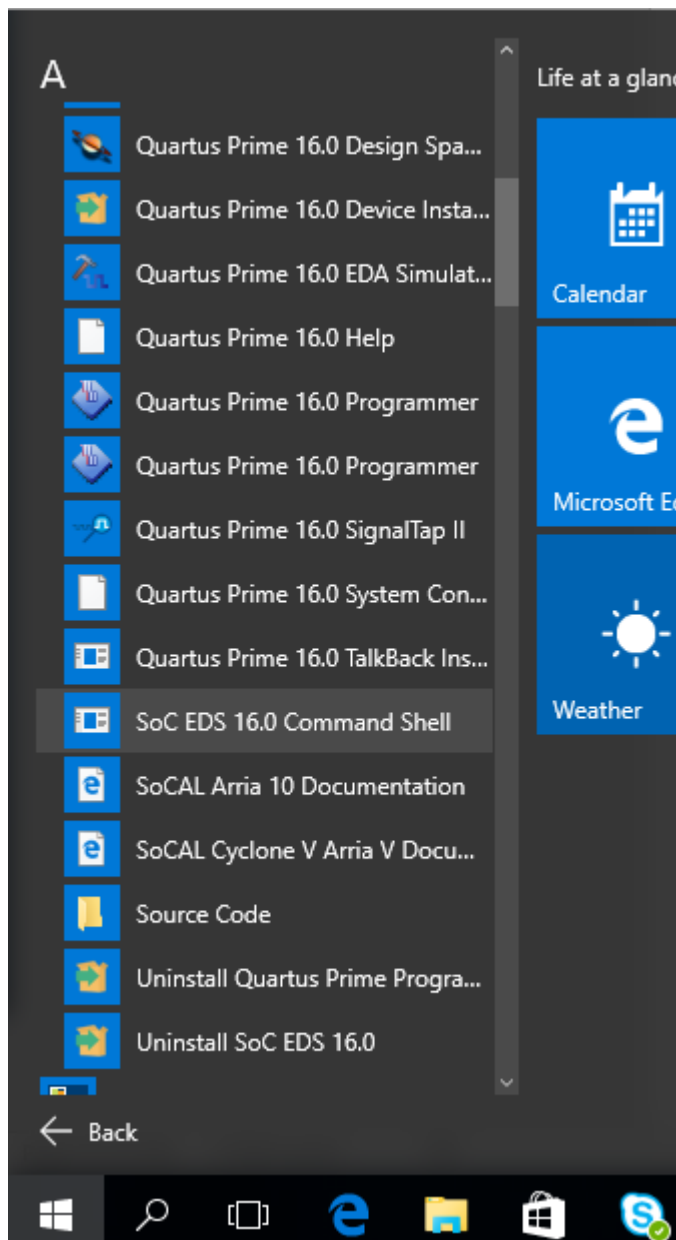
A device tree is a data structure that describes the underlying hardware to a firmware or an operating system. The UEFI boot loader uses the device tree blob (DTB) to handle variations of hardware configuration without having to modify a single line of source code. This flexibility is particularly important for highly configurable Intel SoC FPGAs.

The device tree generator tool is a part of the SoC FPGA EDS and creates the device tree for SoC systems that contain FPGA designs created using Platform Designer. The generated device tree describes the HPS peripherals, selected FPGA soft IP and peripherals that are configuration dependent.

1.5.2.1.1. Windows Environment

1. Start an embedded command shell.

Figure 21. Open an Embedded Command Shell



2. Go to the a10_soc_devkit_ghrd folder.

```
$ cd <Working_Directory>/a10_soc_devkit_ghrd/
```

3. Start the BSP Editor.

```
$ bsp-editor&
```

4. In the BSP Editor window, select **File ► New HPS BSP...** to open a new BSP window.
5. In the new BSP window:

- a. Click the browse (...) button next to the **Preloader Setting Directory** text box and browse to the `<your_working_directory>/a10_soc_devkit_ghrd/hps_isw_handoff` folder.
- b. Select the **Operating system** to be **U-Boot Bootloader (Arria 10 HPS)** in the pull-down menu.
- c. Leave the **Use default locations** enabled to create the new BSP in the `<your_working_directory>/a10_soc_devkit_ghrd/software/uboot_bsp` folder.
- d. Click **OK** to create the BSP settings file and close the New BSP window.

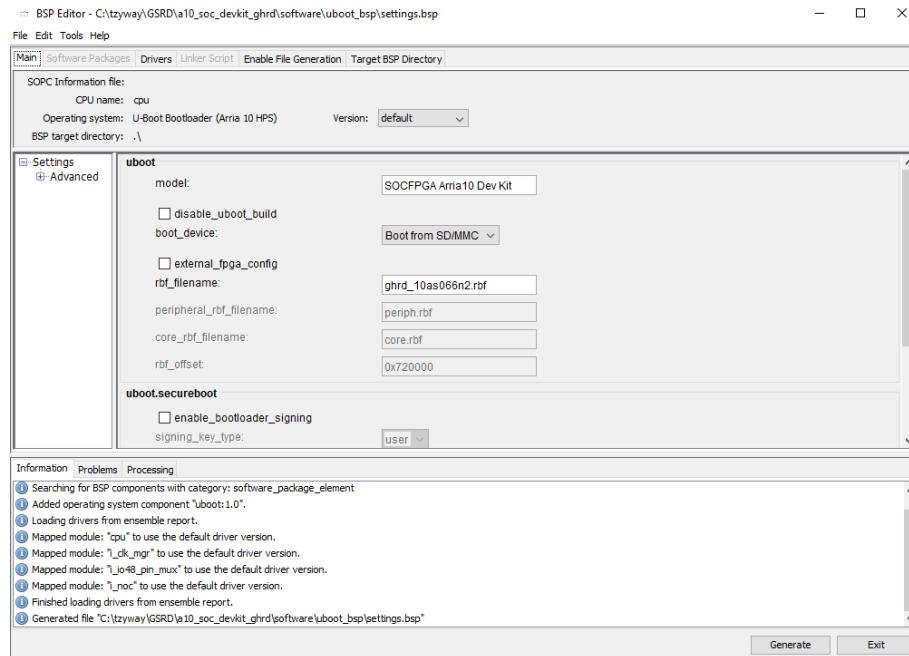
Figure 22. New BSP Window Settings

The screenshot shows the 'New BSP' dialog box with the following settings:

- Hardware**
 - Preloader settings directory: C:\tzyway\GSRD\A10_soc_devkit_ghrd\hps_isw_handoff
- Software**
 - Operating system: U-Boot Bootloader (Arria 10 HPS)
 - Version: default
 - ☒ Use default locations
 - BSP target directory: C:\tzyway\GSRD\A10_soc_devkit_ghrd\software\uboot_bsp
 - BSP Settings File name: C:\tzyway\GSRD\A10_soc_devkit_ghrd\software\uboot_bsp\settings.bsp
 - ☒ Enable Settings File relative paths
 - ☐ Enable Additional Td script
 - Additional Td script: (empty)

6. In the BSP Editor window, edit the BSP settings as follows:
 - boot_device: **Boot from SD/MMC** (default)
 - model: **SOCFPGA Arria 10 Dev Kit** (default)
 - rbf_filename: **ghrd_10as066n2.rbf** (default was socfpga.rbf)
 - peripheral_rbf_filename: **periph.rbf** (default)
 - core_rbf_filename: **core.rbf** (default)
 - disable_uboot_build: **unchecked** (default)

Figure 23. BSP Settings



7. In the BSP Editor window, click **Generate** in the lower right hand corner to produce the source code and the U-Boot device tree.
8. When the file generation is complete, click **Close** in the **BSP Editor** window. The following files are generated in the `<your_working_directory>/a10_soc_devkit_ghrd/software/uboot_bsp` folder:
 - config.mk
 - devicetree.dts
 - Makefile
 - settings.bsp
 - uboot.ds

1.5.2.1.2. Linux Environment

1. Start an embedded command shell and go to the `a10_soc_devkit_ghrd` folder.

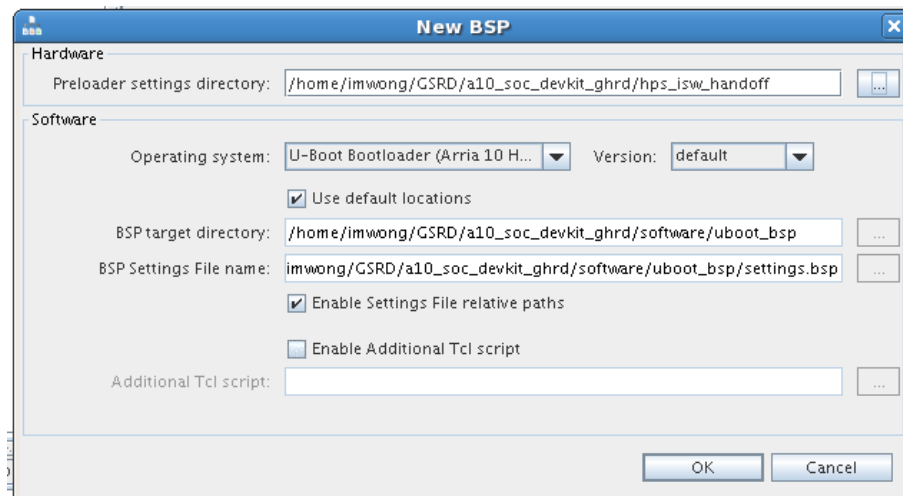
```
$ ~/altera/16.0/embedded/embedded_command_shell.sh
$ cd <Working_Directory>/a10_soc_devkit_ghrd/
```

2. Start the BSP Editor.

```
$ bsp-editor&
```
3. In the BSP Editor window, select **File ► New HPS BSP...** to open a New BSP window.
4. In the new BSP window:
 - a. Click the browse (...) button next to the **Preloader Setting Directory** text box and browse to the `<Working_Directory>/a10_soc_devkit_ghrd/hps_isw_handoff` folder.

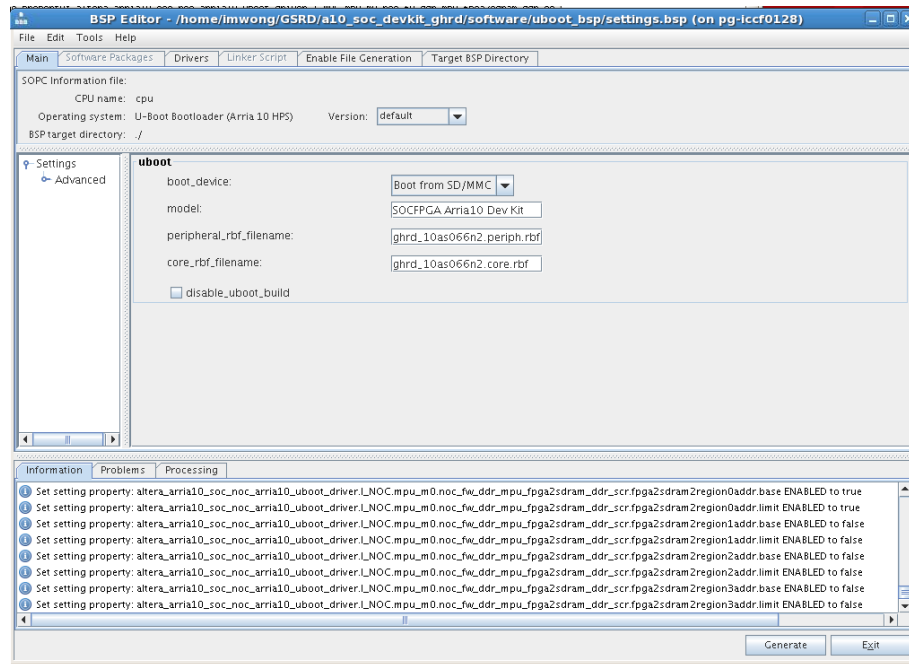
- b. Select the **Operating system** to be **U-Boot Bootloader (Arria 10 HPS)** in the pull-down menu.
- c. Leave the **Use default locations** enabled to create the new BSP in the `<your_working_directory>/a10_soc_devkit_ghrd/software/uboot_bsp` folder.
- d. Click **OK** to create the BSP settings file and close the New BSP window.

Figure 24. New BSP Window Settings



5. In the BSP Editor window, edit the BSP settings as follows:
 - boot_device: **Boot from SD/MMC** (default)
 - model: **SOCFPGA Arria 10 Dev Kit** (default)
 - peripheral_rbf_filename: **ghrd_10as066n2.periph.rbf** (default is periph.rbf)
 - core_rbf_filename: **ghrd_10as066n2.core.rbf** (default is core.rbf)
 - disable_uboot_build: **unchecked** (default)

Figure 25. BSP Settings



6. In the BSP Editor window, click **Generate** in the lower right hand corner to produce the source code and the U-Boot device tree.
7. When the file generation is complete, click **Close** in the BSP Editor window. The following files are generated in the `<your_working_directory>/a10_soc_devkit_ghrd/software/uboot_bsp` folder:
 - config.mk
 - devicetree.dts
 - Makefile
 - settings.bsp
 - uboot.ds

1.5.2.2. Generating a Boot Loader DTB File for the 16.0 Release and Below

This task describes how to compile the U-Boot device tree source (DTS) into the device tree binary (DTB).

1. In the command shell, go to the U-boot folder and type the following commands:

```
$ cd <your_working_directory>/a10_soc_devkit_ghrd/software/uboot_bsp
$ dtc -O dtb -o devicetree.dtb -I dts devicetree.dts
```

These commands create the devicetree.dtb file in `<your_working_directory>/a10_soc_devkit_ghrd/software/uboot_bsp` folder.

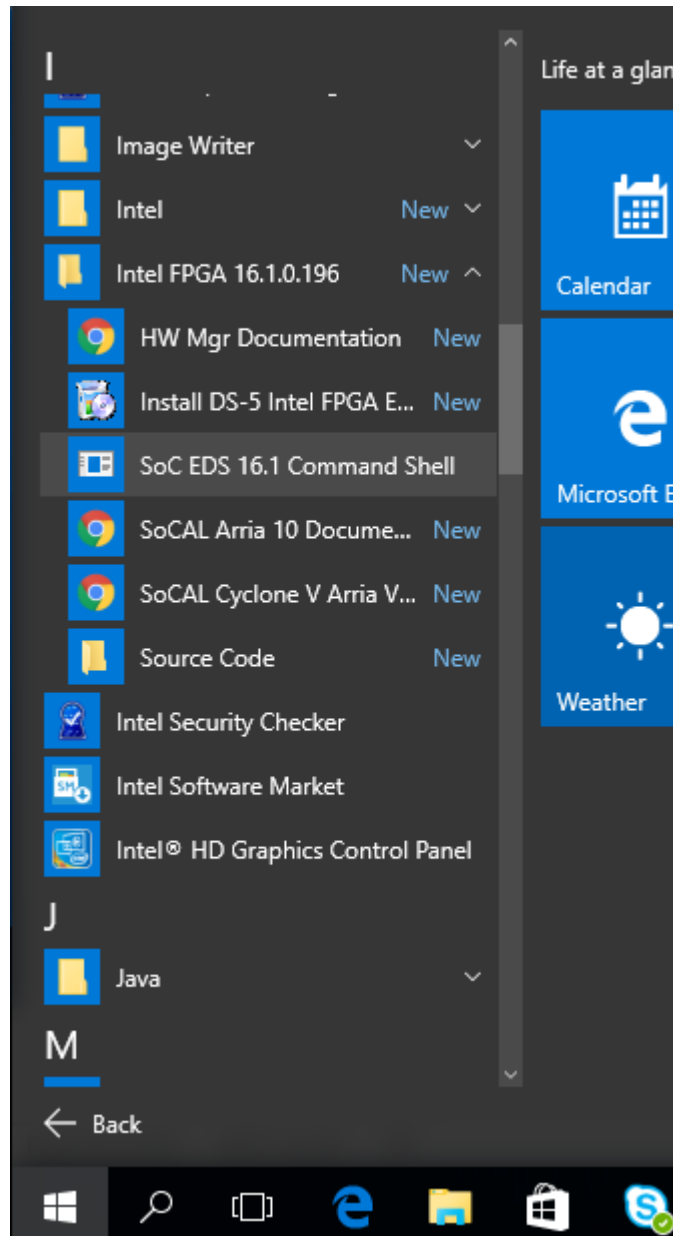
1.5.2.3. Generating the Boot Loader DTS File for the 16.1 Release

This section describes how to run the Boot Loader Generator (BSP Editor) in the Windows and Linux environments to generate the device tree using the Intel Quartus Prime Pro Edition 16.1 release.

1.5.2.3.1. Windows Environment

1. Start an embedded command shell.

Figure 26. Open an Embedded Command Shell



2. Go to the a10_soc_devkit_ghrd folder.

```
$ cd <your_working_directory>/a10_soc_devkit_ghrd/
```

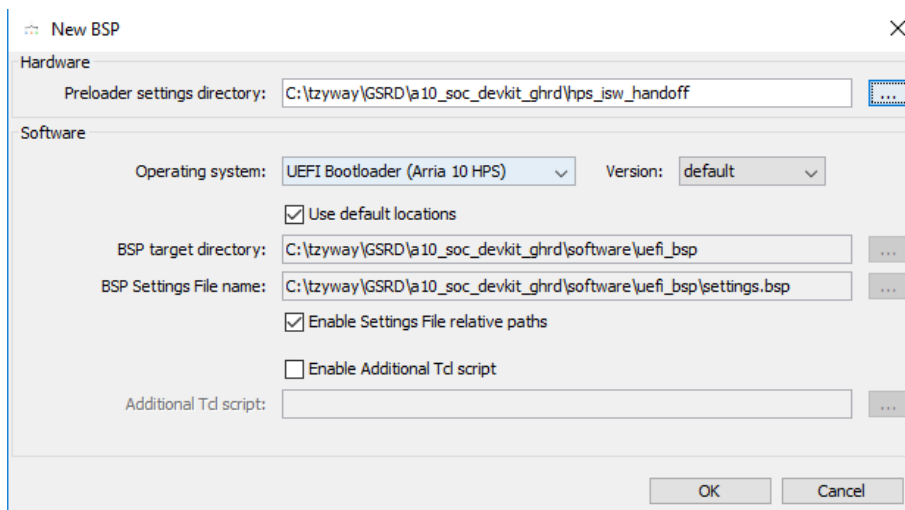
3. Start the BSP Editor.

```
$ bsp-editor&
```

4. In the BSP Editor window, select **File ► New HPS BSP...** to open a new BSP window.
5. In the new BSP window:

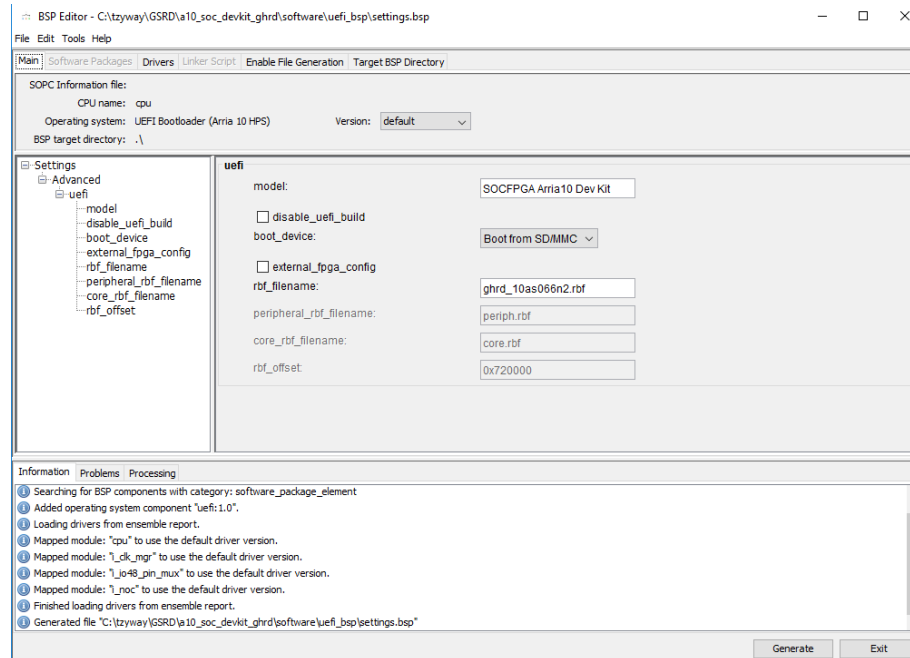
- a. Click the browse (...) button next to the **Preloader Setting Directory** text box and browse to the `<your_working_directory>/a10_soc_devkit_ghrd/hps_isw_handoff` folder. Click **Open** to return to the new BSP window.
- b. Select the **Operating system** to be **UEFI Bootloader (Arria 10 HPS)** in the pull-down menu.
- c. Leave the **Use default locations** enabled to create the new BSP in the `<Working_Directory>/a10_soc_devkit_ghrd/software/uefi_bsp` folder.
- d. Click **OK** to create the BSP settings file and close the New BSP window.

Figure 27. New BSP Window Settings



6. In the BSP Editor window, edit the BSP settings as follows:
 - boot_device: **Boot from SD/MMC** (default)
 - model: **SOCFPGA Arria 10 Dev Kit** (default)
 - disable_uefi_build: **unchecked** (default)
 - rbf_filename: **ghrd_10as066n2.rbf** (default was socfpga.rbf)
 - peripheral_rbf_filename: **periph.rbf** (default)
 - core_rbf_filename: **core.rbf** (default)
 - rbf_offset: **0x720000** (default)

Figure 28. BSP Settings



7. In the BSP Editor window, click **Generate** in the lower right hand corner to produce the source code and the UEFI device tree.
8. When the file generation is complete, click **Close** in the BSP Editor window. The following files are generated in the `<your_working_directory>/a10_soc_devkit_ghrd/software/uboot_bsp` folder:
 - config.mk
 - devicetree.dts
 - Makefile
 - settings.bsp

1.5.2.3.2. Linux Environment

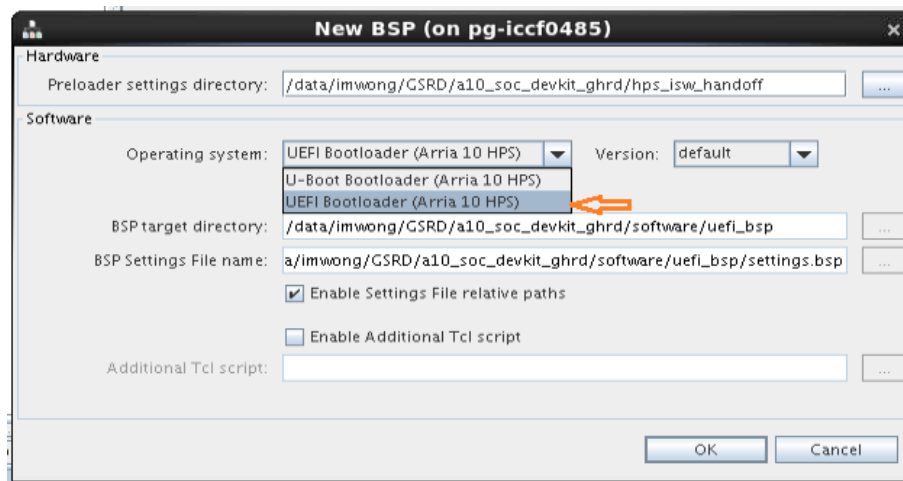
1. Start an embedded command shell and go to the `a10_soc_devkit_ghrd` folder.


```
$ ~/altera/16.1/embedded/embedded_command_shell.sh
$ cd <your_working_directory>/a10_soc_devkit_ghrd/
```
2. Start the BSP Editor.


```
$ bsp-editor&
```
3. In the BSP Editor window, select **File ► New HPS BSP...** to open a New BSP window.
4. In the new BSP window:
 - a. Click the browse (...) button next to the **Preloader Setting Directory** text box and browse to the `<your_working_directory>/a10_soc_devkit_ghrd/hps_isw_handoff` folder. Click **Open** to return to the new BSP window.

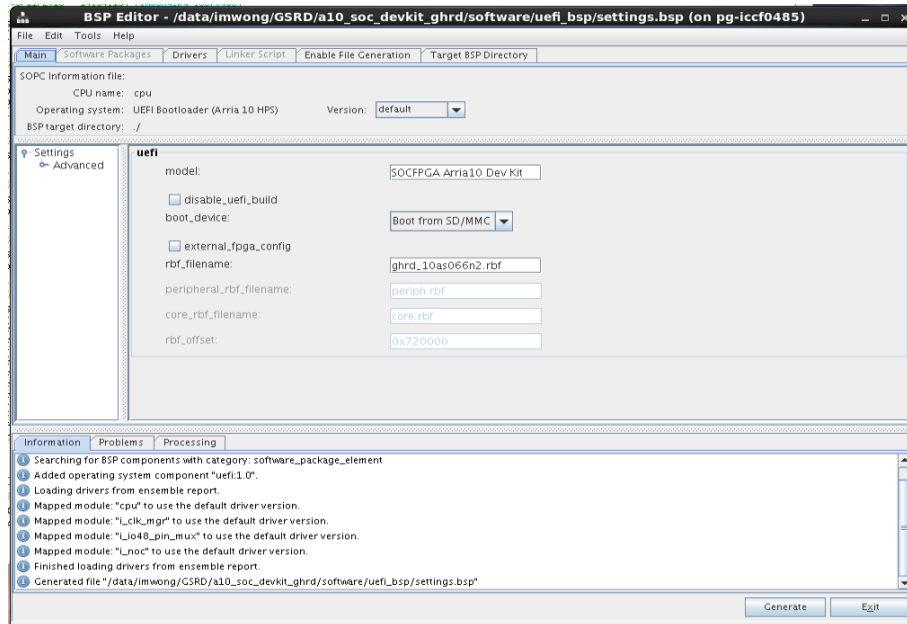
- b. Select the **Operating system** to be **UEFI Bootloader (Arria 10 HPS)** in the pull-down menu.
- c. Leave the **Use default locations** enabled to create the new BSP in the `<your_working_directory>/a10_soc_devkit_ghrd/software/uefi_bsp` folder.
- d. Click **OK** to create the BSP settings file and close the New BSP window.

Figure 29. New BSP Window Settings



5. In the BSP Editor window, edit the BSP settings as follows:
 - boot_device: **Boot from SD/MMC** (default)
 - model: **SOCFPGA Arria 10 Dev Kit** (default)
 - rbf_filename: **ghrd_10as066n2.rbf** (default was socfpga.rbf)
 - peripheral_rbf_filename: **periph.rbf** (default)
 - core_rbf_filename: **core.rbf** (default)
 - disable_uboot_build: **0x720000** (default)

Figure 30. BSP Editor Settings



6. In the BSP Editor window, click **Generate** in the lower right hand corner to produce the source code and the UEFI device tree.
7. When the file generation is complete, click **Close** in the BSP Editor window. The following files are generated in the `<Working_Directory>/a10_soc_devkit_ghrd/software/uefi_bsp` folder:
 - config.mk
 - devicetree.dts
 - Makefile
 - settings.bsp

1.5.2.4. Generating a Boot Loader DTB File for the 16.1 Release

This task describes how to compile the UEFI device tree source (DTS) into the device tree binary (DTB).

1. In the command shell, go to the UEFI folder and type the following commands:

```
$ cd <your_working_directory>/a10_soc_devkit_ghrd/software/uefi_bsp
$ dtc -O dtb -o devicetree.dtb -I dts devicetree.dts
```

These commands create the devicetree.dtb file in `<your_working_directory>/a10_soc_devkit_ghrd/software/uefi_bsp` folder.

1.5.3. Building the UEFI Boot Loader

To build a UEFI boot loader, you must obtain the UEFI source code and compile the UEFI source with the supported toolchain.

You can compile the UEFI source code either in a Windows or in a Linux system.

1.5.3.1. Prerequisites

For Windows System

If you are using a Windows system, you must install the following components on your machine:

- Git (Download from [Git](#))
- SoC FPGA EDS: this design suite come with a python tool
- Arm Development Studio 5 Intel SoC FPGA Edition for Windows

For Linux System

If you are using a Linux system, you must install packages. Depending on your Linux distribution, the command to install the packages is different:

If you are using a Ubuntu distribution, type:

```
$ sudo apt-get install uuid-dev build-essential
```

If you using a Fedora distribution, type:

```
$ sudo yum install uuid-devel libuuid-devel
```

For building UEFI, the Python package is required. If your host does not have Python installed, you can obtain the package from the SoC FPGA EDS installation path, by typing:

```
export PATH=$SOCEDS_DEST_ROOT/host_tools/python/bin/:$PATH
```

Note:

If you receive an error on `hashlib.py`, ensure that you have `libssl.so.1.0.0` installed on your machine. You can install by typing:

```
$ export LD_LIBRARY_PATH=$SOCEDS_DEST_ROOT/host_tools/python/lib/:$LD_LIBRARY_PATH  
$ /sbin/ldconfig
```

1.5.3.2. Supported Compiler Toolchain

The supported UEFI compiler toolchain is:

- Linaro*:
 - arm-linux-gnueabi-hf-gcc (crosstool-NG linaro-1.13.1-4.8-2014.04 – Linaro GCC 4.8-2014.04) 4.8.3 20140401 (pre-release))
 - This toolchain is available within the SoC FPGA EDS installation package.
- Arm Toolchain (armcc, armlink)
 - This tool chain is available within the SoC FPGA EDS Arm DS-5 Intel SoC FPGA Edition installation package.

1.5.3.3. Obtaining the UEFI Source Code

The UEFI source code is located in [GitHub](#). The following steps show you how to get the UEFI source code.

1. Open a terminal.
2. Create a new directory path to check out the UEFI source code from the GitHub.

```
$ mkdir /data/<username>/<Path_to_UEFI_Source_Code>
```

3. Change your directory to this UEFI working directory and clone the UEFI source from the git trees.

```
$ cd /data/<username>/<Path_to_UEFI_Source_Code>
```

```
$ git clone https://github.com/altera-opensource/uefi-socfpga
```

4. When completed, change to the uefi-socfpga folder and perform a git checkout.

```
$ cd uefi-socfpga
```

```
$ git checkout -b socvp_socfpga_udk2015 - track origin/socvp_socfpga_udk2015
```

5. Type the following command to generate a hard reset and make git point to the respective git tag for your board:

- RevC1 board:

```
$ git reset --hard tags/rel_socfpga_arria10_soceds_16.0
```

- Rev B board:

```
$ git reset --hard tags/rel_socfpga_arria10_soceds_15.1.1
```

- Rev A.1 or RevA.2 board:

```
$ git reset --hard tags/rel_socfpga_arria10_soceds_15.0.1
```

1.5.3.4. Preparing the Handoff DTB

If you are using an Intel SoC FPGA development kit and the standard GHRD, you can find the handoff device tree blob (DTB) installed as part of SoC FPGA EDS. The exact location of this file is `<SoCEDs_installation_directory>/examples/hardware/a10_soc_devkit_ghrd/software/boot_loader/devicetree.dtb`.

If you are using a custom board or creating your own design, you must run **make** in your U-boot folder to build your customize boot loader and the U-boot handoff device tree blob (DTB). Refer to the sub-sections in *Generating a Boot Loader DTB file* for instructions.

Once the compilation is successful, you can find the `devicetree.dtb` file in the `<your_working_directory>/a10_soc_devkit_ghrd/software/u-boot_bsp` folder.

Replace your `<your_uefi_directory>/uefi-socfpga/AlteraPlatformPkg/Arria10SoCPkg/Arria10SoCPkg.dtb` file with the newly generated `<your_working_directory>/a10_soc_devkit_ghrd/software/u-boot_bsp/devicetree.dtb` file. Alternatively, you can add the **HANDOFF_DTB=<dtb_file_path>** parameter to the **make** command later as mentioned in the *Compile UEFI* section.

```
$ make HANDOFF_DTB=<your_working_directory>/a10_soc_devkit_ghrd/software/u-boot_bsp/devicetree.dtb
```

Related Information

- [Generating a Boot Loader DTB File for the 16.0 Release and Below](#) on page 26
- [Generating a Boot Loader DTB File for the 16.1 Release](#) on page 32

1.5.3.4.1. Updating the Device Tree Blob

A typical device tree contains all or some of the following:

- Number of CPUs
- Size and location of various RAMs
- Buses and Bridges
- Peripheral device connections
- Interrupt controllers and IRQ line connections
- Specific device driver configurations, such as:
 - Ethernet MAC address
 - Peripheral input clock

If any of these configurations change in your design, you must update your DTB. Some examples of a change might be:

- Changing your boot flash daughter card type
- Clock changes
- Pin changes
- Firewall and bridge configuration changes

1.5.3.5. Compiling the UEFI

This section describes how to compile the UEFI source code with the Linaro toolchain and Arm toolchain in Windows and Linux environments.

Note: Reference to the make command is made in the following sections. To learn more about the make command, type:

```
$ make help
```

This command displays information on how to use **make** as well as options and examples.

1.5.3.5.1. Compiling the UEFI in a Windows Environment

This task describes how to compile the UEFI with the Linaro GCC compiler.

1. Open the command prompt.
2. Change your directory to your working directory.

```
$ cd <your_working_directory>/uefi-socfpga
```

3. Set the **SOCEDS_DEST_ROOT** and the **DS5_ROOT** according to the location of the SoC FPGA EDS and Arm DS-5 Intel SoC FPGA Edition tools, respectively.
4. Configure the **RVCT_TOOLS_PATH** correctly in the setup.bat file. For Intel Quartus Prime Pro Edition v16.1, set RVCT_TOOLS_PATH=%DS5_ROOT%\sw\ARMCompiler5.06u3\bin\ in the setup.bat file. In the previous setup file it is set to RVCT_TOOLS_PATH=%DS5_ROOT%\sw\ARMCompiler5.06u2\bin\ in

setup.bat file. However, no ARMCompiler5.06u2 exists in the folder. Only the ARMCompiler5.06u3, ARMCompiler6.5 and ARMCompiler49Addon exist in the filepath. Therefore, you must correct this parameter path assignment. If you do not correct the path, the following reporting errors are generated:

```
Cannot find armcc.exe
Please install DS-5
```

5. At the command prompt type: \$ setup.
6. Build the UEFI:

```
$ make clean
$ make
```

Note: When you build the UEFI, ensure your path to the UEFI source code is not too long. You may encounter some errors when the path is too long in the Windows environment. If you see an error similar to the screenshot below, shorten your path to the UEFI source code and recompile the source code again to remove the error.

Figure 31. UEFI Build Errors from Lengthy Path Name

```
Command Prompt
RELEASE_GCC48\ARM\AlteraPlatformPkg\Library\AlteraPeePostDdrMemoryAllocationLib\AlteraPeePostDdrMemoryAllocationLib\OUTP
UT\AlteraPeePostDdrMemoryAllocationLib.lib] Error 1
make[2]: Leaving directory 'C:/user/jianghoi/Desktop/uefi/16_0/ai0_soc_dev_kit_ghrd/software/uefi_bsp/uefi-socfpga/Build
/Arria10SoCPkg/RELEASE_GCC48/ARM/AlteraPlatformPkg/Library/AlteraPeePostDdrMemoryAllocationLib/AlteraPeePostDdrMemoryAll
ocationLib'

build...
: error 7000: Failed to execute command
C:/user/jianghoi/Desktop/UEFI/16_0/ai0_soc_dev_kit_ghrd/software/uefi_bsp/uefi-socfpga\tools\gcc493-x86\bin\mak
e_tbuild [C:/user/jianghoi/Desktop/uefi/16_0/ai0_soc_dev_kit_ghrd/software/uefi_bsp/uefi-socfpga\Build\Arria10SoCPkg\REL
EASE_GCC48\ARM\AlteraPlatformPkg\Library\AlteraPeePostDdrMemoryAllocationLib\AlteraPeePostDdrMemoryAllocationLib]

build...
: error F002: Failed to build module
C:/user/jianghoi/Desktop/uefi/16_0/ai0_soc_dev_kit_ghrd/software/uefi_bsp/uefi-socfpga\AlteraPlatformPkg\Library
\AlteraPeePostDdrMemoryAllocationLib\AlteraPeePostDdrMemoryAllocationLib.inf [ARM, GCC48, RELEASE]

- Failed -
Build end time: 10:16:55, Nov.30 2015
Build total time: 00:00:50

ArmPlatformPkg\Scripts\Makefile:56: recipe for target 'all' failed
make[1]: *** [all] Error 1
make[1]: Leaving directory 'C:/user/jianghoi/Desktop/UEFI/16_0/ai0_soc_dev_kit_ghrd/software/uefi_bsp/uefi-socfpga'
Makefile:432: recipe for target 'build_firmware' failed
make: *** [build_firmware] Error 2

C:/user/jianghoi/Desktop/UEFI/16_0/ai0_soc_dev_kit_ghrd/software/uefi_bsp/uefi-socfpga>
```

1.5.3.5.2. Compiling the UEFI in the Linux Environment

1. Open a terminal window and enter the following command:

```
$ cd <your_uefi_directory>/uefi-socfpga
$ make clean
```

Note: The **make clean** command deletes the entire <your_uefi_directory>/uefi-socfpga/Build/ folder and also cleans the BaseTools folder.

2. Compile the UEFI boot loader for the by typing the following command:

```
$ make
```


The build process takes less than three minutes.

Note: Typing the make command is equivalent to typing: `make DEVICE=a10 COMPILER=gcc`.

- When the build is successful, you see a **Build Done** message.

1.5.3.5.3. Compiling the UEFI with the ARMCC Compiler

To compile the UEFI with the ARMCC compiler open a terminal window and enter the following commands:

```
$ cd <your_uefi_directory>/uefi-socfpga
$ make clean
$ make DEVICE=a10 COMPILER=armcc HANDOFF_DTB=a10_soc_devkit_ghrd_qspi/software/
boot loader/devicetree.dtb
```

1.5.3.6. Generated Files from UEFI Compile

Compiling the UEFI source creates four files in the `<your_uefi_directory>/uefi-socfpga/Build/` folder:

Table 7. Generated Files from UEFI Compile

File	Description
<code>~ /uefi-socfpga/Build/PEI.256 (256 KB)</code>	- This file is the main UEFI boot loader and contains the device tree for the HPS. The code initializes the HPS, which can boot directly to baremetal application, RTOS or Linux. You can extend the UEFI boot phase to boot to the Driver Execution Environment ROM (DXE.ROM) and configure the UEFI network and shell support. - The .256 in the file name indicates the on-chip RAM size is 256 KB. Because the file size is 256 KB, it loads directly into the HPS on-chip using Arm DS-5 Intel SoC FPGA Edition Development Studio. - This file generates the Pre EFI Initialization ROM (PEI.ROM) file. The PEI.ROM file contains four duplicate images of the UEFI boot loader
<code>~ /uefi-socfpga/Build/PEI.ROM (1MB = 256KB X 4)</code>	This file is programmed onto the flash daughter card. The size of this file is four times bigger because the boot ROM can support up to four backup images. For example, if the first image (256KB) is corrupted, the boot ROM loads the second image and so on.
<code>~ /uefi-socfpga/Build/load_uefi_fw.ds</code>	This is the Arm DS-5 Intel SoC FPGA Edition script template. It is imported to the Arm DS-5 Intel SoC FPGA Edition tool and loads the UEFI firmware for debug and development purposes. This script loads the debug symbols for the user. Currently, it only supports the GCC compiler. ARMCC is not supported.
<code>~ /uefi-socfpga/Build/DXE.ROM</code>	This file loads the optional second stage of the UEFI boot loader when you want to boot the UEFI shell and utilize the TFTP feature or run a UEFI application. The difference between the PEI.ROM file and the DXE.ROM file are: - The PEI.ROM file loads and executes from the HPS on-chip RAM. The DXE.ROM file executes from the DDR SDRAM. - The PEI.ROM file generally initializes the HPS while the DXE.ROM file targets the Ethernet MAC and PHY providing full network stack support.

1.5.4. Creating an SD Card Image

This section describes how to create an SD card boot image by using the precompiled Linux binaries package as a beginning template. This section also provides instructions on how to compile a simple "HelloWorld" Bare Metal application and how to update individual elements on the SD card.

Task Time: 30 minutes

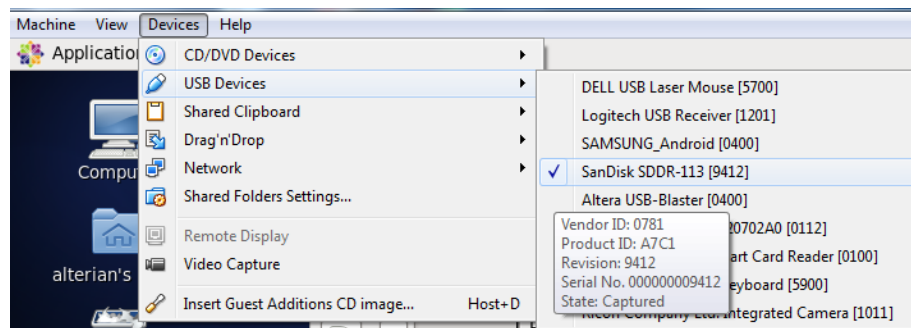
1.5.4.1. Creating an SD Card Image on Linux

1. Open a terminal window.
2. Make a copy of the GSRD into your working directory.
3. Download the pre-built SD card image from [here](#).
4. Use the Arria10_Linux_SDCard.tar.gz as a base image and modify the image to boot with UEFI boot loader.
5. Expand the compressed Linux SD card image.

```
$ tar xzvf Arria10_Linux_SDCard.tar.gz
```

6. Insert the SD card reader into your machine. Enable the San Disk SDDR-113 by navigating to **Devices** ➤ **USB Devices** ➤ **SanDisk SDDR-113 [9412]**.

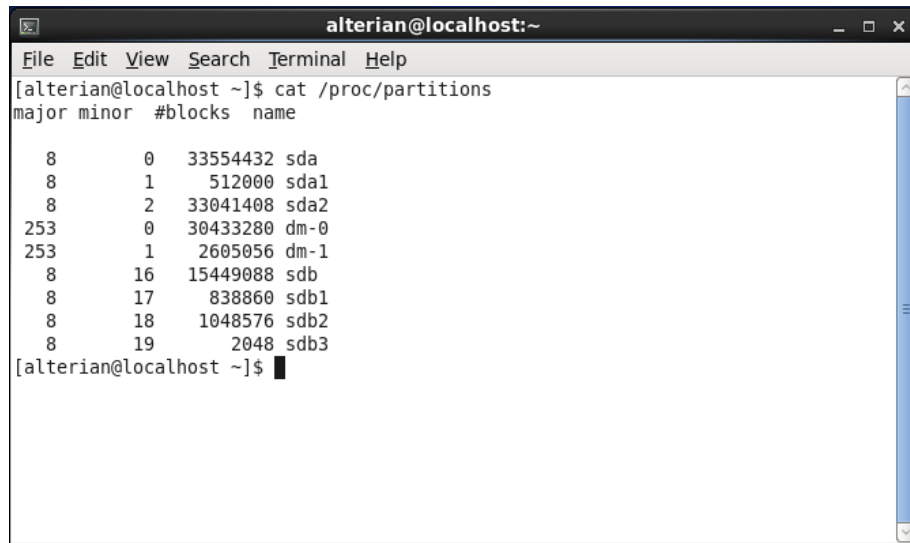
Figure 32. Enabling SanDisk SDDR-113 [9412]



7. Determine the device associated with the SD card on the host by running the following command before and after inserting the card in the reader:

```
$ cat /proc/partitions
```

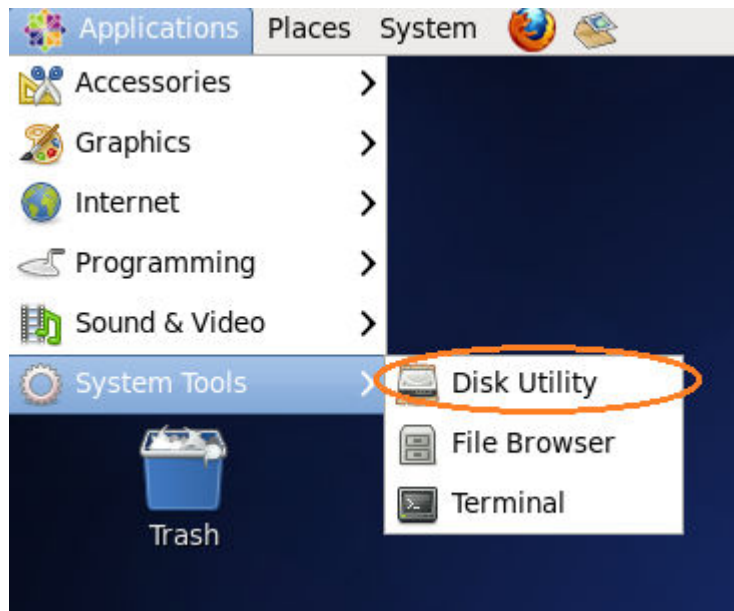
Figure 33. SD Card Determination Using cat Command



```
alterian@localhost:~  
File Edit View Search Terminal Help  
[alterian@localhost ~]$ cat /proc/partitions  
major minor #blocks name  
  
8 0 33554432 sda  
8 1 512000 sda1  
8 2 33041408 sda2  
253 0 30433280 dm-0  
253 1 2605056 dm-1  
8 16 15449088 sdb  
8 17 838860 sdb1  
8 18 1048576 sdb2  
8 19 2048 sdb3  
[alterian@localhost ~]$
```

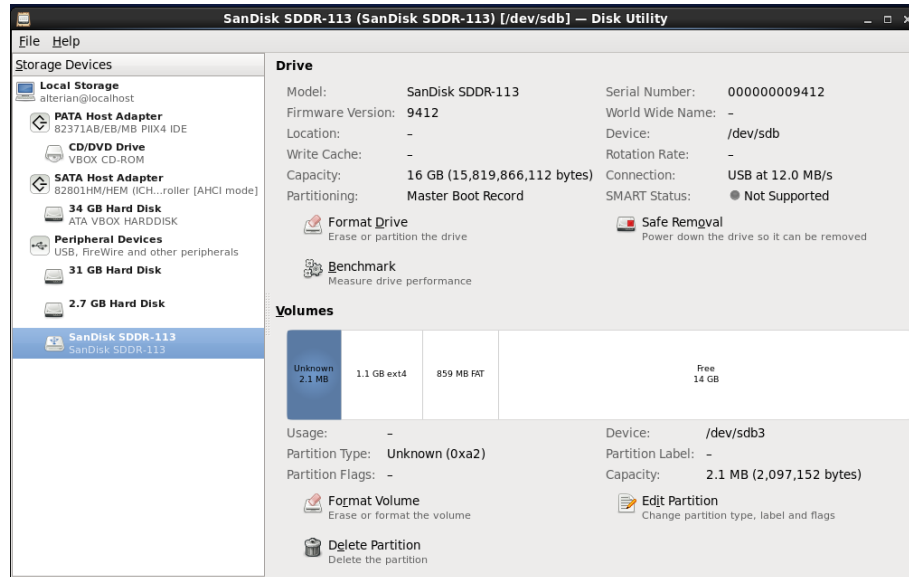
Alternatively, you can accomplish the same through the following menu selection on your machine: **Applications > System Tools > Disk Utility**

Figure 34. SD Card Determination Using Disk Utility



This selection opens a Disk Utility Window:

Figure 35. Disk Utility Window

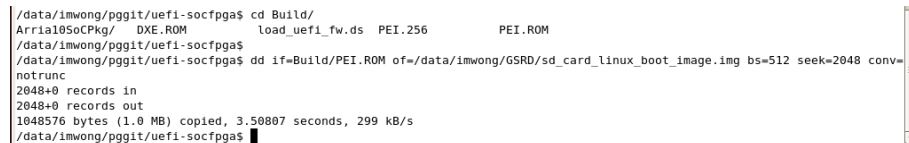


- Use the `dd` utility to overwrite the `Arria10_Linux_SDCard.img` file with the generated `PEI.ROM` and write the image into the SD card.

```
$ cd <your_uefi_directory>/uefi-socfpga

$ dd if=Build/PEI.ROM of=<your_SDCard_directory>/ Arria10_Linux_SDCard.img
bs=512 seek=2048 conv=notrunc
```

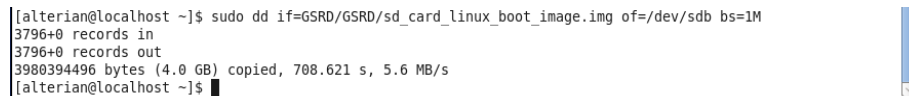
Figure 36. Image Overwrite Using dd Utility



- Insert the SD card reader into your host machine and use the `dd` utility to write the image into the SD card.

```
$ sudo dd if=<your_SDCard_directory>/ Arria10_Linux_SDCard.img of=/dev/sdb
bs=1M
```

Figure 37. Image Write Using dd Utility

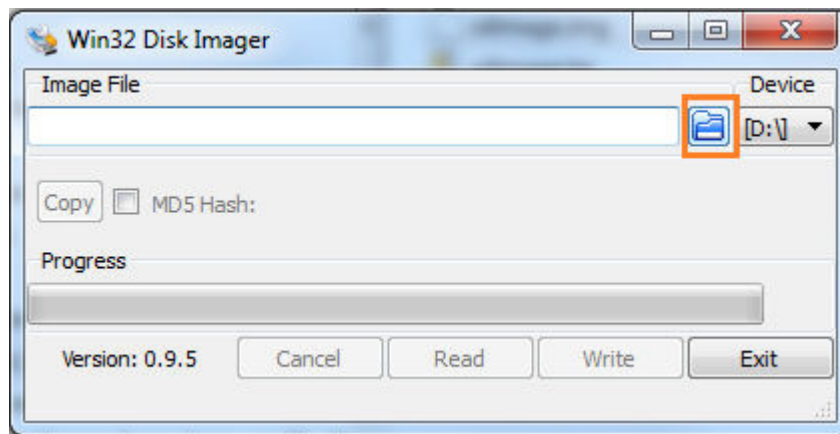


1.5.4.2. Creating an SD Card Image on Windows

This task explains how to write the SD image to the SD card on a Windows PC.

1. Download the pre-built SD card image from [here](#).
2. Use the `Arria10_Linux_SDCard.tar.gz` as a base image and modify the image to boot with UEFI boot loader.
3. Extract the compressed SD card image from the compressed archive `Arria10_Linux_SDCard.tar.gz` using WinZip or or a similar tool to create the `Arria10_Linux_SDCard.img` file.
4. Use the Win32DiskImager to write the image to the SD card. Please refer to the Win32DiskImager Tool Installation instructions for steps to install this tool on your machine.
5. Insert the SD card reader into your machine.
6. Start Win32DiskImage by selecting **Start > All Programs > Win32DiskImager** from the Windows program search or by double-clicking the Win32DiskImager icon on your desktop.
7. When the **Win32Disk Imager** window appears, click the icon highlighted below and browse to `<your_SDCard_Image_Directory>`.

Figure 38. File Browse in Win32 Disk Imager Dialog Box



8. Select **Arria10_Linux_SDCard.img** and click **Open**.
9. Click **Write**. The writing process may take some time. Check the Progress bar for status updates.
10. After a **Done** status displays and a **Complete** dialog box appears, click **OK** to exit the **Complete** dialog box.
11. Select **Exit** to close the **Win32DiskImager** window.

1.5.4.3. Compile the Bare Metal Application Source Code

This task describes how to modify and build the HelloWorld BareMetal application example that comes with the SoC FPGA EDS 15.0. The HelloWorld application writes an output to the first serial interface.

1. Open a terminal.
2. Make a copy of the HelloWorld BareMetal application example code into your working directory. For example, if your working directory is /data/<username>/GSRD, type:

```
$ cd /data/<username>/GSRD

$ cp -r ~/altera/15.0/embedded/examples/software/Altera-SoCFPGA-HelloWorld-Baremetal-ARMCC.tar.gz/ .
```

3. Extract the compressed HelloWorld project from archive.

```
$ tar -xvf Altera-SoCFPGA-HelloWorld-Baremetal-ARMCC.tar.gz
```

Figure 39. HelloWorld Application Extraction

```
-rw-r--r-- 1 imwong softtrust 3464616 Apr 14 20:08 zImage
/data/imwong/GSRD$ tar -xvf Altera-SoCFPGA-HelloWorld-Baremetal-ARMCC.tar.gz
Altera-SoCFPGA-HelloWorld-Baremetal-ARMCC/
Altera-SoCFPGA-HelloWorld-Baremetal-ARMCC/arria10_scatter.scf
Altera-SoCFPGA-HelloWorld-Baremetal-ARMCC/Makefile
Altera-SoCFPGA-HelloWorld-Baremetal-ARMCC/Altera-SoCFPGA-HelloWorld-BareMetal-ARMCC-Debug.launch
Altera-SoCFPGA-HelloWorld-Baremetal-ARMCC/hello.c
Altera-SoCFPGA-HelloWorld-Baremetal-ARMCC/cyclonev_scatter.scf
Altera-SoCFPGA-HelloWorld-Baremetal-ARMCC/.cproject
Altera-SoCFPGA-HelloWorld-Baremetal-ARMCC/.project
```

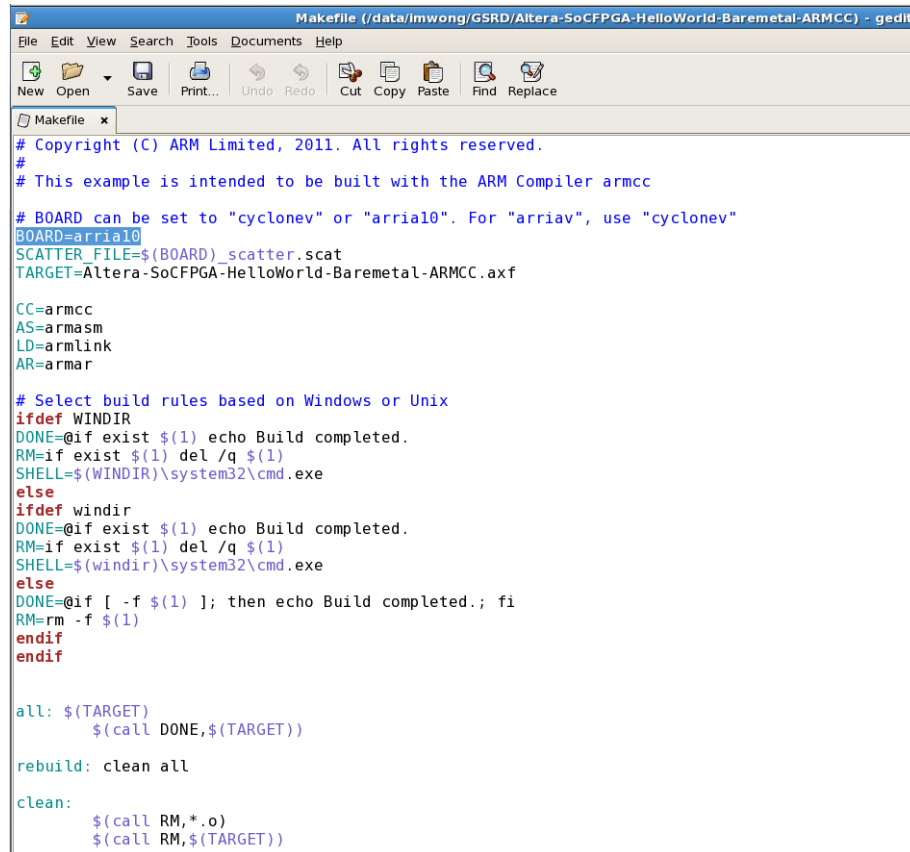
4. When complete, change to the Altera-SoCFPGA-HelloWorld-Baremetal-ARMCC folder and edit the Makefile.

```
$ cd Altera-SoCFPGA-HelloWorld-Baremetal-ARMCC

$ gedit Makefile
```

5. Change line 6 in the Makefile from BOARD=cycloneV to BOARD=arria10.

Figure 40. Makefile Edit



```
# Copyright (C) ARM Limited, 2011. All rights reserved.
# This example is intended to be built with the ARM Compiler armcc
# BOARD can be set to "cyclonev" or "arria10". For "arria10", use "cyclonev"
BOARD=arria10
SCATTER_FILE=$(BOARD)_scatter.scats
TARGET=Altera-SoCFPGA-HelloWorld-Baremetal-ARMCC.axf

CC=armcc
AS=armasm
LD=armlink
AR=armar

# Select build rules based on Windows or Unix
ifdef WINDIR
DONE=@if exist $(1) echo Build completed.
RM=if exist $(1) del /q $(1)
SHELL=$(WINDIR)\system32\cmd.exe
else
ifdef windir
DONE=@if exist $(1) echo Build completed.
RM=if exist $(1) del /q $(1)
SHELL=$(windir)\system32\cmd.exe
else
DONE=@if [ -f $(1) ]; then echo Build completed.; fi
RM=rm -f $(1)
endif
endif

all: $(TARGET)
    $(call DONE,$(TARGET))

rebuild: clean all

clean:
    $(call RM,*.o)
    $(call RM,$(TARGET))
```

6. Click the **Save** icon and exit the Makefile.
7. Open the `arria10_scatter.scats` file by typing:

```
$ gedit arria10_scatter.scats
```
8. Modify the `arria10_scatter.scats` file as shown in the screenshots below to make the Bare Metal application compile for DDR SDRAM instead of on-chip RAM. The text that you must change in the `arria10_scatter.scats` is highlighted in orange:

Figure 41. Original arria10_scatter.scot File

```

*****
; Copyright (c) 2015 Altera All Rights Reserved.
*****

; Scatter-file for OnChip RAM based example

; This scatter-file places application code, data, stack and heap at suitable addresses in the memory map.

; Altera Arria10 SoC-FPGA has 256kB of internal OnChip RAM at base address 0xFFE00000
0CRAM 0xFFE00000 0x40000
{
    APP_CODE +0
    {
        * (+R0, +RW, +ZI)
    }
    ARM_LIB_STACKHEAP 0xFFE20000 EMPTY 0x20000 ; Application heap and stack
    { }
}
    
```

The replacement text is shown below:

Figure 42. Modified arria10_scatter.scot File

```

*****
; Copyright (c) 2015 Altera All Rights Reserved.
*****

; Scatter-file for SDRAM RAM based example

; This scatter-file places application code, data, stack and heap at suitable addresses in the memory map.

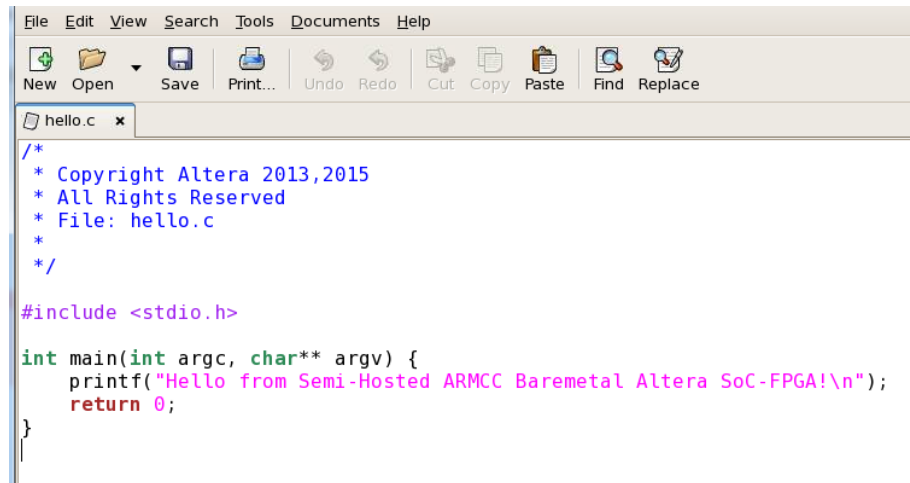
; Start at 1MB SDRAM RAM at base address 0x00100000
SDRAM 0x00100000 0x00400000
{
    APP_CODE +0
    {
        * (+R0, +RW, +ZI)
    }
    ARM_LIB_STACKHEAP 0x00400000 EMPTY 0x00100000 ; Application heap and stack
    { }
}
    
```

9. Click the **Save** icon and exit arria10_scatter.scot file.
10. Open the helloworld.c source file.

```
$ gedit helloworld.c
```

11. Edit the file according to your own custom code. Below is an example of a HelloWorld program that allows you to see "Hello World from Baremetal Application!" messages displaying on the board.

Figure 43. Original hello.c File



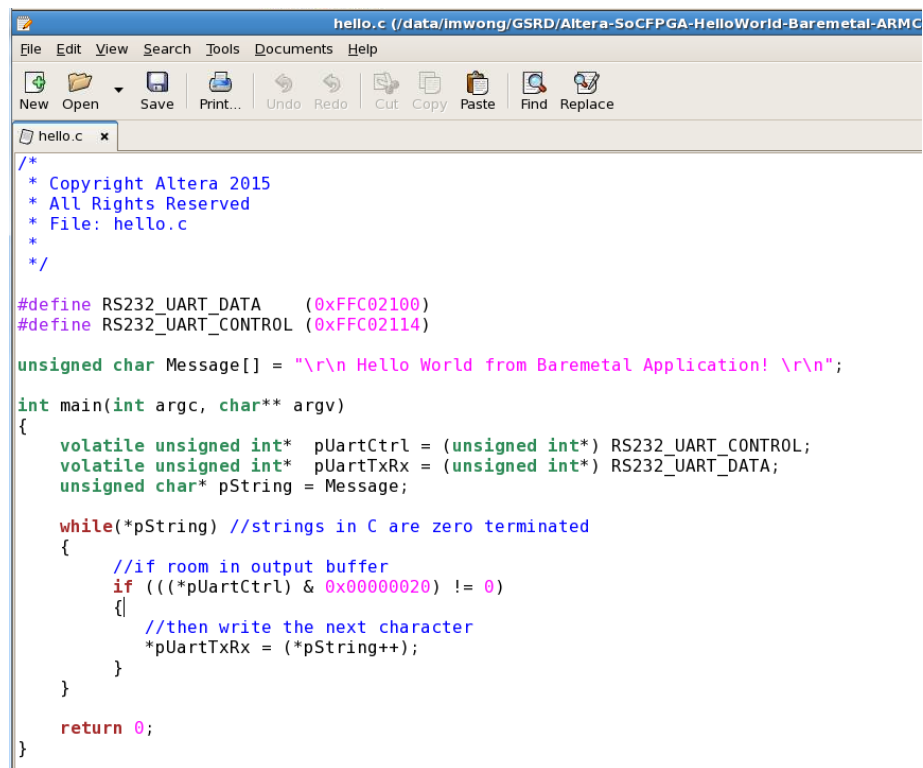
```
/*
 * Copyright Altera 2013,2015
 * All Rights Reserved
 * File: hello.c
 */

#include <stdio.h>

int main(int argc, char** argv) {
    printf("Hello from Semi-Hosted ARMCC Baremetal Altera SoC-FPGA!\n");
    return 0;
}
```

The screenshot below shows the modified hello.c file.

Figure 44. Modified hello.c File



```
/*
 * Copyright Altera 2015
 * All Rights Reserved
 * File: hello.c
 */

#define RS232_UART_DATA (0xFFC02100)
#define RS232_UART_CONTROL (0xFFC02114)

unsigned char Message[] = "\r\n Hello World from Baremetal Application! \r\n";

int main(int argc, char** argv)
{
    volatile unsigned int* pUartCtrl = (unsigned int*) RS232_UART_CONTROL;
    volatile unsigned int* pUartTxRx = (unsigned int*) RS232_UART_DATA;
    unsigned char* pString = Message;

    while(*pString) //strings in C are zero terminated
    {
        //if room in output buffer
        if (((*pUartCtrl) & 0x00000020) != 0)
        {
            //then write the next character
            *pUartTxRx = (*pString++);
        }
    }

    return 0;
}
```

12. Click the **Save** icon and exit the `hello.c` file.
13. Type `make` at the prompt to start the compilation.
14. Convert the `.axf` file to `.elf` file format.

```
$ fromelf --bin Altera-SoCFPGA-HelloWorld-Baremetal-ARMCC.axf --
output=bootimage.bin
```

Figure 45. .axf to .elf File Conversion

```
[09/06/2015 - 09:38,52] [imwong.pg-iccf0163]
/data/imwong/GSRD/Altera-SoCFPGA-HelloWorld-Baremetal-ARMCC
$ fromelf --bin Altera-SoCFPGA-HelloWorld-Baremetal-ARMCC.axf --output=bootimage.bin
[09/06/2015 - 09:39,35] [imwong.pg-iccf0163]
/data/imwong/GSRD/Altera-SoCFPGA-HelloWorld-Baremetal-ARMCC
$ ls -la
total 84
drwxr-xr-x 2 imwong sofftrust 4096 Jun 5 17:29 .
drwxr-xr-x 4 imwong sofftrust 4096 Jun 5 14:31 ..
-rw-r--r-- 1 imwong sofftrust 10252 Jun 3 17:30 Altera-SoCFPGA-HelloWorld-Baremetal-ARMCC.axf
-rw-r--r-- 1 imwong sofftrust 9661 Apr 11 00:18 Altera-SoCFPGA-HelloWorld-BareMetal-ARMCC-Debug.launch
-rw-r--r-- 1 imwong sofftrust 204 Jun 3 17:31 app.bin
-rwxr-xr-x 1 imwong sofftrust 537 Jun 3 17:07 arria10_scatter.scats
-rw-r--r-- 1 imwong sofftrust 204 Jun 9 09:39 bootimage.bin
-rw-r--r-- 1 imwong sofftrust 268 Jun 5 17:29 bootimage.bin.mkimage
-rw-r--r-- 1 imwong sofftrust 11055 Apr 11 00:18 .cproject
-rwxr-xr-x 1 imwong sofftrust 572 Apr 11 00:18 cyclonev_scatter.scats
-rw-r--r-- 1 imwong sofftrust 733 Jun 3 17:24 hello.c
-rw-r--r-- 1 imwong sofftrust 7252 Jun 3 17:30 hello.o
-rw-r--r-- 1 imwong sofftrust 1016 Jun 3 17:01 Makefile
-rwxr-xr-x 1 imwong sofftrust 2459 Apr 11 00:18 .project

[09/06/2015 - 09:39,40] [imwong.pg-iccf0163]
/data/imwong/GSRD/Altera-SoCFPGA-HelloWorld-Baremetal-ARMCC
$
```

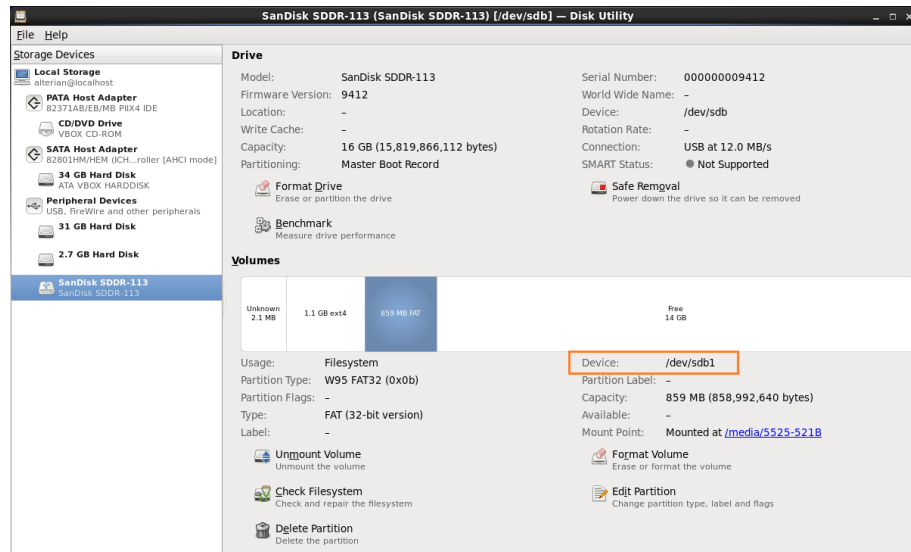
1.5.4.4. Updating Individual Elements on the SD Card

This task describes how to update the following elements individually into the SD card:

- `ghrd_10as066n2.periph.rbf`
- `ghrd_10an066n2.core.rbf`
- `bootimage.bin` (Bare Metal Image)
- `kernel-uImage.bin` (Integrity Image)
- `vxWorks.bin` (VxWorks Image)

1. Open the SD card to view the FAT partition. Go to **Application > System Tool > Disk Utility**.
2. Observe that the FAT partition location at `/dev/sdb1`.

Figure 46. Disk Utility FAT Partition View



3. Open a Terminal.
4. Mount /dev/sdb1 (FAT) on the host machine and copy the `core.rbf` and `periph.rbf` files and application image into the FAT partition using the commands shown below:

```
$ sudo mount /dev/sdb1 sdcard/
$ sudo cp GSRD/GSRD/ghrd_10as066n2.periph.rbf sdcard/
$ sudo cp GSRD/GSRD/ghrd_10as066n2.core.rbf sdcard/
$ sudo cp GSRD/GSRD/bootimage.bin sdcard/
$ sudo cp GSRD/GSRD/kernel-uImage.bin sdcard/
$ sudo cp GSRD/GSRD/vxWorks.bin sdcard/
$ sudo umount sdcard
```

1.5.4.4.1. Programming the Bare Metal Image

This task describes how to configure your program to boot the Bare Metal application program automatically on your target board after power up.

1. Open a terminal window and enter the following command:

```
$ cd <your_uefi_directory>/uefi-socfpga
```

2. Run the following command to open and edit the `<your_uefi_socfpga>/uefi-socfpga/AlteraPlatformPkg/Arria10SoCPkg/Arria10SoCPkg.dsc` file.

```
$ gedit AlteraPlatformPkg/Arria10SoCPkg/Arria10SoCPkg.dsc
```

3. Set the `PcdSdmmc_BOOTIMAGE_FILENAME` parameter in the file to `bootimage.bin`.

Figure 47. Editing Arria10SoCPkg.dsc Parameters

```

*Arria10SoCPkg.dsc (/data/imwong/pggit/uefi-socfpga/AlteraPlatformPkg/Arria10SoCPkg) - gedit
File Edit View Search Tools Documents Help
New Open Save Print... Undo Redo Cut Copy Paste Find Replace

# Suggest set to value 0x1000 when debugging
# If you change the size, to make sure there is no overlap,
# please also adjust PcdCPUCoreStackBase and PcdCPUCorePrimaryStackSize
gAlteraHpsTokenSpaceGuid.PcdMemorySerialLogSize|0x1000

## QSPI/NAND Flash Offset
gAlteraSocFpgaTokenSpaceGuid.PcdQspi0rNand_BOOTLOADER_PFI0R_ADDR |0x00000000
gAlteraSocFpgaTokenSpaceGuid.PcdQspi0rNand_BOOTLOADER_DXR0R_ADDR |0x00680000
gAlteraSocFpgaTokenSpaceGuid.PcdQspi0rNand_RBF_ADDR |0x00720000
gAlteraSocFpgaTokenSpaceGuid.PcdQspi0rNand_BOOTIMAGE_DTB_ADDR |0x00100000
gAlteraSocFpgaTokenSpaceGuid.PcdQspi0rNand_BOOTIMAGE_ADDR |0x00120000

## SD/MMC Flash
gAlteraSocFpgaTokenSpaceGuid.PcdSdmmc_BOOTIMAGE_DTB_FILENAME |"socfpga_arria10_socdk.dtb"
gAlteraSocFpgaTokenSpaceGuid.PcdSdmmc_BOOTIMAGE_FILENAME |"bootimage.bin"

# Boot Image Load Address and Entry Point
# If the image have mkimage header, it will based on mkimage header
# If the image do not have an mkimage header, it will base on PCDs
gAlteraSocFpgaTokenSpaceGuid.PcdBoot_BOOTIMAGE_MEM_LOAD_ADDR |0x100000
gAlteraSocFpgaTokenSpaceGuid.PcdBoot_BOOTIMAGE_CPU_JUMP_ADDR |0x100000

##-----
## Pcd Settings - Timer Frequency
##-----
# Clock source from HPS_CLK1 pin to osc1_clk to OsciTimer0 and OsciTimer1
gAlteraHpsTokenSpaceGuid.Pcd_OsciTimer0_Base|0xFFD00000
gAlteraHpsTokenSpaceGuid.Pcd_OsciTimer0_ClkFreqInHz|100000000

gAlteraHpsTokenSpaceGuid.Pcd_OsciTimer1_Base|0xFFD00100
gAlteraHpsTokenSpaceGuid.Pcd_OsciTimer1_ClkFreqInHz|100000000

```

4. Click **Save**.
5. Re-compile the UEFI Bootloader for the Intel Arria 10 SoC FPGA device using the following command. The build process takes less than 3 minutes.

```
$ make fast
```

6. When the build completes successfully, a **Build Done** message appears.

Figure 48. Build Done Message

```

xterm (on pg-icc0126)

Generate Region at Offset 0x0
Region Size = 0x160000
Region Name = FV

Generating FV_DXE_BEFORE_COMPRESSION FV

Generate Region at Offset 0x0
Region Size = 0x160000
Region Name = FV
Fd File Name:ALTERA_HPS_DBGH_EFI_PART2

Generate Region at Offset 0x0
Region Size = 0x80000
Region Name = FV

Generating FV_DXE_COMPRESSED FV

Generate Region at Offset 0x0
Region Size = 0x80000
Region Name = FV

GUID cross reference file can be found at /data/imwong/pggit/uefi-socfpga/Build/Arria10SoCPkg/RELEASE_GCC48/FV/Guid.xref

FV Space Information
FV_SEC_UNCOMPRESSED [632Full] 4096 total, 2948 used, 1248 free
FV_DXE_BEFORE_COMPRESSION [100Full] 1253272 total, 1253272 used, 0 free
FV_PEI_UNCOMPRESSED [67Full] 180220 total, 120832 used, 59388 free
FV_DXE_COMPRESSED [74Full] 524288 total, 388904 used, 135384 free
Build report can be found at /data/imwong/pggit/uefi-socfpga/report.log

- Done -
Build end time: 14:27:19, Jun,12 2015
Build total time: 00:01:02

make[1]: Leaving directory '/data/imwong/pggit/uefi-socfpga'
mkimage -Fb -ROM - Build/PEI_ROM Build/DXE_ROM
Picked up JAVA_TOOL_OPTIONS: -Xmx1024m
Picked up JAVA_TOOL_OPTIONS: -Xmx1024m
Creating IS-B script : Build/load_uefi_fw.ds
Build Done.

[12/06/2015 - 14:27.19] [imwong,pg-icc0126]
/data/imwong/pggit/uefi-socfpga
$ cd Build/

[12/06/2015 - 14:27.31] [imwong,pg-icc0126]
/data/imwong/pggit/uefi-socfpga/Build
$ ls -la
total 1824
drwxr-xr-x 3 imwong softtrust 4096 Jun 12 14:27 .
drwxr-xr-x 44 imwong softtrust 4096 Jun 10 17:54 ..
drwxr-xr-x 3 imwong softtrust 4096 May 29 14:53 Arria10SoCPkg

```

7. Use the dd utility to overwrite the sd_card_linux_boot_image.img file with the generated PEI.ROM.

```
$ cd <your_uefi_directory>/uefi-socfpga  
  
$ dd if=Build/PEI.ROM of=/<your_SDCard_directory>/  
sd_card_linux_boot_image.img bs=512 seek=2048 conv=notrunc
```

8. Insert the SD card reader into your host machine.
9. Use the dd utility or Win32DiskImager to write the sd_card_linux_boot_image.img into the SD Card.

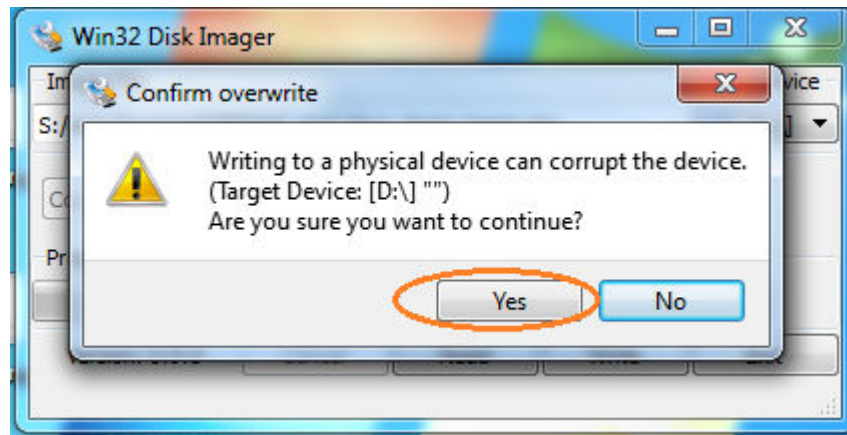
If you are using the dd utility type:

```
$ sudo dd if=/<your_SDCard_directory>/sd_card_linux_boot_image.img of=/dev/  
sdb bs=1M
```

If you are using the Win32DiskImager:

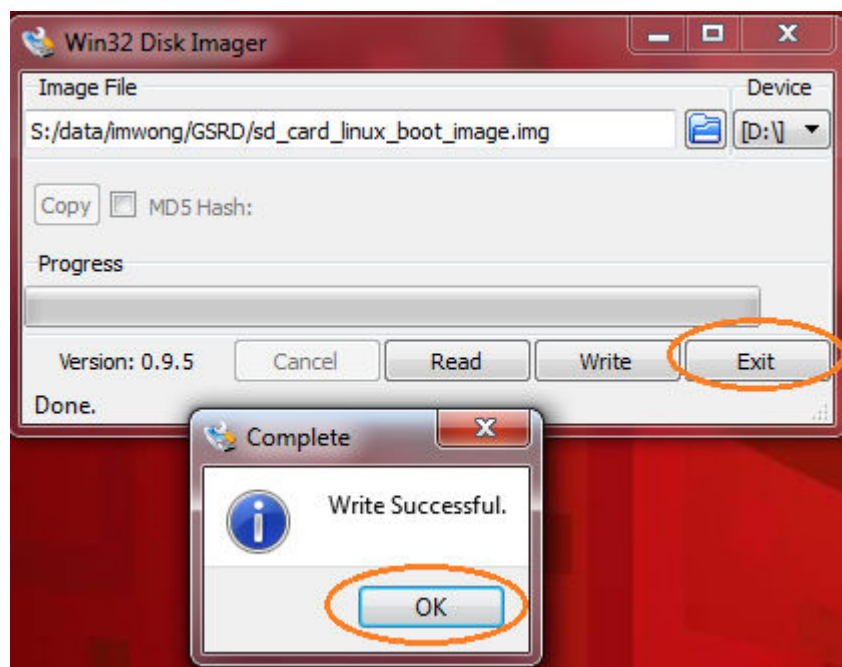
- a. Double-click the Win32DiskImager Icon.
- b. When the **Win32DiskImager** dialog box appears, browse to select the sd_card_linux_boot_image.img file and click **Open**.
- c. Select the **Write** button.
- d. When the **Confirm overwrite** dialog box appears, click **Yes** to continue. A progress bar appears.

Figure 49. Win32 Disk Imager Overwrite Confirmation



- e. After the write operation completes, a **Complete** dialog box appears. Click **OK** to exit the dialog box and then click **Exit** in the **Image File** window.

Figure 50. Exit Win32 Disk Imager



- f. Remove the SD card reader from your host.

1.5.4.4.2. Programming the Integrity Real-Time Operating System Image

This task describes how to boot the Integrity Real-Time Operating System (RTOS) automatically on your target board after power up.

1. Open a terminal window and enter the following command:

```
$ cd <your_uefi_directory>/uefi-socfpga
```

2. Run the following command to open and edit the <your_uefi_socfpga>/uefi-socfpga/AlteraPlatformPkg/Arria10SoCPkg/Arria10SoCPkg.dsc file.

```
$ gedit AlteraPlatformPkg/Arria10SoCPkg/Arria10SoCPkg.dsc
```

3. Set the PcdSdmmc_BOOTIMAGE_FILENAME parameter in the file to kernel-uImage.bin.

Figure 51. Editing Arria10SoCPkg.dsc Parameters

```

# Pit Stop utility
# Suggest set to value 0 for RELEASE build
# Suggest set to value 1 to always enable
# Suggest set to value 2+ for timeout if no key pressed
gAlteraSocFpgaTokenSpaceGuid.PcdEnablePitStopUtility|5

# On-Chip-Memory Serial Log Base address and Size
# Come in handy for PlatformInit debug as alternative
# when UART fail and Semihosting does not fit the situation
# Set PcdMemorySerialLogBase to 0 to disable this feature
# Suggest set to value 0 for RELEASE build
# Suggest set to value 0xFFE31000 when debugging
gAlteraHpsTokenSpaceGuid.PcdMemorySerialLogBase|0xFFE31000
# Suggest set to value 0x1000 when debugging
# If you change the size, to make sure there is no overlap,
# please also adjust PcdCpuCoresStackBase and PcdCpuCorePrimaryStackSize
gAlteraHpsTokenSpaceGuid.PcdMemorySerialLogSize|0x1000

## QSPI/NAND Flash Offset
gAlteraSocFpgaTokenSpaceGuid.PcdQspi0rNand_BOOTLOADER_PFIOM_ADDR |0x00000000
gAlteraSocFpgaTokenSpaceGuid.PcdQspi0rNand_BOOTLOADER_DSEROM_ADDR |0x00680000
gAlteraSocFpgaTokenSpaceGuid.PcdQspi0rNand_RBF_ADDR |0x00720000
gAlteraSocFpgaTokenSpaceGuid.PcdQspi0rNand_BOOTIMAGE_DTB_ADDR |0x00100000
gAlteraSocFpgaTokenSpaceGuid.PcdQspi0rNand_BOOTIMAGE_ADDR |0x00120000

## SD/MMC Flash
gAlteraSocFpgaTokenSpaceGuid.PcdSdmmc_BOOTIMAGE_DTB_FILENAME |"socfpga_arria10_socdk.dtb"
gAlteraSocFpgaTokenSpaceGuid.PcdSdmmc_BOOTIMAGE_FILENAME |"kernel-0Image.bin"

# Boot Image Load Address and Entry Point
# If the image have mkimage header, it will based on mkimage header
# If the image do not have an mkimage header, it will base on PCDs
gAlteraSocFpgaTokenSpaceGuid.PcdBoot_BOOTIMAGE_MEM_LOAD_ADDR |0x100000
gAlteraSocFpgaTokenSpaceGuid.PcdBoot_BOOTIMAGE_CPU_JUMP_ADDR |0x100000

##-----
## Pcd Settings - Timer Frequency
##-----

```

4. Click **Save**.
5. Re-compile the UEFI Bootloader for the Intel Arria 10 SoC FPGA device using the following command. The build process takes less than 3 minutes.

```
$ make fast
```

6. When the build completes successfully, a **Build Done** message appears.

Figure 52. Build Done Message

```

Generate Region at Offset 0x0
Region Size = 0x160000
Region Name = FV

Generating FV_DXE_BEFORE_COMPRESSION FV

Generate Region at Offset 0x0
Region Size = 0x160000
Region Name = FV
Fd File Name:ALTERH_PPS_IRAWL_EFI_PART2

Generate Region at Offset 0x0
Region Size = 0x80000
Region Name = FV

Generating FV_DXE_COMPRESSED FV

Generate Region at Offset 0x0
Region Size = 0x80000
Region Name = FV

GUID cross reference file can be found at /data/imwong/pggit/uefi-socfpga/Build/Arria10SoCPkg/RELEASE_GCC48/FV/Guid.vref

FV Space Information
FV_SEC_UNCOMPRESSED [688Full] 4096 total, 2048 used, 1248 free
FV_DXE_BEFORE_COMPRESSION [1003Full] 1253272 total, 1253272 used, 0 free
FV_PEI_UNCOMPRESSED [677Full] 180220 total, 120832 used, 59388 free
FV_DXE_COMPRESSED [742Full] 524288 total, 389304 used, 135384 free
Build report can be found at /data/imwong/pggit/uefi-socfpga/report.log

- Done -
Build end time: 14:27:18, Jun,12 2015
Build total time: 00:01:02

make[1]: Leaving directory '/data/imwong/pggit/uefi-socfpga'
mkimage.FD to .ROM : Build/PEI.ROM Build/DXE.ROM
Picked up JHWA_TOOL_OPTIONS: -mx1024m
Picked up JHWA_TOOL_OPTIONS: -mx1024m
Creating BS-5 script : Build/load_uefi_fw.ds
Build Done.

[12/06/2015 - 14:27:19] [imwong.pg-iccf0126]
/data/imwong/pggit/uefi-socfpga
$ cd Build/

[12/06/2015 - 14:27:31] [imwong.pg-iccf0126]
/data/imwong/pggit/uefi-socfpga/Build
$ ls -la
total 1824
drwxr-xr-x 3 imwong softtrust 4096 Jun 12 14:27 .
drwxr-xr-x 44 imwong softtrust 4096 Jun 10 17:54 ..
drwxr-xr-x 3 imwong softtrust 4096 May 29 14:55 Arria10SoCPkg
  
```

7. Use the dd utility to overwrite the sd_card_linux_boot_image.img file with the generated PEI.ROM.

```

$ cd <your_uefi_directory>/uefi-socfpga

$ dd if=Build/PEI.ROM of=<your_SDCard_directory>/
sd_card_linux_boot_image.img bs=512 seek=2048 conv=notrunc
  
```

8. Insert the SD card reader into your host machine.
9. Use the dd utility or Win32DiskImager to write the sd_card_linux_boot_image.img into the SD Card.

If you are using the dd utility type:

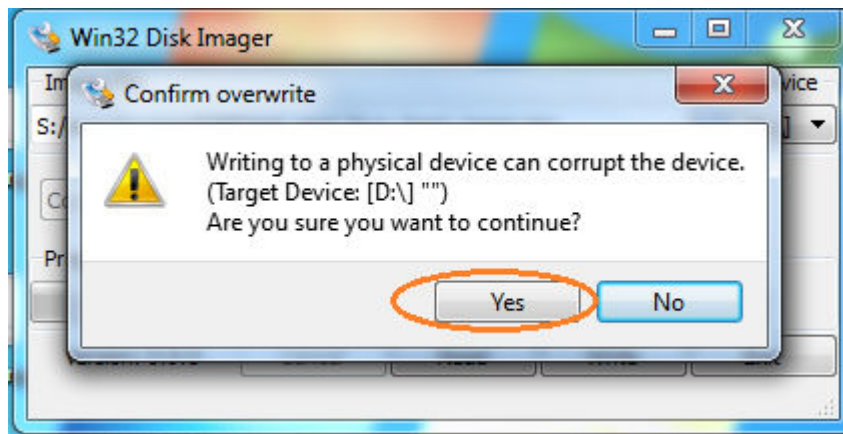
```

$ sudo dd if=<your_SDCard_directory>/sd_card_linux_boot_image.img of=/dev/
sdb bs=1M
  
```

If you are using the Win32DiskImager:

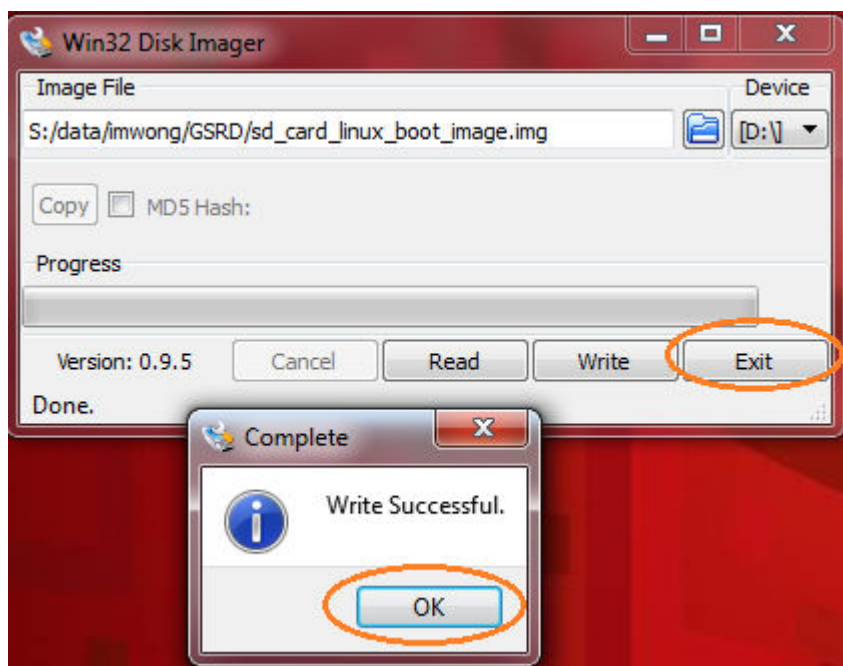
- a. Double-click the **Win32DiskImager** Icon.
- b. When the **Win32 Disk Imager** dialog box appears, browse to select the sd_card_linux_boot_image.img file and click **Open**.
- c. Select the **Write** button.
- d. When the **Confirm overwrite** dialog box appears, click **Yes** to continue. A progress bar appears.

Figure 53. Win32 Disk Imager Overwrite Confirmation



- e. After the write operation completes, a **Complete** dialog box appears. Click **OK** to exit the dialog box and then click **Exit** in the **Image File** window.

Figure 54. Exit Win32 Disk Imager



- f. Remove the SD card reader from your host.

1.5.4.4.3. Programming the VxWorks® Real-Time Operating System Image

This task describes how to boot the Integrity Real-Time Operating System (RTOS) automatically on your target board after power up.

1. Open a terminal window and enter the following command:

```
$ cd <your_uefi_directory>/uefi-socfpga
```

- Run the following command to open and edit the `<your_uefi_socfpga>/uefi-socfpga/AlteraPlatformPkg/Arria10SoCPkg/Arria10SoCPkg.dsc` file.

```
$ gedit AlteraPlatformPkg/Arria10SoCPkg/Arria10SoCPkg.dsc
```

- Set the `PcdSdmmc_BOOTIMAGE_FILENAME` parameter in the file to `kernel-uImage.bin`.

Figure 55. Editing Arria10SoCPkg.dsc Parameters

```
Arria10SoCPkg.dsc (/data/lmwong/pggit/uefi-socfpga/AlteraPlatformPkg/Arria10SoCPkg) - gedit
File Edit View Search Tools Documents Help
New Open Save Print... Undo Redo Cut Copy Paste Find Replace
Arria10SoCPkg.dsc
# Pit Stop utility
# Suggest set to value 0 for RELEASE build
# Suggest set to value 1 to always enable
# Suggest set to value 2+ for timeout if no key pressed
gAlteraSocFpgaTokenSpaceGuid.PcdEnablePitStopUtility|5

# On-Chip-Memory Serial Log Base address and Size
# Come in handy for PlatformInit debug as alternative
# when UART fail and Semihosting does not fit the situation
# Set PcdMemorySerialLogBase to 0 to disable this feature
# Suggest set to value 0 for RELEASE build
# Suggest set to value 0xFFE31000 when debugging
gAlteraHpsTokenSpaceGuid.PcdMemorySerialLogBase|0xFFE31000
# Suggest set to value 0x1000 when debugging
# If you change the size, to make sure there is no overlap,
# please also adjust PcdCpuCoresStackBase and PcdCpuCorePrimaryStackSize
gAlteraHpsTokenSpaceGuid.PcdMemorySerialLogSize|0x1000

## QSPI/NAND Flash Offset
gAlteraSocFpgaTokenSpaceGuid.PcdQspiNand_BOOTLOADER_PFIROM_ADDR |0x00000000
gAlteraSocFpgaTokenSpaceGuid.PcdQspiNand_BOOTLOADER_DXEROM_ADDR |0x00680000
gAlteraSocFpgaTokenSpaceGuid.PcdQspiNand_RBF_ADDR |0x00720000
gAlteraSocFpgaTokenSpaceGuid.PcdQspiNand_BOOTIMAGE_DTB_ADDR |0x00100000
gAlteraSocFpgaTokenSpaceGuid.PcdQspiNand_BOOTIMAGE_ADDR |0x00120000

## SD/MMC Flash
gAlteraSocFpgaTokenSpaceGuid.PcdSdmmc_BOOTIMAGE_DTB_FILENAME |"socfpga_arria10_socdk.dtb"
gAlteraSocFpgaTokenSpaceGuid.PcdSdmmc_BOOTIMAGE_FILENAME |"vxWorks.bin"
# Boot Image Load Address and Entry Point
# If the image have mkimage header, it will based on mkimage header
# If the image do not have an mkimage header, it will base on PCDS
gAlteraSocFpgaTokenSpaceGuid.PcdBoot_BOOTIMAGE_MEM_LOAD_ADDR |0x100000
gAlteraSocFpgaTokenSpaceGuid.PcdBoot_BOOTIMAGE_CPU_JUMP_ADDR |0x100000

##-----
## Pcd Settings - Timer Frequency
##-----
```

- Click **Save**.
- Re-compile the UEFI Bootloader for the Intel Arria 10 SoC FPGA device using the following command. The build process takes less than 3 minutes.

```
$ make fast
```

- When the build completes successfully, a **Build Done** message appears.

Figure 56. Build Done Message

```
xterm (on pg-iccf0126)

Generate Region at Offset 0x0
  Region Size = 0x160000
  Region Name = FV

Generating FV_DXE_BEFORE_COMPRESSION FV

Generate Region at Offset 0x0
  Region Size = 0x160000
  Region Name = FV
Fd File Name: ALTERA_HPS_DRAH_EFI_PART2

Generate Region at Offset 0x0
  Region Size = 0x80000
  Region Name = FV

Generating FV_DXE_COMPRESSED FV

Generate Region at Offset 0x0
  Region Size = 0x80000
  Region Name = FV

GUID cross reference file can be found at /data/imwong/pggit/uefi-socfpga/Build/Arria10SoCPkg/RELEASE_GCC48/FV/Guid.xref

FV Space Information
FV_SEC_UNCOMPRESSED [683Full] 4096 total, 2940 used, 1248 free
FV_DXE_BEFORE_COMPRESSION [1002Full] 1253272 total, 1253272 used, 0 free
FV_PEI_UNCOMPRESSED [672Full] 180220 total, 120632 used, 59588 free
FV_DXE_COMPRESSED [742Full] 524288 total, 388904 used, 135384 free
Build report can be found at /data/imwong/pggit/uefi-socfpga/report.log

- Done -
Build end time: 14:27:18, Jun.12 2015
Build total time: 00:01:02

make[1]: Leaving directory '/data/imwong/pggit/uefi-socfpga'
mkimage -F to .ROM : Build/PEI.ROM Build/DXE.ROM
Picked up JAWA_TOOL_OPTIONS: -Xmx1024m
Picked up JAVA_TOOL_OPTIONS: -Xmx1024m
Creating IS-5 script : Build/load_uefi_fw.ds
Build Done.

[12/06/2015 - 14:27.19] [imwong.pg-iccf0126]
/data/imwong/pggit/uefi-socfpga
$ cd Build/
[12/06/2015 - 14:27.31] [imwong.pg-iccf0126]
/data/imwong/pggit/uefi-socfpga/Build
$ ls -la
total 1824
drwxr-xr-x 3 imwong sofrtrust 4096 Jun 12 14:27 .
drwxr-xr-x 44 imwong sofrtrust 4096 Jun 10 17:54 ..
drwxr-xr-x 3 imwong sofrtrust 4096 May 23 14:53 Arria10SoCPkg
```

7. Use the dd utility to overwrite the sd_card_linux_boot_image.img file with the generated PEI.ROM.

```
$ cd <your_uefi_directory>/uefi-socfpga

$ dd if=Build/PEI.ROM of=/<your_SDCard_directory>/
sd_card_linux_boot_image.img bs=512 seek=2048 conv=notrunc
```

8. Insert the SD card reader into your host machine.
9. Use the dd utility or Win32DiskImager to write the sd_card_linux_boot_image.img into the SD Card.

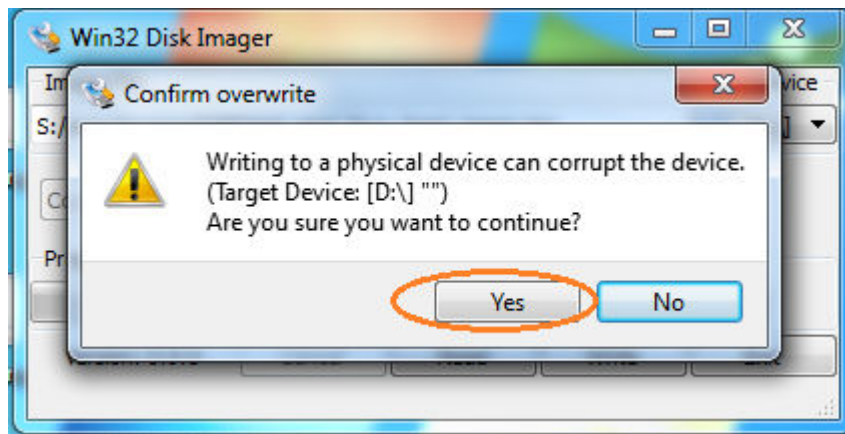
If you are using the dd utility type:

```
$ sudo dd if=/<your_SDCard_directory>/sd_card_linux_boot_image.img of=/dev/
sdb bs=1M
```

If you are using the Win32DiskImager:

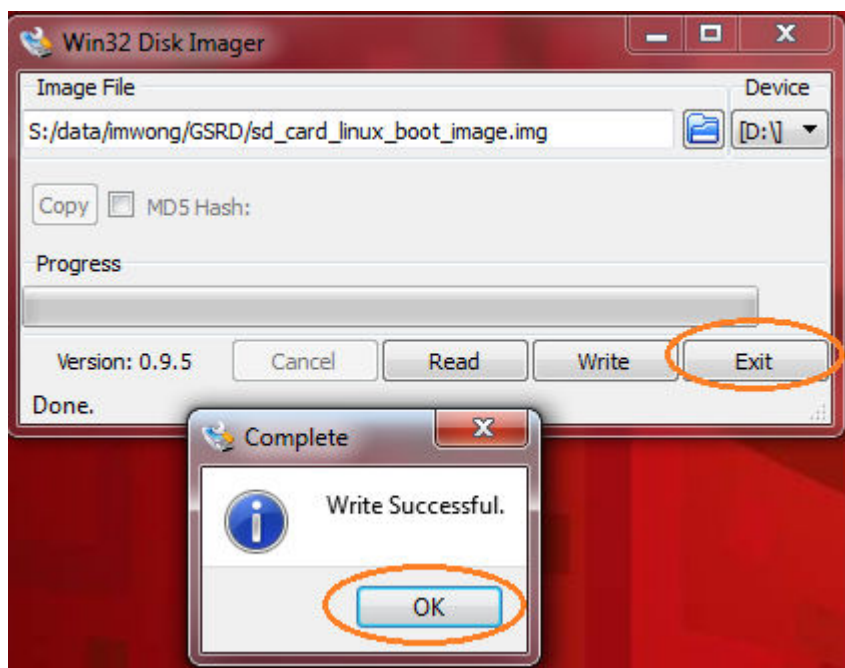
- a. Double-click the Win32DiskImager Icon.
- b. When the **Win32DiskImager** dialog box appears, browse to select the sd_card_linux_boot_image.img file and click **Open**.
- c. Select the **Write** button.
- d. When the **Confirm overwrite** dialog box appears, click **Yes** to continue. A progress bar appears.

Figure 57. Win32 Disk Imager Overwrite Confirmation



- e. After the write operation completes, a **Complete** dialog box appears. Click **OK** to exit the dialog box and then click **Exit** in the **Image File** window.

Figure 58. Exit Win32 Disk Imager



- f. Remove the SD card reader from your host.

1.5.5. Creating a QSPI Image

This section describes how to update the GSRD to enable booting from QSPI on your target board.

Approximate completion time: 80 minutes

1.5.5.1. Updating the Golden Hardware Reference Design to Boot from QSPI

The SoC FPGA EDS GHRD enables booting from SDMMC by default. There are two options available for you to change the booting from SDMMC to QSPI:

- Manually change the configuration to QSPI by:
 - Opening the HPS in Platform Designer to modify the pin muxing and enable peripherals
 - Changing pin assignments in Intel Quartus Prime Pro Edition
 - Changing the top level Verilog file to include the QSPI signals instead of the SD/MMC signals.

Manually changing the configuration is time-intensive.

- Another, more efficient option is by taking advantage of the Makefile feature. The example shown in this sections uses this method to perform the boot conversion from SDMMC to QSPI.
1. Start an embedded command shell. Make a copy of the ghrd folder into your QSPI working directory.

```
$ ~/altera/15.0/embedded/embedded_command_shell.sh
$ mkdir <your_qspi_working_directory>/a10_soc_devkit_ghrd_qspi
$ cd <your_qspi_working_directory>/a10_soc_devkit_ghrd_qspi
$ tar xvzf ~/altera/15.0/embedded/examples/hardware/a10_soc_devkit_ghrd/tgz/
*.tar.gz
```

2. When complete, run the following commands to enable booting from QSPI and generate the Platform Designer file:

```
$ sed -i 's/HPS_BOOT_DEVICE := SDMMC/HPS_BOOT_DEVICE := QSPI/g' Makefile
$ sed -i 's/QUARTUS_CPF_ENABLE_SPLIT_RBF := 1/
QUARTUS_CPF_ENABLE_SPLIT_RBF := 0/g' Makefile
$ make scrub_clean
$ make quartus_generate_top quartus_generate_qsf_qpf qsys_generate_qsys
```

After a series of messages ending in Info: save_system
ghrd_10as066n2.qsys, a prompt appears.

1.5.5.2. Compiling the Hardware and Software for QSPI

This task describes how to use the GHRD Makefile to compile both the hardware and the software (U-Boot).

1. Start and embedded command shell and go into your QSPI GHRD folder.

```
$ ~/altera/15.0/embedded/embedded_command_shell.sh
$ cd <your_qspi_working_directory>/a10_soc_devkit_ghrd_qspi
```

2. Run the make all command. The compilation process may take some time to complete.

```
$ make all
```

The make all command creates the following files:

Table 8. Output Files from make all Command

File	Description
~ /a10_soc_devkit_ghrd_qspi/software/boot loader/ devicetree.dtb	Boot Loader Device Tree Image
~ /a10_soc_devkit_ghrd_qspi/output_files/ghrd_10as066n2.rbf	I/O Ring and FPGA Fabric Configuration File

1.5.5.3. Compiling the UEFI Source Code with the Updated QSPI Device Tree Blob

This task describes how to re-compile the UEFI source code using the updated device tree blob (DTB) generated for the QSPI booting with the Linaro toolchain.

1. Open a Terminal window and enter the following command:

```
$ cd <your_uefi_directory>/uefi-socfpga
```

2. Compile the UEFI boot loader for Intel Arria 10 SoC FPGA device using the following command. This build process takes less than three minutes.

```
$ make DEVICE=a10 COMPILER=gcc HANDOFF_DTB=[QSPI bootloader DTB file path]
```

An example of a *[QSPI boot loader DTB file path]* is:

```
/data/<username>/GSRD/a10_soc_devkit_ghrd_qspi/software/bootloader/  
devicetree.dtb
```

Once the build completes, the following completion information highlighted by the orange text box appears:

Figure 59. make Completion

```
xterm (on pg-iccf0313)
Generate Region at Offset 0x0
Region Size = 0x160000
Region Name = FV

Generating FV_DXE_BEFORE_COMPRESSION FV
*****
Generate Region at Offset 0x0
Region Size = 0x160000
Region Name = FV
Fd File Name:ALTERA_HPS_DRAM_EFI_PART2

Generate Region at Offset 0x0
Region Size = 0x80000
Region Name = FV

Generating FV_DXE_COMPRESSED FV
*****
Generate Region at Offset 0x0
Region Size = 0x80000
Region Name = FV

GUID cross reference file can be found at /data/imwong/pggit/uefi-socfpga/Build/Arria10SoCPkg/RELEASE_GCC48/FV/Guid.xref

FV Space Information
FV_SEC_UNCOMPRESSED [692Full] 4096 total, 2848 used, 1248 free
FV_DXE_BEFORE_COMPRESSION [1002Full] 1253272 total, 1253272 used, 0 free
FV_PEI_UNCOMPRESSED [572Full] 180220 total, 120768 used, 59452 free
FV_DXE_COMPRESSED [742Full] 524288 total, 385904 used, 138384 free
Build report can be found at /data/imwong/pggit/uefi-socfpga/report.log

- Done -
Build end time: 17:51:05, Jun,05 2015
Build total time: 00:01:20

make[1]: Leaving directory '/data/imwong/pggit/uefi-socfpga'
mkpinage .FB to .ROM : Build/PEI.ROM Build/DXE.ROM
Creating IS-5 script : Build/load_uefi_fw.ds
Build Done
/data/imwong/pggit/uefi-socfpga$ make program_qspi
quartus_hps --help=

Option: -c < cable name> | --cable=< cable name>

Option to specify which programming hardware or cable to use.

The full syntax is as follows (depending on whether the hardware is on your
local machine or a remote machine):

    "< cable_name> [< port>]"
    "< cable_name> on < host_name/IP_address> [< port>]"

You don't need to specify the hostname or port if they are unambiguous so
just specifying the name of the cable will be sufficient if there is only one
cable of that type available (on a local or remote machine).

The following syntax is supported for backward compatibility,
as is the cable index (the number returned by jtagconfig to identify
a cable):

    < host_name/IP_address>::< cable_name> [< port>]@< baud_rate>

Examples:
    ByteBlasterMV
    "byteblaster [lpt1]"
    "USB-Blaster on remote-machine [com1]"
```

These steps generate four files in your *<your_uefi_directory>/uefi-socfpga/Build/* folder:

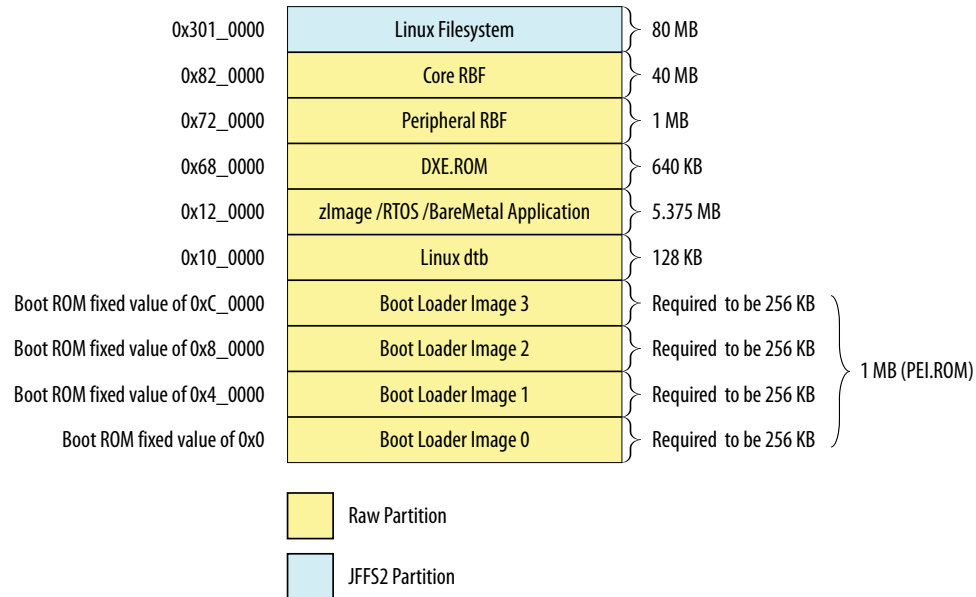
- DXE.ROM
- load_uefi_fw.ds
- PEI.256
- PEI.ROM

1.5.5.4. QSPI Layout

The following diagram depicts the QSPI Layout. Items to note within the diagram are:

- Boot loader images 0, 1, 2, and 3 are loaded with PEI.ROM
- Linux DTB and Linux filesystem are only programmed when you are booting Linux
- The next stage boot image can be either a zImage, an RTOS binary image or a Bare Metal application binary image.
- DXE.ROM is an optional second phase of the UEFI boot loader
- RBF is generated from the SOF file using the quartus_cpf tool

Figure 60. QSPI Layout



1.5.5.5. Updating Individual Elements into the QSPI

This task describes how to update the following elements individually into the QSPI memory:

- ghrd_10as066n2.rbf
- PEI.ROM (UEFI boot loader binary image)
- bootimage.bin (Bare Metal Image)
- kernel-uImage.bin (Integrity Image)
- vxWorks.bin (VxWorks Image)

- Connect the JTAG Intel FPGA Download Cable II from your host to the Intel Arria 10 SoC FPGA development board.
- Install the QSPI daughter card into J23.
- Power up the board.
- To program the compiled UEFI image into the QSPI, type the following commands:

```
$ cd <your_uefi_directory>/uefi-socfpga
$ program_qspi
```

Or you can alternatively type:

```
$ quartus_hps -c1 -oP Build/PEI.ROM --addr=0x00000000
```

- Program the single RBF into the QSPI.

- If you have a GHRD design, you can use the make command:

```
$ cd <your_qspi_working_directory>/a10_soc_devkit_ghrd_qspi
$ make program_qspi-rbf
```


- If you want to program the single RBF manually without the GHRD Makefile, type:

```
$ mkimage -A arm -T firmware -C none -O u-boot -a 0 -e 0 -n "RBF" -d
ghrd_10as066n2.rbf ghrd_10as066n2.rbf.mkimage

$ quartus_hps -c1 -oP --addr=0x00720000 ghrd_10as066n2.rbf.mkimage
```

6. If you are booting the DXE phase, program the DXE.ROM file into the QSPI.

```
$ quartus_hps -c1 -oP Build/DXE.ROM --addr=0x00620000
```

1.5.5.5.1. Programming the Bare Metal Image

This task describes how to program the Bare Metal image into QSPI.

1. Go to your generated Baremetal HelloWorld image directory. For example:

```
$ cd /data/<username>/GSRD/Altera-SoCFPGA-HelloWorld-Baremetal-ARMCC
```

2. Run the following command to add mkimage header to bootimage.bin to indicate the filesize.

```
$ mkimage -A arm -T firmware -C none -a 0x100000 -e 0x100000 -n "UserApp" -d
bootimage.bin bootimage.bin.mkimage
```

3. Type the following command to flash the boot image into the QSPI.

```
$ quartus_hps -c1 -d 2 -o P --addr=0x00120000 bootimage.bin.mkimage
```

Figure 61. Commands for Programming the Bare Metal Image

```

xterm (on pg-icf0313)
$ ls
Altera-SoCFPGA-HelloWorld-Baremetal-ARMCC,axf Altera-SoCFPGA-HelloWorld-Baremetal-ARMCC-Debug,launch app.bin arial0_scatter.scnt bootimage.bin cyclonev_scatter.scnt hello.c hello.o Makefile
[05/06/2015 - 17:28:50] [lwong.pg-icf0313]
$ mkimage -A arm -T firmware -C none -a 0x100000 -e 0x100000 -n "RBF" -d bootimage.bin bootimage.bin.mkimage
Image Name: RBF
Created: Fri Jun 5 17:29:16 2015
Image Type: ARM Linux Firmware (uncompressed)
Data Size: 204 Bytes = 0.20 kB = 0.00 MB
Load Address: 0x0100000
Entry Point: 0x0100000
[05/06/2015 - 17:29:16] [lwong.pg-icf0313]
$ quartus_hps -c1 -d 2 -o P --addr=0x00120000 bootimage.bin.mkimage
Info: Running Quartus II 64-bit Programmer
Info: Version 15.0.0 Build 145 (4/22/2015 SJ Full Version)
Info: Copyright (C) 1991-2015 Altera Corporation. All rights reserved.
Info: Your use of Altera Corporation's design tools, logic functions
Info: and other software and tools, and its APEP partner logic
Info: functions, and any output files from any of the foregoing
Info: (including device programming or simulation files), and any
Info: associated documentation or information are expressly subject
Info: to the terms and conditions of the Altera Program License
Info: Subscription Agreement, the Altera Quartus II License Agreement,
Info: the Altera MegaCore Function License Agreement, or other
Info: applicable license agreement, including, without limitation,
Info: that your use is for the sole purpose of programming logic
Info: devices manufactured by Altera and sold by Altera or its
Info: authorized distributors. Please refer to the applicable
Info: agreement for further details.
Info: Processing started: Fri Jun 5 18:04:23 2015
Info: Command: quartus_hps -c1 -d 2 -o P --addr=0x00120000 bootimage.bin.mkimage
Current hardware list: USB-BlasterII on PG-IM006-430 [USB-1]
Found HPS at device 2
HPS Device IDCODE: 0x4B00477
HPS Port is located at port 0
HPS Port is located at port 1
Double check device identification ...
Warning: Overwrite detected IDCODE
Setup non-secure transaction ...
Boot Info: 1.00 QSPI Flash
Clock Select: 0
Start HPS Quad SPI Flash programming ...
Initialize QSPI peripheral and Flash controller ...
Read Silicon ID of Quad SPI Flash ...
Quad SPI Flash silicon ID is 0x001B820
Flash device matched
Manufacturer: MICRON
Device: QSPI 1504
Enable Four Byte Addressing ...
Sector Erase Info: Start Addr at 0x00120000 for 1 sector(s)
Sector Erase Quad SPI Flash at 0x00120000
Program Quad SPI Flash ...
Info: Quartus II 64-bit Programmer was successful, 0 errors, 0 warnings
Info: Peak virtual memory: 150 megabytes
Info: Processing ended: Fri Jun 5 18:04:30 2015
Info: Elapsed time: 00:00:07
Info: Total CPU time (on all processors): 00:00:00
[05/06/2015 - 18:04:30] [lwong.pg-icf0313]
$

```

1.5.5.5.2. Programming the Integrity Real-Time Operating System Image

1. Go to your Integrity image directory. An example of a directory path is /data/<username>/sdcard/a10_soc.

```
$ cd /data/<username>/sdcard/a10_soc
```

2. Type the following command to flash the Integrity RTOS image into the QSPI.

Note: This kernel-uImage.bin file is already included in the header.

```
$ quartus_hps -c 1 - 2 -o P --addr=0x00120000 kernel-uImage.bin
```

Figure 62. Flashing the Integrity RTOS Image into QSPI

```
xterm (on pg-icc0313)
[05/06/2015 - 18:09:23] [imwong.pg-icc0313]
/data/imwong/sdcard
$ cd a10_soc/
[05/06/2015 - 18:09:31] [imwong.pg-icc0313]
/data/imwong/sdcard/a10_soc
$ quartus_hps -c 1 -d 2 -o P --addr=0x00120000 kernel-uImage.bin
Info: Running Quartus II 64-Bit Programmer
Info: Version 15.0.0 Build 145 04/22/2015 SJ Full Version
Info: Copyright (C) 1991-2015 Altera Corporation. All rights reserved.
Info: Your use of Altera Corporation's design tools, logic functions
Info: and other software and tools, and its AHPP partner logic
Info: functions, and any output files from any of the foregoing
Info: (including device programming or simulation files), and any
Info: associated documentation or information are expressly subject
Info: to the terms and conditions of the Altera Program License
Info: Subscription Agreement, the Altera Quartus II License Agreement,
Info: the Altera MegaCore Function License Agreement, or other
Info: applicable license agreement, including, without limitation,
Info: that your use is for the sole purpose of programming logic
Info: devices manufactured by Altera and sold by Altera or its
Info: authorized distributors. Please refer to the applicable
Info: agreement for further details.
Info: Processing started: Fri Jun 5 18:09:37 2015
Info: Command: quartus_hps -c 1 -d 2 --operation=P --addr=0x00120000 kernel-uImage.bin
Current hardware is: USB-BlasterII on PG-IMWONG-430 [USB-1]
Found HPS at device 2
HPS Device IDCODE: 0x4BA00477
AHB Port is located at port 0
APB Port is located at port 1
Double check device identification ...
Warning: Overwrite detected IDCODE
Setup non-secure transaction ...
Boot Info: 1.8V QSPI Flash
Clock Select: 0
Start HPS Quad SPI flash programming ...
Initialize QSPI peripheral and flash controller ...
Read Silicon ID of Quad SPI flash ...
Quad SPI Flash silicon ID is 0x1021B820
Flash device matched
Manufacturer: MICRON
Device: QSPI_1024
Enable Four Byte Addressing ...
Sector Erase Quad SPI flash ...
Sector Erase Info: Start Addr at 0x00120000 for 10 sector(s)
Sector Erase Quad SPI Flash at 0x00120000
Sector Erase Quad SPI Flash at 0x00130000
Sector Erase Quad SPI Flash at 0x00140000
Sector Erase Quad SPI Flash at 0x00150000
Sector Erase Quad SPI Flash at 0x00160000
Sector Erase Quad SPI Flash at 0x00170000
Sector Erase Quad SPI Flash at 0x00180000
Sector Erase Quad SPI Flash at 0x00190000
Sector Erase Quad SPI Flash at 0x001A0000
Sector Erase Quad SPI Flash at 0x001B0000
Program Quad SPI flash ...
Info: Quartus II 64-Bit Programmer was successful. 0 errors, 0 warnings
Info: Peak virtual memory: 187 megabytes
Info: Processing ended: Fri Jun 5 18:11:42 2015
Info: Elapsed time: 00:02:05
Info: Total CPU time (on all processors): 00:00:11
[05/06/2015 - 18:11:42] [imwong.pg-icc0313]
/data/imwong/sdcard/a10_soc
$
```

1.5.5.5.3. Programming the VxWorks* Real-Time Operating System Image

1. Go to your VxWorks* image directory. The directory used in this example is: /data/<username>/sdcard/a10_soc.

```
$ cd /data/<username>/sdcard/a10_soc
```

2. Add the mkimage header to your vxWorks.bin to your to indicate the filesize.

```
$ mkimage -A arm -T firmware -C none -a 0x100000 -e 0x100000 -n "vxWorks" -d vxWorks.bin vxWorks.bin.mkimage
```

3. Flash the boot image into the QSPI by typing:

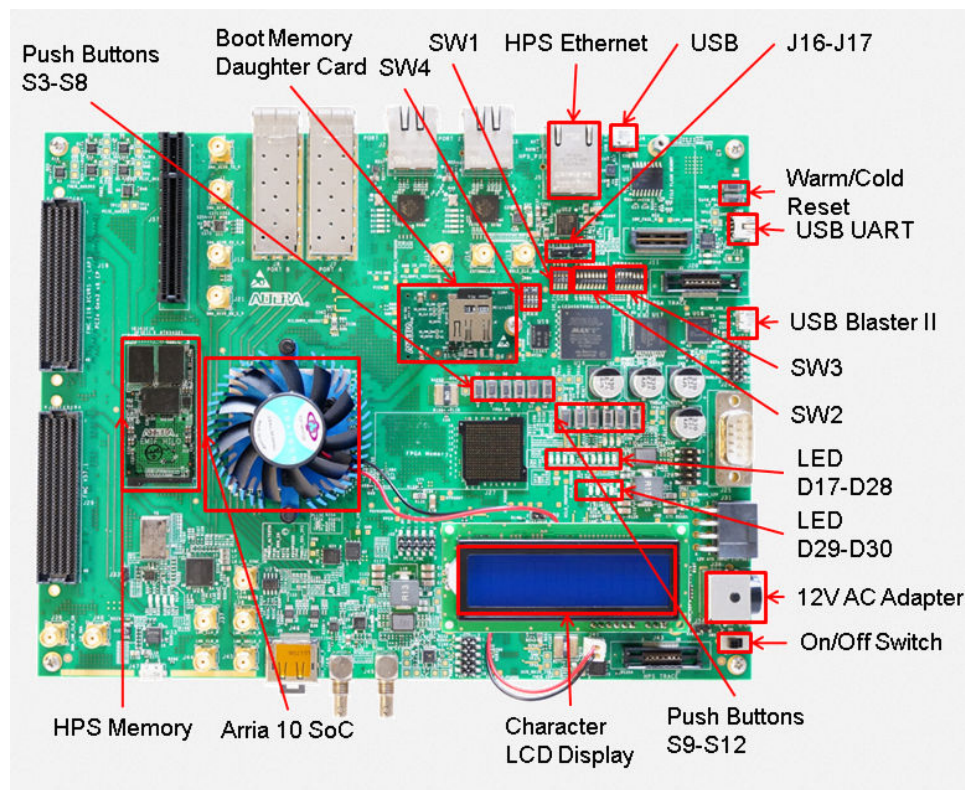
```
quartus_hps -c 1 -d 2 --o P --addr=0x00120000 vxWorks.bin.mkimage
```

1.5.6. Booting the Board with SD/MMC

1.5.6.1. Configuring the Board

This section presents the necessary board settings for running the UEFI boot loader on the Intel Arria 10 SoC FPGA development board.

Figure 63. Intel Arria 10 SoC FPGA Development Board



Confirm that the memories are installed in the following way:

- A DDR4 memory card is installed on the HPS memory socket.
- An SD boot card is installed on the boot flash socket.

Confirm that the board switches are configured as in the tables below:

Table 9. Switch Configuration

Switch Type	Required Setting	Function
SW1 Settings		
SW1.1 – I2C flag	OFF	ON (0): System MAX5 is the I ² C master. OFF (1): HPS is the I ² C master.
RevA.1 and RevA.2 Board		
SW1.2 – factory_load	ON	ON (0): Load the user design from flash at power up. OFF (1): Load the factory design from flash at power up.
SW1.3 – msel1	ON	ON (up): MSEL [1] is 0. OFF (down): MSEL [1] is 1.
SW1.4 – msel0	ON	ON (up): MSEL [0] is 0. OFF (down): MSEL [0] is 1.
RevB.1 Board		
SW1.2 – dc_power_ctrl	ON	ON (0): Do not turn on FMC/PCIE power directly. OFF (1): Turn on FMC/PCIE power directly.
SW1.3 – factory_load	ON	ON (0): Load the user design from flash at power up. OFF (1): Load the factory design from flash at power up.
SW1.4 – security_mode	ON	Reserved
SW2 Settings : ALL ON		
SW3 Settings		
SW3.1 – Arria10	OFF	ON – Arria10 JTAG Bypass OFF – Arria10 JTAG Enable
SW3.2 – IO MAXV	ON	ON – MAXV JTAG Bypass OFF – MAXV JTAG Enable
SW3.3 – FMCA	ON	ON – FMCA JTAG Bypass OFF – FMCA JTAG Enable
SW3.4 – FMCB	ON	ON – FMCB JTAG Bypass OFF – FMCB JTAG Enable
SW3.5 – PCIE	ON	ON – PCIE JTAG Bypass OFF – PCIE JTAG Enable
SW3.6 – MSTR0	OFF	See "Master Switch Settings" Table
SW3.7 – MSTR1	OFF	
SW3.8 – MSTR2	OFF	
SW4 Settings: ALL OFF (RevA.1 & RevA.2 Board)		
SW4 Settings: (RevB.1 Board)		
SW4.1 – reserve	OFF	-
SW4.2 – MSEL0	ON	ON=1 OFF=0
SW4.3 – MSEL1	OFF	
SW4.4 – MSEL2	OFF	

Table 10. Master Switch Settings

MSTR[2]	MSTR[1]	MSTR[0]	Master Selected
ON	ON	ON	Boot
OFF	ON	ON	FMCA (not implemented yet)
ON	OFF	ON	FMCB (not implemented yet)
ON	ON	OFF	FTRACE
OFF	OFF	OFF	ON-BOARD UBII
ON	OFF	OFF	MAXV A Programming Mode (only for advanced user)
OFF	ON	OFF	GUI Mode

Confirm that the following jumpers are configured as follows:

- Place jumpers on J16 and J17
- Leave all other jumpers unplaced

1.5.6.2. Configuring the Serial Connection

The Intel Arria 10 SoC FPGA development board has a built-in FTDI USB-to-serial converter chip that allows the host computer to see the board as a virtual serial port. CentOS 6.5 has built-in drivers for the FTDI USB-to-serial converter chip. Therefore, you do not need to install a driver. In Windows, the SoC FPGA EDS installation automatically installs the required drivers if necessary.

The serial communication parameters listed below:

- Baud rate: 115200 bps
- Parity: none
- Flow control: none
- Stop bits: 1
- Data bits: 8

In Windows, a terminal emulator such as Tera Term can be used to connect to the Intel Arria 10 SoC FPGA development board. Refer to the *Tera Term Installation* section for the instructions on how to install Term Term and how to configure serial port settings from the tool menu.

You can use the Linux minicom utility to connect to the Intel Arria 10 SoC FPGA development board. Refer to the *Minicom Installation* section for instructions on how to install the minicom on CentOS and Ubuntu and how to configure the serial port settings from the tool menu.

Related Information

- [Tera Term Installation](#) on page 127
- [Minicom Installation](#) on page 136

1.5.6.3. Booting a Bare Metal Application from SD/MMC

1. Complete the steps in [Creating an SD Card Image on Linux](#) on page 38 or [Creating an SD Card Image on Windows](#) on page 40 depending on your OS.
2. Complete the corresponding steps in [Compile the Bare Metal Application Source Code](#) on page 41 and [Programming the Bare Metal Image](#) on page 47.
3. Insert the programmed SD/MMC card into J23.
4. Start a serial terminal program.
5. Power up the board. During this time, there is no console output for approximately 15 seconds.
6. For a normal boot flow, you see a series of output messages on your serial terminal program ending with the following completion message:

Figure 64. BareMetal Boot Completion Message Output

```
Default:
USB Blaster: set SW3 DIP switch <off on on on on off off off>
Use BSTREAM: set SW3 DIP switch <off on on on on on on on >

Booting...
Opening file:
  Filename : bootimage.bin
  Date    : 2015-04-02
  Time    : 15:22:10
  Size    : 192 Bytes
Copying file bootimage.bin to RAM at 0x00100000
Done.
Control transferred to 0x00100000 with R0 = 0x00000000 R1 = 0x00000000 R2 = 0x00000000

Hello World from Baremetal Application!
```

7. If you are in DXE phase, the following output message displays on your serial terminal program for post-DDR extended diagnostic boot flow.

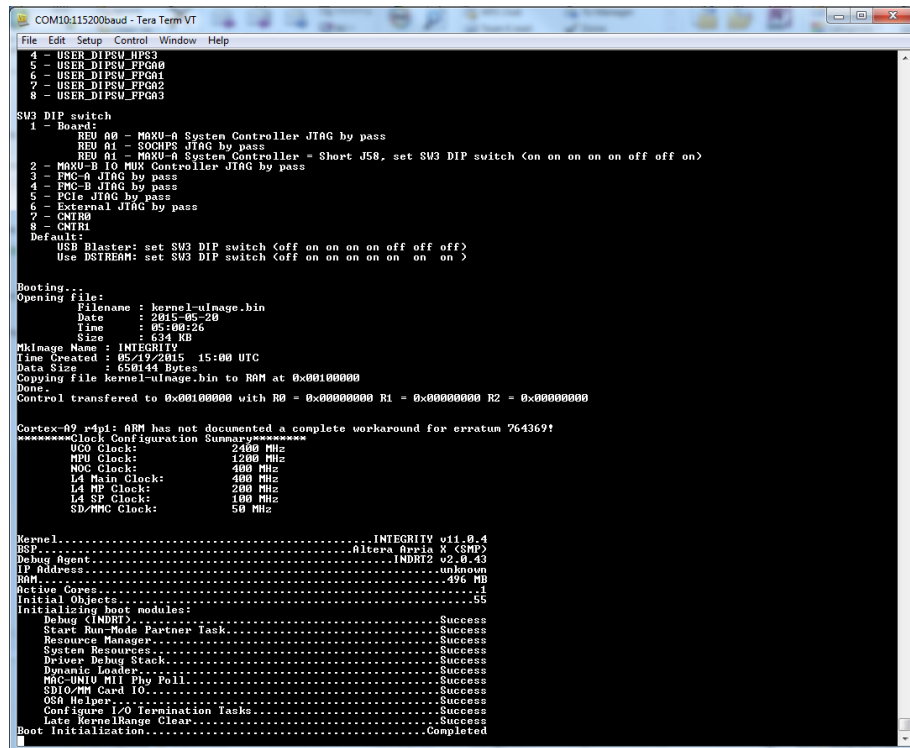
Figure 65. DXE Phase Output Message

```
COM11 - PuTTY
BSEL is set to SD/MMC
Boot Flash's dedicated IO Pin is Mux to 1,4 bit bus mode SD/MMC
Boot Source Type = SD/MMC based on BSEL
Opening file:
  Filename : DXE.ROM
  Date    : 2000-01-01
  Time    : 00:00:00
  Size    : 512 KB
Copying DXE.ROM from Flash to RAM at 0x02000000
Done.
DXE Core entry at 0x010002B5
  SDMMC: InitializeSdMmc ()
  SDMMC: EnablePowerToTheCard () A10 Dev Kit Board Workaround.
  SDMMC: ChangeCardClockFrequency ( 0x02 )
  SDMMC: ChangeCardClockFrequency ( 0x01 )
The default boot selection will start in 10 seconds
[1] Shell
[2] Boot Manager
```

1.5.6.4. Booting Integrity Real-Time Operating System from SD/MMC

1. Complete the steps in [Creating an SD Card Image on Linux](#) on page 38 section or [Creating an SD Card Image on Windows](#) on page 40 depending on your OS.
2. Complete the corresponding steps in [Programming the Integrity Real-Time Operating System Image](#) on page 50.
3. Insert the programmed SD/MMC daughter card into J23.
4. Start a serial terminal program.
5. Power up the board. During this time, there is no console output for approximately 15 seconds.
6. For a normal boot flow, you see a series of output messages on your serial terminal program ending with the following completion message:

Figure 66. Integrity RTOS Boot Completion Message Output



```
COM10115200baud - Tera Term VT
File Edit Setup Control Window Help

5 - USER_D1PSU_IPF3
6 - USER_D1PSU_FPGA0
7 - USER_D1PSU_FPGA1
8 - USER_D1PSU_FPGA2
8 - USER_D1PSU_FPGA3

SW3 DIP switch
1 - Board:
  REU A0 - MAXU-A System Controller JTAG by pass
  REU A1 - SOCHPS JTAG by pass
  REU A1 - MAXU-A System Controller - Short J58, set SW3 DIP switch <on on on on off off on>
2 - MAXU-B 10Mhz Controller JTAG by pass
3 - FMC-A JTAG by pass
4 - FMC-B JTAG by pass
5 - PCIe JTAG by pass
6 - External JTAG by pass
7 - CNTR0
8 - CNTR1
Default:
USB Blaster: set SW3 DIP switch <off on on on on on off off off>
Use DSTREAM: set SW3 DIP switch <off on on on on on on on on>

Booting...
Opening File:
  Filename : kernel-ultimate.bin
  Date : 2015-05-20
  Time : 05:00:26
  Size : 634 KB
MkImage Name : INTEGRITY
Time Created : 05/19/2015 15:00 UTC
Data Size : 650144 Bytes
Copying File kernel-ultimate.bin to RAM at 0x00100000
Done.
Control transferred to 0x00100000 with R0 = 0x00000000 R1 = 0x00000000 R2 = 0x00000000

Cortex-A9 ->api: ARM has not documented a complete workaround for erratum 764369!
*****Clock Configuration Summary*****
UC0 Clock: 2400 MHz
MPU Clock: 1200 MHz
MCC Clock: 400 MHz
L4 Main Clock: 400 MHz
L4 AP Clock: 200 MHz
L4 SF Clock: 100 MHz
SD/MMC Clock: 50 MHz

Kernel.....INTEGRITY v11.0.4
DSP.....Altera Arria X (SMP)
Debug Agent.....INDRT2 v2.0.43
IP Address.....unknown
RAM.....496 MB
Active Cores.....1
Initial Objects.....55
Initializing boot modules:
  Debug (INDRT2).....Success
  Start Run-Mode Partition Task.....Success
  Resource Manager.....Success
  System Resources.....Success
  Destroy Debug Stack.....Success
  Dynamic Loader.....Success
  MHC-UNIV M11 Phy Poll.....Success
  SDIO/MM Card IO.....Success
  OSR Helper.....Success
  Configure I/O termination tasks.....Success
  Late KernelRange Clear.....Success
Boot Initialization.....Completed
```

1.5.6.5. Booting from VxWorks Real-Time Operating System from SD/MMC

1. Complete the steps in [Creating an SD Card Image on Linux](#) on page 38 section or [Creating an SD Card Image on Windows](#) on page 40 section depending on your OS.
2. Complete the corresponding steps in [Programming the VxWorks* Real-Time Operating System Image](#) on page 53.
3. Insert the programmed SD/MMC daughter card into J23.
4. Start a serial terminal program.
5. Power up the board. During this time, there is no console output for approximately 15 seconds.
6. For a normal boot flow, you see a series of output messages on your serial terminal program ending with the following completion message:

COM10:115200baud - Tera Term VT

File Edit Setup Control Window Help

OK.

```

probe sdStorage
attach sdStorage:0

UxWorks 7 SMP

Core Kernel version: 1.0.10.1
Build date: May 15 2015 14:59:56

Copyright Wind River Systems, Inc.
1984-2015

Board: Altera SOCFFPGA Arria 10 Dev Kit
CPU Count: 2
OS Memory Size: 71023MB
EDRR Policy Mode: Deployed
Debug Agent: Started (always)
Stop Mode Agent: Started (always)

Adding 10074 symbols for standalone.
-> Instantiating <sd0:1 as rawFs, device = 0x40001
Instantiating <sd0:2 as rawFs, device = 0x50001

```

1.5.7.1. Configuring the Board

- A DDR4 memory card is installed on the HPS memory socket.
- A QSPI daughter card is installed on the boot flash socket.

Table 11. Switch Configuration

Switch Type	Required Setting	Function
SW1 Settings		
SW1.1 – I2C flag	OFF	ON (0): System MAX5 is the I ² C master. OFF (1): HPS is the I ² C master.
RevA.1 and RevA.2 Board		
SW1.2 – factory_load	ON	ON (0): Load the user design from flash at power up. OFF (1): Load the factory design from flash at power up.
SW1.3 – msel1	ON	ON (up): MSEL [1] is 0. OFF (down): MSEL [1] is 1.
SW1.4 – msel0	ON	ON (up): MSEL [0] is 0. OFF (down): MSEL [0] is 1.
RevB.1 Board		
SW1.2 – dc_power_ctrl	ON	ON (0): Do not turn on FMC/PCIE power directly. OFF (1): Turn on FMC/PCIE power directly.
SW1.3 – factory_load	ON	ON (0): Load the user design from flash at power up. OFF (1): Load the factory design from flash at power up.
SW1.4 – security_mode	ON	Reserved
SW2 Settings : ALL ON		

continued...

Switch Type	Required Setting	Function
SW3 Settings		
SW3.1 – Arria10	OFF	ON – Arria10 JTAG Bypass OFF – Arria10 JTAG Enable
SW3.2 – IO MAXV	ON	ON – MAXV JTAG Bypass OFF – MAXV JTAG Enable
SW3.3 – FMCA	ON	ON – FMCA JTAG Bypass OFF – FMCA JTAG Enable
SW3.4 – FMCB	ON	ON – FMCB JTAG Bypass OFF – FMCB JTAG Enable
SW3.5 – PCIE	ON	ON – PCIE JTAG Bypass OFF – PCIE JTAG Enable
SW3.6 – MSTR0	OFF	See "Master Switch Settings" Table
SW3.7 – MSTR1	OFF	
SW3.8 – MSTR2	OFF	
SW4 Settings: ALL OFF (RevA.1 & RevA.2 Board)		
SW4 Settings: (RevB.1 Board)		
SW4.1 – reserve	OFF	-
SW4.2 – MSEL0	ON	ON=1 OFF=0
SW4.3 – MSEL1	OFF	
SW4.4 – MSEL2	OFF	

Table 12. Master Switch Settings

MSTR[2]	MSTR[1]	MSTR[0]	Master Selected
ON	ON	ON	Boot
OFF	ON	ON	FMCA (not implemented yet)
ON	OFF	ON	FMCB (not implemented yet)
ON	ON	OFF	FTRACE
OFF	OFF	OFF	ON-BOARD UBII
ON	OFF	OFF	MAXV A Programming Mode (only for advanced user)
OFF	ON	OFF	GUI Mode

Confirm that the following jumpers are configured as follows:

- Place jumpers on J16 and J17
- Leave all other jumpers unplaced

1.5.7.2. Configuring the Serial Connection

The Intel Arria 10 SoC FPGA development board has a built-in FTDI USB-to-serial converter chip that allows the host computer to see the board as a virtual serial port. CentOS 6.5 has built-in drivers for the FTDI USB-to-serial converter chip. Therefore, you do not need to install a driver. In Windows, the SoC FPGA EDS installation automatically installs the required drivers if necessary.

The serial communication parameters listed below:

- Baud rate: 115200 bps
- Parity: none
- Flow control: none
- Stop bits: 1
- Data bits: 8

In Windows, a terminal emulator such as Tera Term can be used to connect to the Intel Arria 10 SoC FPGA development board. Refer to the *Tera Term Installation* section for the instructions on how to install Term Term and how to configure serial port settings from the tool menu.

You can use the Linux minicom utility to connect to the Intel Arria 10 SoC FPGA development board. Refer to the *Minicom Installation* section for instructions on how to install the minicom on CentOS and Ubuntu and how to configure the serial port settings from the tool menu.

Related Information

- [Tera Term Installation](#) on page 127
- [Minicom Installation](#) on page 136

1.5.7.3. Booting a Bare Metal Application from QSPI

This task describes how to use a UEFI boot loader to boot the HelloWorld Bare Metal application on the board.

1. Complete the steps in [Updating the Golden Hardware Reference Design to Boot from QSPI](#) on page 57, [Compiling the Hardware and Software for QSPI](#) on page 57 and [Compiling the UEFI Source Code with the Updated QSPI Device Tree Blob](#) on page 58.
2. Complete the steps in [Programming the Bare Metal Image](#) on page 61.
3. Insert the programmed QSPI daughter card into J23.
4. Start a serial terminal program.
5. Power up the board. During this time, there is no console output for approximately 15 seconds.
6. For a normal boot flow, you see a series of output messages on your serial terminal program ending with the following completion message:

Figure 68. Bare Metal Boot Completion Message Output

```

COM10115200baud - Tera Term VT
File Edit Setup Control Window Help

J57 - PCIe x8 Connector
J5 - RGMI 10/100/1000 Ethernet port (PHY FN: KS29031RNKCR)
J2&J3 - GMII Gigabit Ethernet ports (PHY FN: 88E1111-B2-NDGCG800)
J7&J8 - Optical (SFP) Transceiver Cage & Connector:

J23 - Flash Card options:
1 - NAND Flash(x8) : 128MB (MT29F1G08ABBEAH4)
2 - QSPI Flash : 128MB (MT25QU01GDBN8E12-0S1T)
3 - SD Micro card : 4GB (Kingston)

J27 - FPGA memory H1LO Card options:
1 - 2GB DDR4 (256Mb x 72 x 1 rank)
2 - 2GB DDR3 (256Mb x 72 x 1 rank)
3 - 4GB DDR3 (256Mb x 72 x 2 ranks)
4 - 16MB QDDR (4Mb x 36)
5 - 128MB RLDRAM3 (16Mb x 72)

J26 - HPS memory H1LO Card options:
1 - 1GB DDR4 (256Mb x 40 x 1 Rank)
2 - 1GB DDR3 (256Mb x 40 x 1 Rank)
3 - 2GB DDR3 (256Mb x 40 x 2 Ranks)

SV1 DIP switch
1 - I2C flag
2 - FACTORY_LOAD
3 - MSEL pins 1
4 - MSEL pins 0

SV2 DIP switch
1 - USER_DIPSU_HPS0
2 - USER_DIPSU_HPS1
3 - USER_DIPSU_HPS2
4 - USER_DIPSU_HPS3
5 - USER_DIPSU_FPGA0
6 - USER_DIPSU_FPGA1
7 - USER_DIPSU_FPGA2
8 - USER_DIPSU_FPGA3

SV3 DIP switch
1 - Board:
REV A0 - MAXU-A System Controller JTAG by pass
REV A1 - SOCHPS JTAG by pass
REV A1 - MAXU-A System Controller - Short J58, set SV3 DIP switch (on on on on off off on)
2 - MAXU-B IO MUX Controller JTAG by pass
3 - FMC-A JTAG by pass
4 - FMC-B JTAG by pass
5 - PCIe JTAG by pass
6 - External JTAG by pass
7 - CNTR0
8 - CNTR1
Default:
USB Blaster: set SV3 DIP switch (off on on on on off off off)
Use DSTREAM: set SV3 DIP switch (off on on on on on on on)

Booting...
Mkimage Name : RBF
Time created : 06/05/2015 09:29 UTC
Data Size : 204 Bytes
Copying Bootimage from Flash Offset 0x00120040 to Memory Address 0x00100000 where Bootimage's size is 0x000000CC bytes
Control transfered to 0x00100000 with R0 = 0x00000000 R1 = 0x00000000 R2 = 0x00000000

Hello World from Baremetal Application!

Execution stopped at: 0x00100008 due to application reaching end of execution
0
  
```

7. If you are in DXE phase, the following output message displays on your serial terminal program for post-DDR extended diagnostic boot flow.

Figure 69. DXE Phase Output Message

```

COM11 - PuTTY

BSEL is set to SD/MMC
Boot Flash's dedicated IO Pin is Max to 1,4 bit bus mode SD/MMC
Boot Source Type = SD/MMC based on BSEL
Opening file:
  Filename : DXE.ROM
  Date : 2000-01-01
  Time : 00:00:00
  Size : 512 KB
Copying DXE.ROM from Flash to RAM at 0x02000000
Done.
DXE Core entry at 0x010002B5
  SDMMC: InitializeSdMmc ()
  SDMMC: EnablePowerToTheCard () A10 Dev Kit Board Workaround.
  SDMMC: ChangeCardClockFrequency ( 0x02 )
  SDMMC: ChangeCardClockFrequency ( 0x01 )
The default boot selection will start in 10 seconds
[1] Shell
[2] Boot Manager
  
```

1.5.7.4. Booting Integrity Real-Time Operating System from QSPI

1. Complete the steps in [Updating the Golden Hardware Reference Design to Boot from QSPI](#) on page 57, [Compiling the Hardware and Software for QSPI](#) on page 57 and [Compiling the UEFI Source Code with the Updated QSPI Device Tree Blob](#) on page 58.
2. Complete the steps in the [Programming the Integrity Real-Time Operating System Image](#) on page 62 section.
3. Insert the programmed QSPI daughter card into J23.

4. Start a serial terminal program.
5. Power up the board. During this time, there is no console output for approximately 15 seconds.
6. A series of output messages display on your serial terminal program ending with the following completion message:

Figure 70. Integrity RTOS Boot Completion Message Output

```

COM10:115200baud - Tera Term VT
File Edit Setup Control Window Help

SU2 DIP switch
1 - USER_DIPSU_HPS0
2 - USER_DIPSU_HPS1
3 - USER_DIPSU_HPS2
4 - USER_DIPSU_HPS3
5 - USER_DIPSU_FPGA0
6 - USER_DIPSU_FPGA1
7 - USER_DIPSU_FPGA2
8 - USER_DIPSU_FPGA3

SU3 DIP switch
1 - Board:
  REV A0 - MAXU-A System Controller JTAG by pass
  REV A1 - SOCMP2 JTAG by pass
  REV A1 - MAXU-A System Controller - Short J58, set SW3 DIP switch (on on on on off off on)
2 - MAXU-B IO MUX Controller JTAG by pass
3 - FMC-A JTAG by pass
4 - FMC-B JTAG by pass
5 - PCIe JTAG by pass
6 - External JTAG by pass
7 - CNTR0
8 - CNTR1
Default:
USB Blaster: set SW3 DIP switch (off on on on on off off off)
Use DEFBUS: set SW3 DIP switch (off on on on on on on on)

Booting...
Image Name : INTEGRITY
Time Created : 05/19/2015 15:00 UTC
Data Size : 650144 Bytes
Copying BootImage from Flash Offset 0x00120040 to Memory Address 0x00100000 where BootImage's size is 0x0009EB00 bytes
Control transferred to 0x00100000 with R0 = 0x00000000 R1 = 0x00000000 R2 = 0x00000000

Cortex-A9 r4pi2 ARM has not documented a complete workaround for erratum 764367!
*****Clock Configuration Summary*****
UC0 Clock: 2400 MHz
M0 Clock: 1200 MHz
M0C Clock: 400 MHz
L4 Main Clock: 400 MHz
L4 PE Clock: 200 MHz
L4 SP Clock: 100 MHz
SD/MMC Clock: 50 MHz

Kernel.....INTEGRITY v1.0.4
DSP.....Altera Arria 10 (SMP)
Debug Agent.....INDR2 v2.0.43
IP Address.....unknown
RAM.....496 MB
Active Gates.....1
Initial Objects.....155
Initializing boot modules.....
Debug (INDRT).....Success
Start Run-Mode Partner Task.....Success
Resource Manager.....Success
System Resources.....Success
Driver Debug Stack.....Success
Dynamic Loader.....Success
M0C-DMU M11 By Poll.....Success
SDIO/MM Card IO.....Success
OSD Helper.....Success
Configure I/O Termination Fuses.....Success
Late KernelRange Clear.....Success
Boot Initialization.....Completed
  
```

1.5.7.5. Booting from VxWorks Real-Time Operating System from QSPI

1. Complete the steps in [Updating the Golden Hardware Reference Design to Boot from QSPI](#) on page 57, [Compiling the Hardware and Software for QSPI](#) on page 57 and [Compiling the UEFI Source Code with the Updated QSPI Device Tree Blob](#) on page 58.
2. Complete the steps in [Programming the VxWorks* Real-Time Operating System Image](#) on page 62.
3. Insert the programmed QSPI daughter card into J23.
4. Start a serial terminal program.
5. Power up the board. During this time, there is no console output for approximately 15 seconds.
6. A series of output messages display on your serial terminal program ending with the following completion message:

Figure 71. VxWorks RTOS Boot Completion Message Output

1.5.8. Early I/O Release

Task time: 30 minutes

Early I/O release enables DDR functioning prior to programming the core raw binary file (RBF). This feature provides a faster boot time because the shared I/O and hard memory controller I/O are configured and released for immediate access by the HPS. Early I/O release also enables availability of the shared I/O so that device can obtain the core RBF from the TFTP server instead of flash. The CSS then configures and releases the FPGA fabric allowing the HPS to have access. Early I/O release also provides early response in time critical systems.

The peripheral RBF configures shared I/O, hard memory controller I/O, and hard memory controller settings such as memory type, frequency and timings. The core RBF configures the FPGA fabric, FPGA I/O and FPGA PLL settings. You can choose to combine the peripheral and core RBF into one RBF that configures both the FPGA fabric and the I/O Control Shift registers (IOCSRs). Using a combined RBF creates a performance penalty that is much slower than splitting the RBFs. When you split the RBFs, the core RBF loads into SDRAM instead of on-chip RAM from flash providing a larger data transfer block and boosting performance.

1.5.8.1. Boot Flow

After device power up, the peripheral RBF loads into on-chip RAM from flash. The configuration subsystem (CSS) uses the peripheral RBF to configure the IOCSRs. Once the configuration is done, and the FPGA enters early user mode and the shared I/O exit reset. At this point, the SDRAM is ready for loading a larger size image. Subsequently, the core RBF is loaded into SDRAM. The CSS uses the core RBF to

configure the FPGA fabric and FPGA I/O. Once the configuration finishes, the FPGA enters user mode, the CSS releases the FPGA I/O from reset, and the device can now boot to DXE or OS.

You can load the core RBF automatically upon power up or manually through the Pit Stop command. When you load the core RBF automatically, the core RBF file name (SDMMC) or offset (QSPI/NAND) are defined in the modifiable DSC file. When you load the core RBF manually, you must key in the core RBF file name or offset using the Pit Stop utility.

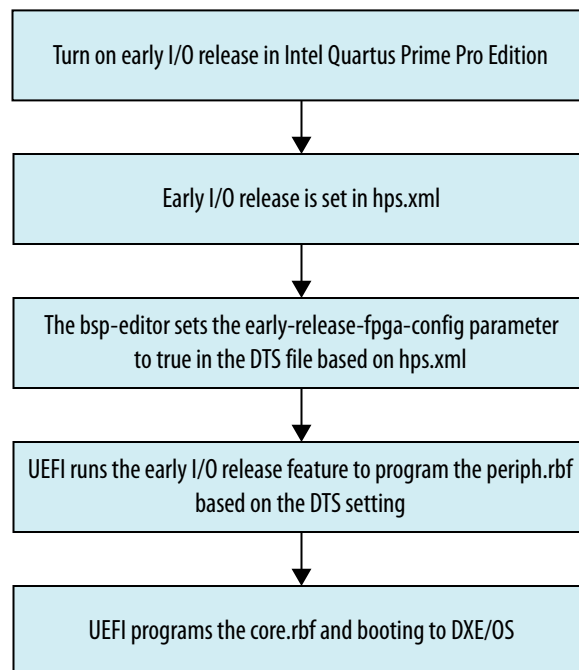
There are three boot flows supported by current UEFI:

- An automatically loaded peripheral RBF and core RBF
- An automatically loaded peripheral RBF and manually loaded core RBF through Pit Stop
- An automatically loaded and combined RBF

1.5.8.2. Enabling Early I/O Release

The figure below shows the high-level steps to enable early I/O release.

Figure 72. Early I/O Release Enable



1.5.8.2.1. Automatically Loaded Peripheral RBF and Core RBF

1. To automatically load the peripheral RBF and core RBF, use your preferred editor to open the <your_uefi_socfpga>/uefi-socfpga/AlteraPlatformPkg/Arria10SoCPkg/Arria10SoCPkg.dsc file.
2. Configure the gAlteraSocFpgaTokenSpaceGuid.PcdAutoProgramCoreRbf value to 1.

Figure 73. Arria10SoCPkg.dsc Parameter Edit

```
191  ## Early IO Release - Peripheral RBF is programmed first and Core RBF is programmed auto / manually later
192  gAlteraSocFpgaTokenSpaceGuid.PcdAutoProgramCoreRbf |1
193  gAlteraSocFpgaTokenSpaceGuid.PcdFileName_CORE_RBF |"ghrd_10as066n2.core.rbf"
194  gAlteraSocFpgaTokenSpaceGuid.PcdQspiOrNand_CORE_RBF_ADDR |0x00820000
195  gAlteraSocFpgaTokenSpaceGuid.PcdSdramBaseAddress_CORE_RBF |0x40
```

1.5.8.2.2. Automatically Loaded Peripheral RBF And Manually Loaded RBF Through Pit Stop

1. To automatically load the peripheral RBF and manually load core RBF, use your preferred editor to open the `<your_uefi_socfpga>/uefi-socfpga/AlteraPlatformPkg/Arria10SoCPkg/Arria10SoCPkg.dsc` file.
2. Configure the `gAlteraSocFpgaTokenSpaceGuid.PcdAutoProgramCoreRbf` value to 0.

Figure 74. Arria10SoCPkg.dsc Parameter Edit

```
191  ## Early IO Release - Peripheral RBF is programmed first and Core RBF is programmed auto / manually later
192  gAlteraSocFpgaTokenSpaceGuid.PcdAutoProgramCoreRbf |0
193  gAlteraSocFpgaTokenSpaceGuid.PcdFileName_CORE_RBF |"ghrd_10as066n2.core.rbf"
194  gAlteraSocFpgaTokenSpaceGuid.PcdQspiOrNand_CORE_RBF_ADDR |0x00820000
195  gAlteraSocFpgaTokenSpaceGuid.PcdSdramBaseAddress_CORE_RBF |0x40
196
```

Refer to the Pit Stop Usage Guide section for the program core RBF command to load the core RBF through the Pit Stop utility manually.

1.5.8.2.3. Automatically Loaded Combined RBF

1. To automatically load the combined RBF, use your preferred editor to open the `<your_uefi_socfpga>/uefi-socfpga/AlteraPlatformPkg/Arria10SoCPkg/Arria10SoCPkg.dsc` file.
2. Configure the `gAlteraSocFpgaTokenSpaceGuid.PcdAutoProgramCoreRbf` value to 1.

Figure 75. Arria10SoCPkg.dsc Parameter Edit

```
191  ## Early IO Release - Peripheral RBF is programmed first and Core RBF is programmed auto / manually later
192  gAlteraSocFpgaTokenSpaceGuid.PcdAutoProgramCoreRbf |1
193  gAlteraSocFpgaTokenSpaceGuid.PcdFileName_CORE_RBF |"ghrd_10as066n2.core.rbf"
194  gAlteraSocFpgaTokenSpaceGuid.PcdQspiOrNand_CORE_RBF_ADDR |0x00820000
195  gAlteraSocFpgaTokenSpaceGuid.PcdSdramBaseAddress_CORE_RBF |0x40
```

1.5.8.3. Device Tree Support

Figure 76. DTS Design for SD/MMC Booting

```
#address-cells = <1>;

#size-cells = <1>;

model = "SOCFPGA Arria10 Dev Kit";

chosen {

    cff-file = "ghrd_10as066n2.periph.rbf";

    early-release-fpga-config;

};
```

Figure 77. DTS Design for QSPI Booting

```
#address-cells = <1>;

#size-cells = <1>;

model = "SOCFPGA Arria10 Dev Kit";

chosen {
    cff_offset = <0x00720000>;
    early-release-fpga-config;
};
```

1.5.8.4. Early I/O Release Output Messages

The following messages display when you enable early I/O release and power up the board:

Figure 78. Early I/O Release Output Messages

```
COM16 - PuTTY
2016-06-07 16:36:38 1048576 DXE.ROM
2016-06-01 05:34:34 360456 GHRD_1~1.RBF ghrd_10as066n2.periph.rbf
f
2016-06-01 05:34:34 14656904 GHRD_1~2.RBF ghrd_10as066n2.core.rbf
2000-01-01 00:00:00 23812 GHRD_1~1.DTB ghrd_10as066n2.dtb
6 File(s) 19665560 bytes
Done Initializing SD/MMC controller
FPGA is not in User Mode
FDT: early IO release is enabled
FPGA Manager Info before configuration begin:
FPGA Manager dclk cnt: 0x00000000
FPGA Manager dclk stat: 0x00000000
FPGA Manager img cfg ctrl 00: 0x00000107
FPGA Manager img cfg ctrl 01: 0x01000001
FPGA Manager img cfg ctrl 02: 0x00000000
FPGA Manager img cfg fifo status: 0x00000200
FPGA Manager imgcfg_stat: 0x01001090
emr : 0
jtagm : 0
imgcfg_FifoFull : 0
imgcfg_FifoEmpty : 1
f2s_msel2 : 0
f2s_msel1 : 0
f2s_msel0 : 0
f2s_nceo_oe : 0
f2s_nconfig_pin : 1
f2s_pr_error : 0
f2s_pr_done : 0
f2s_pr_ready : 0
f2s_cvp_conf_done : 0
f2s_initdone_oe : 1
f2s_condone_pin : 0
f2s_nstatus_oe : 0
f2s_nstatus_pin : 1
f2s_initdone_oe : 0
f2s_usermode : 0
f2s_early_usermode : 0
f2s_crc_error : 0
Open peri/combined Rbf
FDT: RBF file 1 = ghrd_10as066n2.periph.rbf
Opening file:
Filename : ghrd_10as066n2.periph.rbf
```


1.5.9. Booting Linux Using the UEFI Boot Loader

Task time: 60 minutes

SoC FPGA EDS version 16.0 and newer supports booting of a Linux operating system on an Intel Arria 10 SoC FPGA using the UEFI boot loader.

1.5.9.1. Boot Linux Support

The following table shows the supported Linux boot flows.

Table 13. Supported Linux Boot Methods

No.	Boot Method	SD/MMC Supported?	QSPI Supported?
1	Boot the Linux zImage from the PEI phase automatically without user intervention.	Yes	Yes
2	Boot the Linux zImage from the PEI phase manually by using the Pit Stop utility.	Yes	Yes
3	Boot the Linux zImage from the DXE phase automatically without intervention.	Yes	No
4	Boot Linux zImage from DXE phase by typing command at UEFI shell.	Yes	No

1.5.9.2. Prerequisites

You must have the following items to boot Linux on the Intel Arria 10 SoC FPGA development board.

- Intel Arria 10 SoC FPGA development board
- SoC FPGA EDS version 16.0 or newer
- Latest UEFI boot loader source code (minimum tag: rel_socfpga_arria10_soceds_16.0)
- Pre-built Linux SD card image from [here](#).

Note: You must program the UEFI boot loader in your SD card. You can do this by referring to the *Creating an SD Card Image on Linux* section. Follow the steps in this section until you complete the instructions to use the dd utility to overwrite the Arria10_Linux_SDCard.img file with the generated PEI.ROM and write the image into the SD card.

Related Information

- [Creating an SD Card Image on Linux](#) on page 38
- [Creating an SD Card Image on Linux](#) on page 38

1.5.9.3. Booting Linux from the PEI Phase

1.5.9.3.1. Booting Linux from the PEI Phase Automatically

This task describes how to enable booting of Linux from the PEI phase automatically.

1. Use your preferred editor to open the `<your_uefi_socfpga>/uefi-socfpga/AlteraPlatformPkg/Arria10SoCPkg/Arria10SoCPkg.dsc` file.
2. To boot the Linux zImage stored on an SD card or in QSPI flash right after the UEFI PEI phase initialization, configure the `gAlteraSocFpgaTokenSpaceGuid.PcdBoot_LOAD_ZIMAGE_AT_PEI_PHASE` value to 1.
3. Configure the `gAlteraSocFpgaTokenSpaceGuid.PcdBoot_LOAD_UEFI_DXE_PHASE` value to 0.
4. Recompile the source code and update the SD card image with the modified `PEI.ROM` for the changes to take effect.
5. Power cycle the board and observe the following output messages on your serial terminal program:

Figure 79. Serial Terminal Output Message

[illegible]

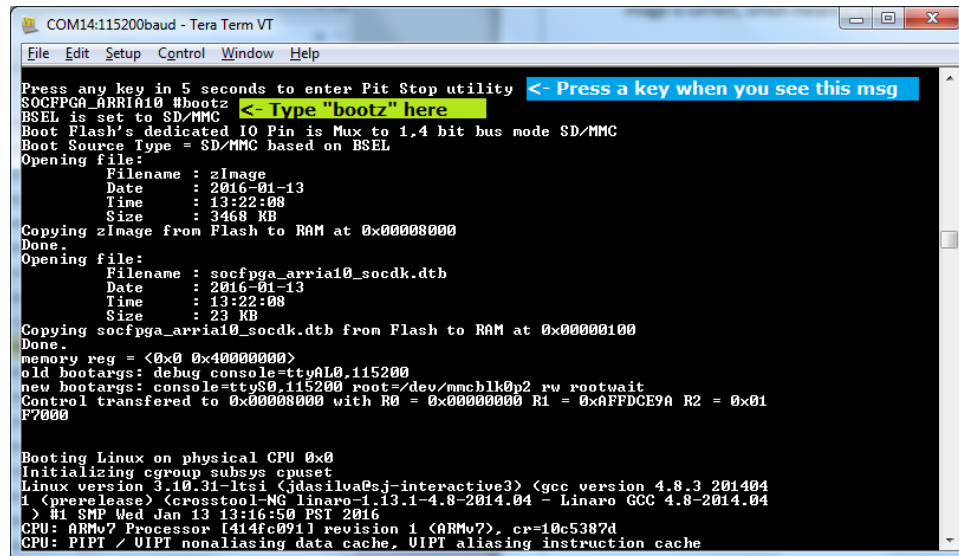
1.5.9.3.2. Booting Linux from the PEI Phase Using SD/MMC or QSPI Manually

If you are currently booting in another operating system and want to boot Linux from the PEI phase manually, execute the following instructions:

1. Boot to the Pit Stop utility. Refer to the *Enabling or Disabling the Pit Stop Utility* section for instructions.
2. Type the `bootz` command.

Note: This method does not require you to recompile the source code, but you must ensure the SD card image is correct. The SD card must have the zImage file inside the FAT32 partition and also have a Linux partition.

Figure 80. Boot Linux Manually Using SD/MMC



```
COM14:115200baud - Tera Term VT
File Edit Setup Control Window Help

Press any key in 5 seconds to enter Pit Stop utility <- Press a key when you see this msg
SOCFPGA_ARRIA10 #bootz <- Type "bootz" here
BSEL is set to SD/MMC
Boot Flash's dedicated IO Pin is Mux to 1.4 bit bus mode SD/MMC
Boot Source Type = SD/MMC based on BSEL
Opening file:
  Filename : zImage
  Date : 2016-01-13
  Time : 13:22:08
  Size : 3468 KB
Copying zImage from Flash to RAM at 0x00000000
Done.
Opening file:
  Filename : socfpga_arria10_socdk.dtb
  Date : 2016-01-13
  Time : 13:22:08
  Size : 23 KB
Copying socfpga_arria10_socdk.dtb from Flash to RAM at 0x00000100
Done.
memory reg = <0x0 0x40000000>
old bootargs: debug console=ttyAL0,115200
new bootargs: console=ttyS0,115200 root=/dev/mmcblk0p2 rw rootwait
Control transfered to 0x00000000 with R0 = 0x00000000 R1 = 0xAFFDCE9A R2 = 0x01F7000
F7000

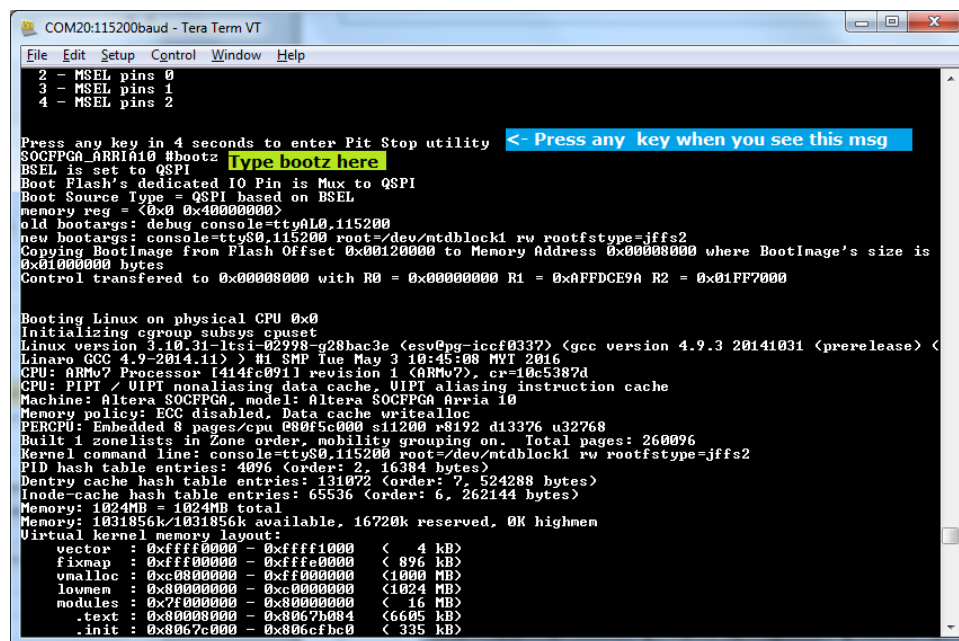
Booting Linux on physical CPU 0x0
Initializing cgroup subsys cpuset
Linux version 3.10.31-ltsi (kdasilva@sj-interactive3) (gcc version 4.8.3 201404
1 (prerelease) (cross-tool-NG linaro-1.13.1-4.8-2014.04 - Linaro GCC 4.8-2014.04
) #1 SMP Wed Jan 13 13:16:50 PST 2016
CPU: ARMv7 Processor [414fc091] revision 1 (ARMv7), cr=10c5387d
CPU: PIPT / UIPT nonaliasing data cache, UIPT aliasing instruction cache

2 - MSEL pins 0
3 - MSEL pins 1
4 - MSEL pins 2

Press any key in 4 seconds to enter Pit Stop utility <- Press any key when you see this msg
SOCFPGA_ARRIA10 #bootz Type bootz here
BSEL is set to QSPI
Boot Flash's dedicated IO Pin is Mux to QSPI
Boot Source Type = QSPI based on BSEL
memory reg = <0x0 0x40000000>
old bootargs: debug console=ttyAL0,115200
new bootargs: console=ttyS0,115200 root=/dev/mtdblock1 rw rootfstype=jffs2
Copying BootImage from Flash Offset 0x00120000 to Memory Address 0x00000000 where BootImage's size is
0x01000000 bytes
Control transfered to 0x00000000 with R0 = 0x00000000 R1 = 0xAFFDCE9A R2 = 0x01FF7000

Booting Linux on physical CPU 0x0
Initializing cgroup subsys cpuset
Linux version 3.10.31-ltsi-02998-g28bac3e (esv@pg-iccf0337) (gcc version 4.9.3 20141031 (prerelease) <
Linaro GCC 4.9-2014.11) #1 SMP Tue May 3 10:45:08 MYT 2016
CPU: ARMv7 Processor [414fc091] revision 1 (ARMv7), cr=10c5387d
CPU: PIPT / UIPT nonaliasing data cache, UIPT aliasing instruction cache
Machine: Altera SOCFPGA, model: Altera SOCFPGA Arria 10
Memory policy: ECC disabled, Data cache writealloc
PERCPU: Embedded 8 pages/cpu @80f5c000 s11200 r8192 d13376 u32768
Built 1 zonelists in Zone order, mobility grouping on. Total pages: 260096
Kernel command line: console=ttyS0,115200 root=/dev/mtdblock1 rw rootfstype=jffs2
PID hash table entries: 4096 (order: 2, 16384 bytes)
Dentry cache hash table entries: 131072 (order: 7, 524288 bytes)
Inode-cache hash table entries: 65536 (order: 6, 262144 bytes)
Memory: 1024MB = 1024MB total
Memory: 1031856k/1031856k available, 16720k reserved, 0K highmem
Virtual kernel memory layout:
  vector : 0xffff0000 - 0xffff1000 ( 4 kB)
  fixmap : 0xffff0000 - 0xffffe000 ( 896 kB)
  unalloc : 0xc0000000 - 0xffff0000 (1000 MB)
  lowmem : 0x80000000 - 0xc0000000 (1024 MB)
  modules : 0x7f000000 - 0x80000000 ( 16 MB)
  .text : 0x80000000 - 0x8067b004 (6605 kB)
  .init : 0x8067c000 - 0x806cfbc0 ( 335 kB)
```

Figure 81. Boot Linux Manually Using QSPI



```
COM20:115200baud - Tera Term VT
File Edit Setup Control Window Help

2 - MSEL pins 0
3 - MSEL pins 1
4 - MSEL pins 2

Press any key in 4 seconds to enter Pit Stop utility <- Press any key when you see this msg
SOCFPGA_ARRIA10 #bootz Type bootz here
BSEL is set to QSPI
Boot Flash's dedicated IO Pin is Mux to QSPI
Boot Source Type = QSPI based on BSEL
memory reg = <0x0 0x40000000>
old bootargs: debug console=ttyAL0,115200
new bootargs: console=ttyS0,115200 root=/dev/mtdblock1 rw rootfstype=jffs2
Copying BootImage from Flash Offset 0x00120000 to Memory Address 0x00000000 where BootImage's size is
0x01000000 bytes
Control transfered to 0x00000000 with R0 = 0x00000000 R1 = 0xAFFDCE9A R2 = 0x01FF7000

Booting Linux on physical CPU 0x0
Initializing cgroup subsys cpuset
Linux version 3.10.31-ltsi-02998-g28bac3e (esv@pg-iccf0337) (gcc version 4.9.3 20141031 (prerelease) <
Linaro GCC 4.9-2014.11) #1 SMP Tue May 3 10:45:08 MYT 2016
CPU: ARMv7 Processor [414fc091] revision 1 (ARMv7), cr=10c5387d
CPU: PIPT / UIPT nonaliasing data cache, UIPT aliasing instruction cache
Machine: Altera SOCFPGA, model: Altera SOCFPGA Arria 10
Memory policy: ECC disabled, Data cache writealloc
PERCPU: Embedded 8 pages/cpu @80f5c000 s11200 r8192 d13376 u32768
Built 1 zonelists in Zone order, mobility grouping on. Total pages: 260096
Kernel command line: console=ttyS0,115200 root=/dev/mtdblock1 rw rootfstype=jffs2
PID hash table entries: 4096 (order: 2, 16384 bytes)
Dentry cache hash table entries: 131072 (order: 7, 524288 bytes)
Inode-cache hash table entries: 65536 (order: 6, 262144 bytes)
Memory: 1024MB = 1024MB total
Memory: 1031856k/1031856k available, 16720k reserved, 0K highmem
Virtual kernel memory layout:
  vector : 0xffff0000 - 0xffff1000 ( 4 kB)
  fixmap : 0xffff0000 - 0xffffe000 ( 896 kB)
  unalloc : 0xc0000000 - 0xffff0000 (1000 MB)
  lowmem : 0x80000000 - 0xc0000000 (1024 MB)
  modules : 0x7f000000 - 0x80000000 ( 16 MB)
  .text : 0x80000000 - 0x8067b004 (6605 kB)
  .init : 0x8067c000 - 0x806cfbc0 ( 335 kB)
```

Related Information

Enabling or Disabling the Pit Stop Utility on page 120

1.5.9.4. Booting Linux from the DXE Phase

1.5.9.4.1. Booting Linux from the DXE Phase Using SD/MMC

This task describes how to enable booting of Linux from the DXE phase.

1. Use your preferred editor to open the `<your_uefi_socfpga>/uefi-socfpga/AlteraPlatformPkg/Arria10SoCPkg/Arria10SoCPkg.dsc` file.
2. To boot the Linux zImage stored on an SD card or in QSPI flash right after the DXE phase, configure the `gAlteraSocFpgaTokenSpaceGuid.PcdBoot_LOAD_ZIMAGE_AT_PEI_PHASE` value to 0.
3. Configure the `gAlteraSocFpgaTokenSpaceGuid.PcdBoot_LOAD_UEFI_DXE_PHASE` value to 1.
4. Recompile the source code and update the SD card image with the modified `DXE.ROM` for the changes to take effect.
5. Once the compilation is successful, copy the following two files into the SD card FAT32 partition:
 - `<your_uefi_socfpga_directory>/uefi-socfpga/Build/DXE.ROM`
 - `<your_uefi_socfpga_directory>/uefi-socfpga/AlteraPlatformPkg/Applications/LinuxLoader/bootz.nsh`

1.5.9.4.2. Booting Linux from the DXE Phase Using SD/MMC Manually

When you boot the Linux zImage stored in QSPI Flash manually, the DXE phase boots until the UEFI shell appears and you type another command for further program execution. This can be done by typing the `bootz` command.

Note: This command runs the UEFI shell script `<your_uefi_socfpga_directory>/uefi-socfpga/AlteraPlatformPkg/Applications/LinuxLoader/bootz.nsh` which you copy into the FAT32 partition.

Figure 82. bootz Command when Booting Linux from the DXE Phase

```

COM14:115200baud - Tera Term VT
File Edit Setup Control Window Help

Shell> fs1:
FS1:> ls
Directory of: FS1:\
01/13/2016 13:22      3,551,640  zImage
01/13/2016 13:22      23,691  socfpga_arria10_socdk.dtb
01/13/2016 13:22     15,822,660  ghed_i0as066n2.rbf
03/18/2016 18:48      1,048,576  DXE.ROM
01/01/2000 00:00      5 File(s)  19,646,799 bytes
                   0 Dir(s)

FS1:> bootz
FS1:> LinuxLoader fs1:zImage -d fs1:socfpga_arria10_socdk.dtb -c "console=ttyS0,115200 root
R0p2 rw rootwait"
Booting Linux on physical CPU 0x0
Initializing cgroup subsys cpuset
Linux version 3.10.31-ltsi (jdasilva@sj-interactive3) (gcc version 4.8.3 20140401 (prerelease
ol-NG linaro-1.13.1-4.8-2014.04 - Linaro GCC 4.8-2014.04) > #1 SMP Wed Jan 13 13:16:50 PST 2
CPU: ARMv7 Processor [414fc091] revision 1 (ARMv7), cr=10c5387d
CPU: PIPT / UPIPT nonaliasing data cache, UPIPT aliasing instruction cache
Machine: Altera SOCFPGA, model: Altera SOCFPGA Arria 10
Memory policy: ECC disabled, Data cache writealloc
PERCPU: Embedded 8 pages/cpu @80f80000 s11200 r8192 d13376 u32768
Built 1 zonelists in Zone order, mobility grouping on. Total pages: 260096
Kernel command line: console=ttyS0,115200 root=/dev/mmcblk0p2 rw rootwait
PID hash table entries: 4096 (order: 2, 16384 bytes)
Dentry cache hash table entries: 131072 (order: 7, 524288 bytes)
Inode-cache hash table entries: 65536 (order: 6, 262144 bytes)
Memory: 1024MB = 1024MB total
Memory: 1031820k/1031820k available, 16756k reserved, 0K highmem
Virtual kernel memory layout:
vector : 0xffff0000 - 0xffff1000   ( 4 KB)
fixmap : 0xffff0000 - 0xffff0000   ( 896 KB)
vmalloc : 0xc0000000 - 0xff000000  (1000 MB)
lowmem  : 0x80000000 - 0xc0000000  (1024 MB)
modules : 0x7f000000 - 0x80000000   ( 16 MB)
 .text : 0x80000800 - 0x806977d0   (6718 KB)
 .init : 0x80698000 - 0x806f1bc0   ( 359 KB)
 .data : 0x806f2000 - 0x8072f2d0   ( 245 KB)
 bss   : 0x8072f2d0 - 0x80770ab4   ( 262 KB)
SLUB: HWalign=64, Order=0-3, MinObjects=0, CPUs=2, Nodes=1

```

1.5.9.4.3. Booting Linux from the DXE Phase Using SD/MMC Automatically

To configure Linux to boot automatically in the DXE phase, you must edit the UEFI shell start-up script. The startup.nsh is the start-up script file that the UEFI shell automatically runs at boot.

1. Use your preferred editor to open the <your_uefi_socfpga>/uefi-socfpga/AlteraPlatformPkg/ShellScript/startup.nsh file.
2. Append the following line to the last line of the file:

```
bootz
```

Figure 83. startup.nsh Edit

```

startup.nsh
1  @echo -off
2  echo Running startup.nsh - The Auto-Start UEFI Shell Script
3  @echo -on
4  @ifconfig -s eth0 static 192.168.0.5 255.255.255.0 192.168.0.1
5  ifconfig -s eth0 dhcp
6  stall 5000000
7  ifconfig -l eth0
8  @echo .
9  bootz
10

```

3. Save the file and recompile the UEFI boot loader code.
4. Update the SD card image with the new PEI.ROM file for the changes to take effect.
5. Power cycle the board and observe the following output messages on your serial terminal program:

Figure 84. Terminal Output Message

```

COM14:115200baud - Tera Term VT
File Edit Setup Control Window Help
UEFI v2.50 < EFI Mar 18 2016 18:44:48, 0x00000000>
Mapping table
FS0: Alias(s):F0:
    Fu<5EDa4200-2C5F-43CB-9DA3-0BAF74B1B30C>
FS1: Alias(s):HD3b;:BLK1:
    VenHu<7777515A-04D8-49E8-8426-EE80938DA484>/HD<1.MBR,0x0008501C,0x206800,0x199998>
BLK0: Alias(s):
    VenHu<7777515A-04D8-49E8-8426-EE80938DA484>
BLK2: Alias(s):
    VenHu<7777515A-04D8-49E8-8426-EE80938DA484>/HD<2.MBR,0x0008501C,0x4000,0x200000>
BLK3: Alias(s):
    VenHu<7777515A-04D8-49E8-8426-EE80938DA484>/HD<3.MBR,0x0008501C,0x800,0x1000>

Running startup.nsh - The Auto-Start UEFI Shell Script
Shell> ifconfig -s eth0 dhcp
Shell> stall 5000000
Shell> ifconfig -l eth0
Shell> LinuxLoader fs1:zImage -d fs1:socfpga_arria10_socdk.dtb -c "console=ttyS0,115200 root=/dev
/mmcblk0p2 rw rootwait"
Booting Linux on physical CPU 0x0
Initializing cgroup subsys cpuset
Linux version 4.8.3-lts1 (jdasilva@sj-interactive3) (gcc version 4.8.3 20140401 (prerelease) <
rosstool-NG linaro-1.13.1-4.8-2014.04 - Linaro GCC 4.8-2014.04>) #1 SMP Wed Jan 13 13:16:50 PST
2016
CPU: ARMv7 Processor [414fc091] revision 1 (ARMv7), cr=10c5387d
CPU: PIPT / UIPT nonaliasing data cache, UIPT aliasing instruction cache
Machine: Altera SOCFPGA, model: Altera SOCFPGA Arria 10
Memory policy: ECC disabled, Data cache writealloc
PERCPU: Embedded 8 pages/cpu @80f80000 s11200 r8192 d13376 u32768
Built 1 zonelists in Zone order, mobility grouping on. Total pages: 260096
Kernel command line: console=ttyS0,115200 root=/dev/mmcblk0p2 rw rootwait
PID hash table entries: 4096 (order: 2, 16384 bytes)
Dentry cache hash table entries: 131072 (order: 7, 524288 bytes)

```

1.5.10. Debugging an Example Project

Task time: 35 minutes

The Arm Development Studio 5 Intel SoC FPGA Edition provides very powerful Bare Metal debugging capabilities. The Arm DS-5 Intel SoC FPGA Edition debugger connects to an Intel Intel FPGA Download Cable II to debug a Bare Metal application.

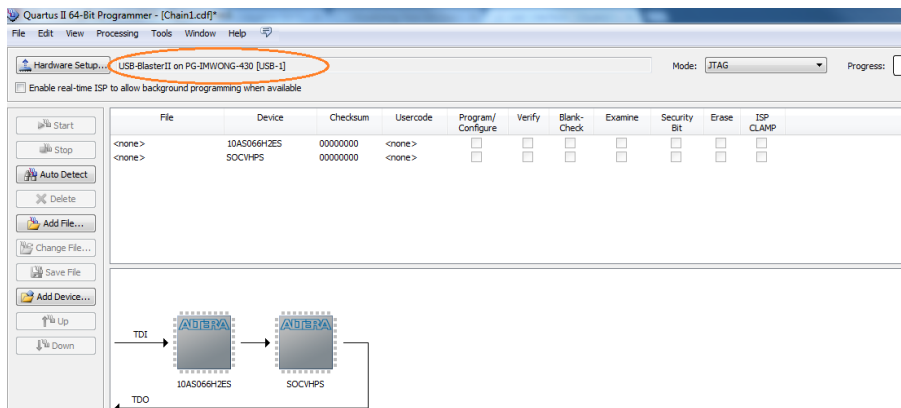
This section describes how to configure the Eclipse Arm DS-5 Intel SoC FPGA Edition platform and start the Arm DS-5 Intel SoC FPGA Edition debugger from the Eclipse IDE using the debug view. The debug view allows you to manage the debugging and running of a program in the workbench. You can use the debug view to set breakpoints and watchpoints in the code and trace variables.

1.5.10.1. Configuring and Starting the Debugger in Eclipse

This task describes how to configure your project setting to use Eclipse with the Arm DS-5 Intel SoC FPGA Edition debugger.

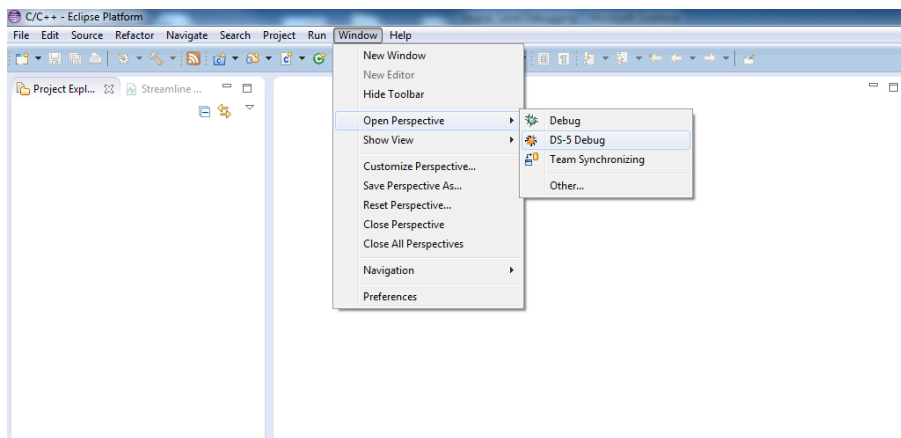
1. Connect the mini USB cable from the Intel Arria 10 SoC FPGA development board Intel FPGA Download Cable II port to the PC.
2. Verify that your JTAG Intel FPGA Download Cable II can detect the devices. The diagram below shows an example.

Figure 85. Verifying Device Detection



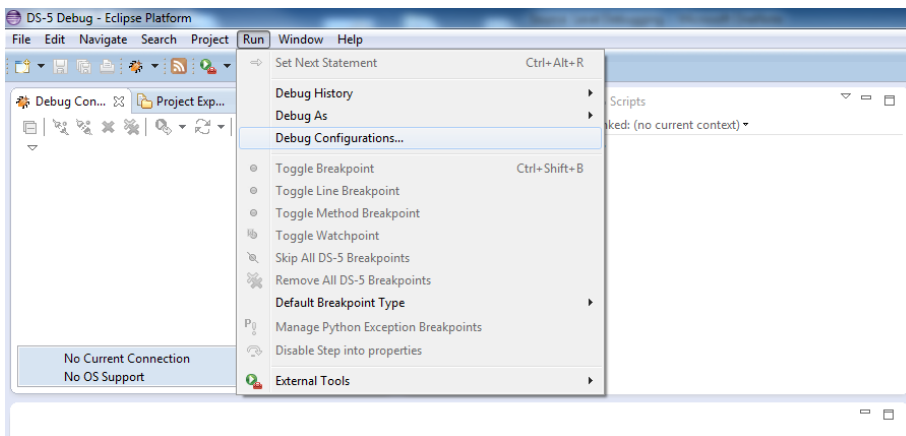
3. Start the Eclipse application if it is not yet started. Select **Start Menu > All Programs > ARM DS-5 > Eclipse for DS-5** to start Eclipse from the program search window. Alternatively, you can start Eclipse from the **Embedded Command Shell** window.
4. The Eclipse tool, part of the Arm DS-5 Intel SoC FPGA Edition, prompts for the workspace folder to be used. You can use the suggested folder and click **OK**.
5. The Arm DS-5 Intel SoC FPGA Edition **Welcome** screen appears. You can use this screen to access documentation, tutorials and videos.
6. Select **Window > Open Perspective > DS-5 Debug** to open the workbench.

Figure 86. Opening the DS-5 Workbench



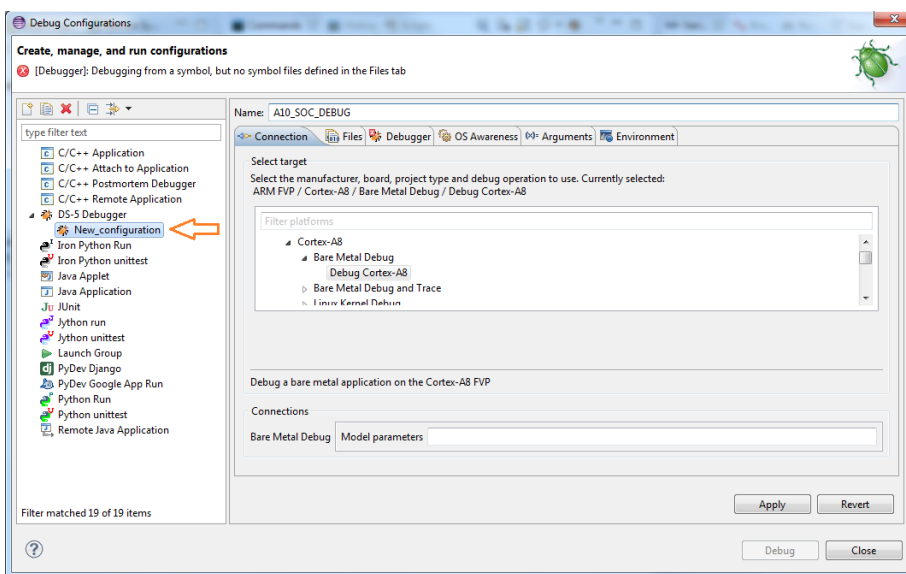
7. Select **Run > Debug Configurations...** to access the launch configurations.

Figure 87. Opening Debug Configurations



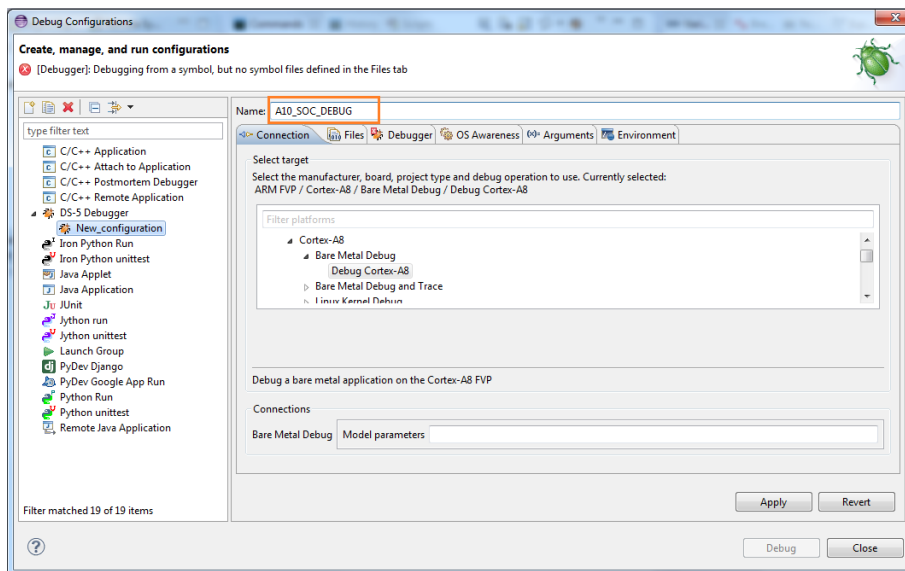
8. In the **Debug Configuration** dialog box left panel, select **DS-5 Debugger** ➤ **New_configuration**

Figure 88. Opening New Configuration



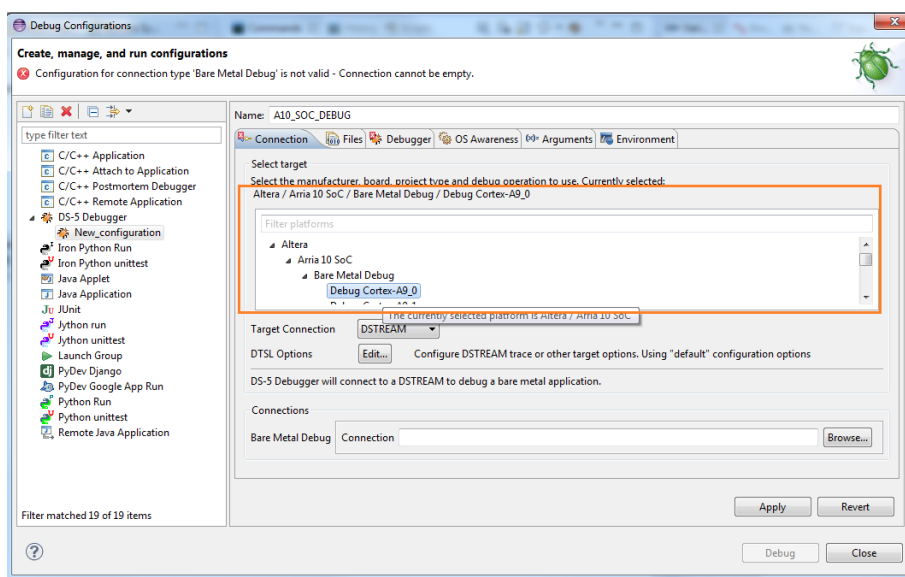
9. Enter your preferred **Name**. In the example shown below, the **Name** text field has an entry of **A10_SOC_DEBUG**.

Figure 89. New Configuration Name Text Field



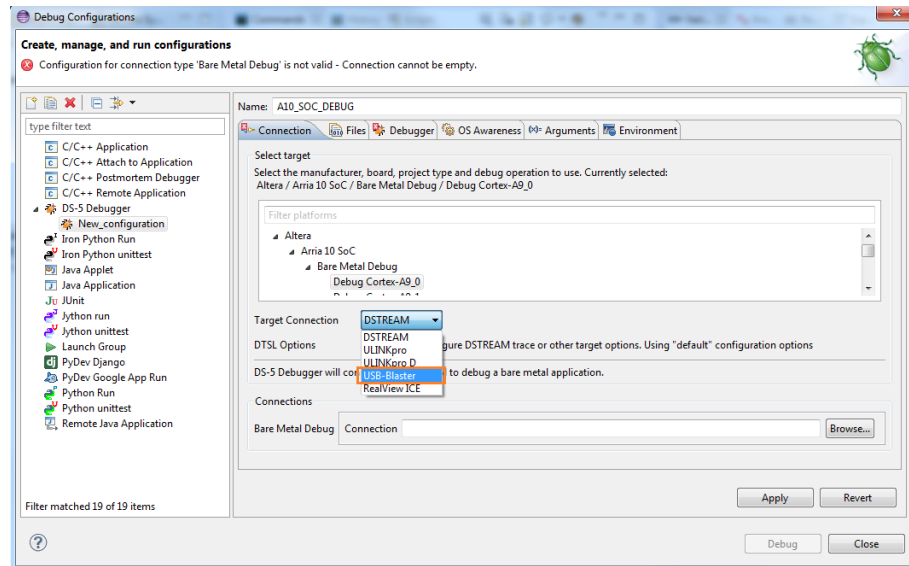
10. Configure the target to be **Altera > Arria 10 SoC > Bare Metal Debug > Debug Cortex-A9_0**.

Figure 90. Configure Target Connection



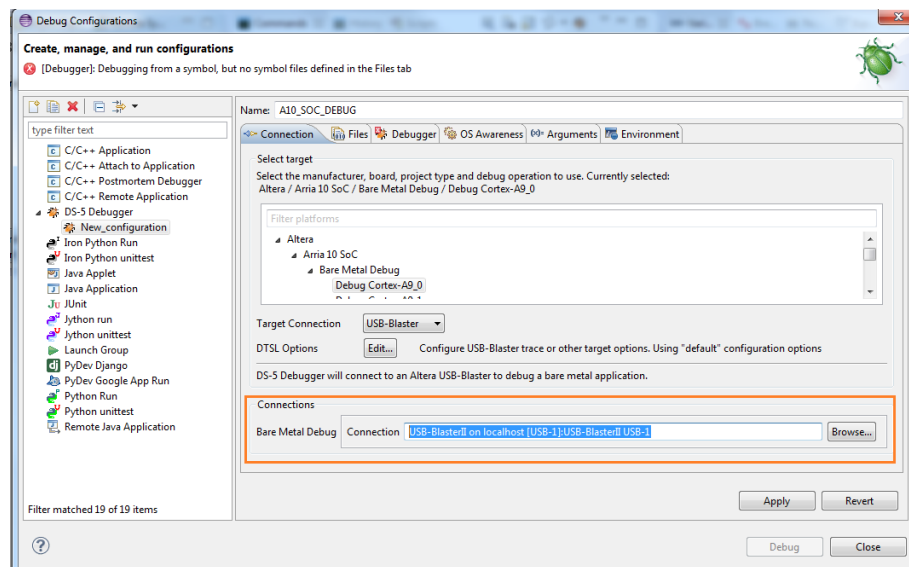
11. In the **Target Connection** pull-down menu, select **USB-Blaster** connection.

Figure 91. Select USB Blaster Connection



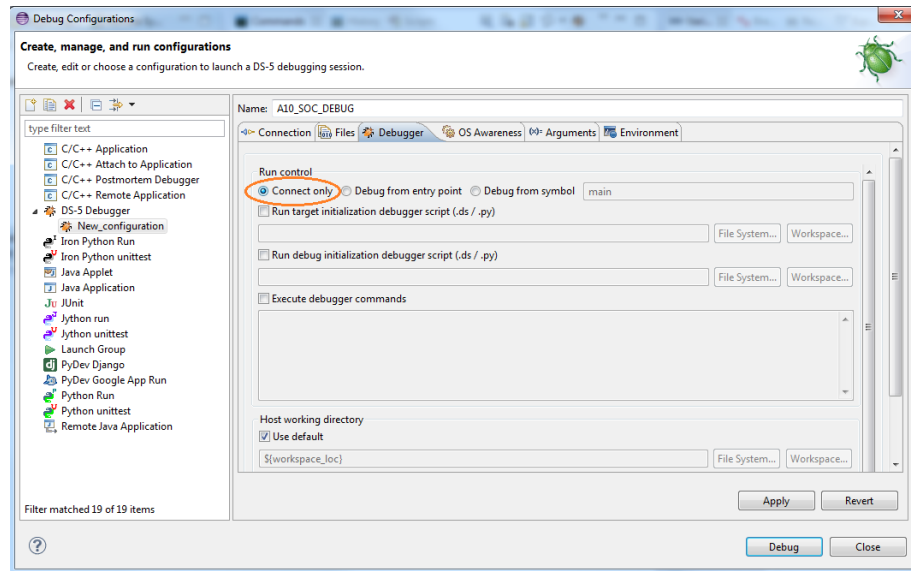
12. For the **Bare Metal Debug Connection**, click **Browse...** and select **USB-BlasterII** on localhost [USB-1]:USB-BlasterII USB-1.

Figure 92. Selecting Bare Metal Debug Connection



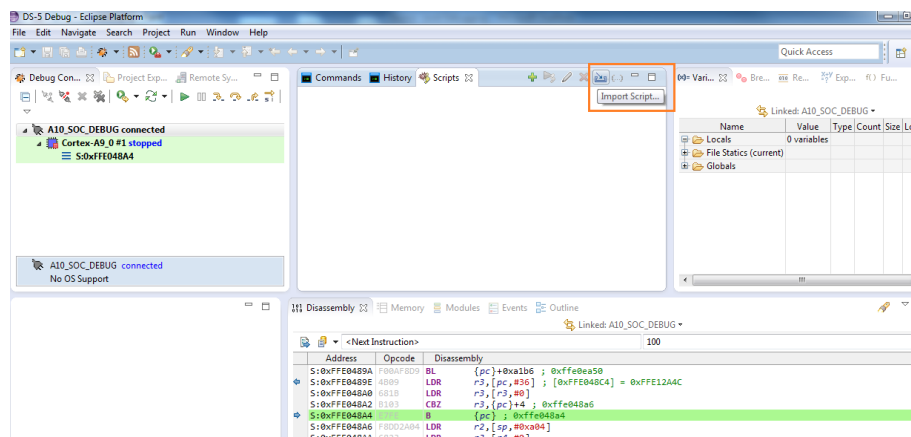
13. Select the **Debugger** tab and select the **Connect only** option.

Figure 93. Connect Only Selection for Run Control



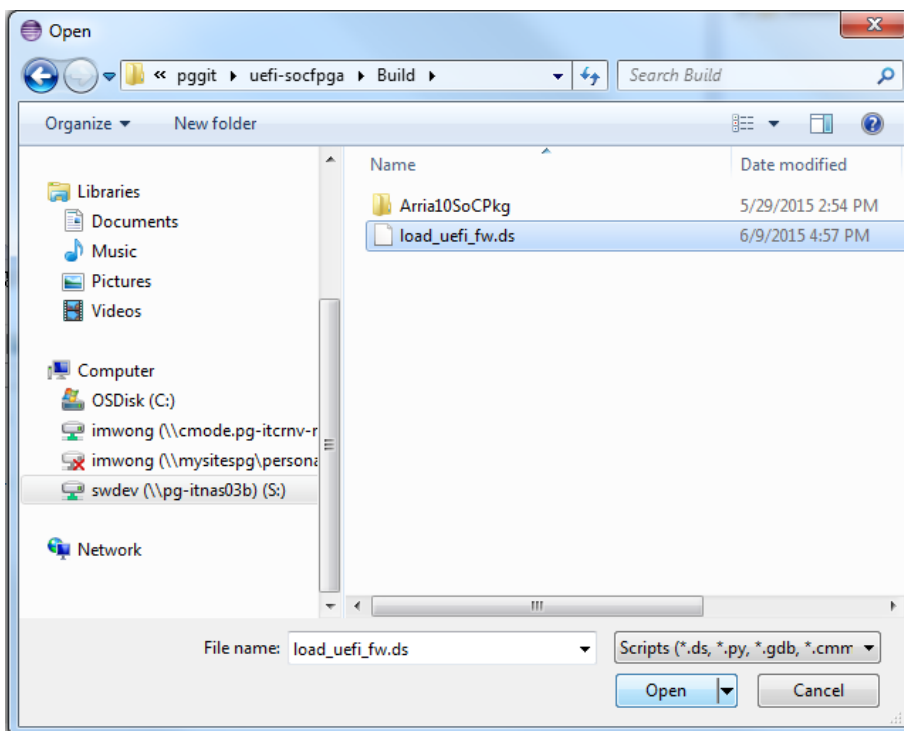
14. Click the **Debug** button.
15. In the **DS-5 Debug** window, select **Import Script...**

Figure 94. Import Script



16. Browse to your Arm DS-5 Intel SoC FPGA Edition script location under Build/load_uefi_fw.ds. In this example, /data/<username>/pggit/uefi-socfpga/Build/load_uefi_fw.ds is selected.

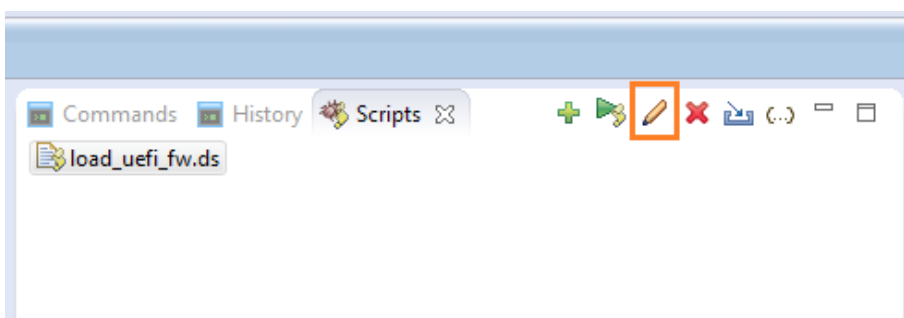
Figure 95. Browsing to the load_uefi_fw.ds File



17. Click **Open**.

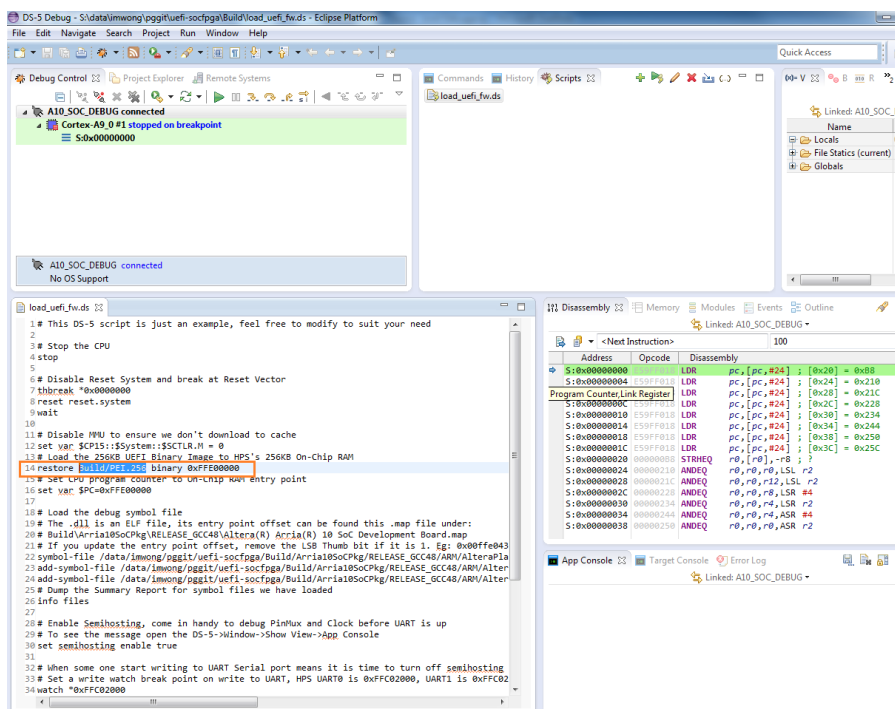
18. Edit your `load_uefi_fw.ds` file by selecting the **pencil icon**.

Figure 96. Editing the load_uefi_fw.ds File



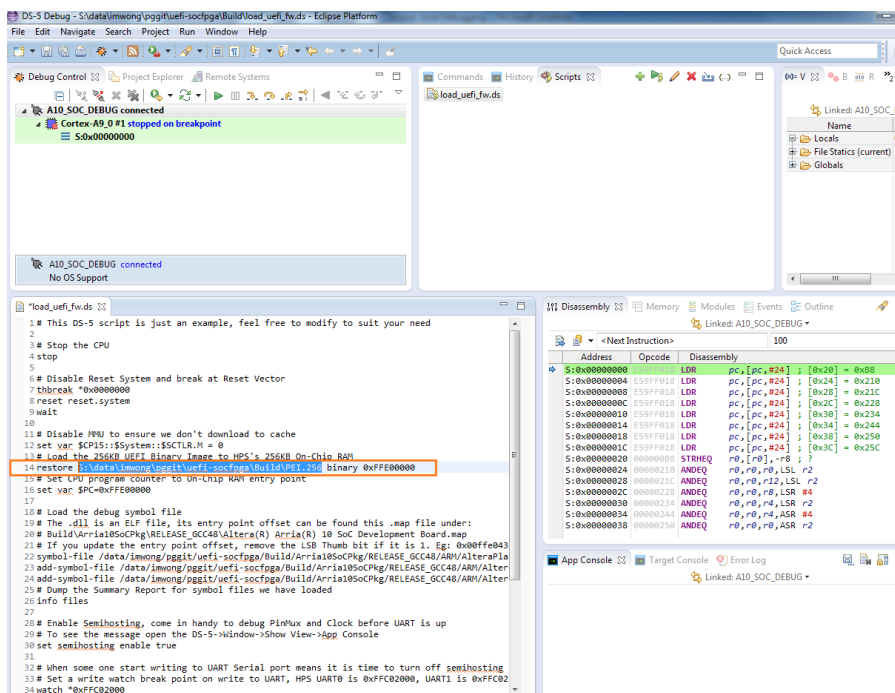
19. Verify the `load_uefi_fw.ds` file loads the correct Build/PEI.256 path. If not, edit the path to locate the correct Build/PEI.256 file path. In this example, the PEI.256 file is found at: `/data/<username>/pggit/uefisocfpga/Build/PEI.256`. Make the following change from the screenshot below:

Figure 97. Original PEI.256 Path



To the following path:

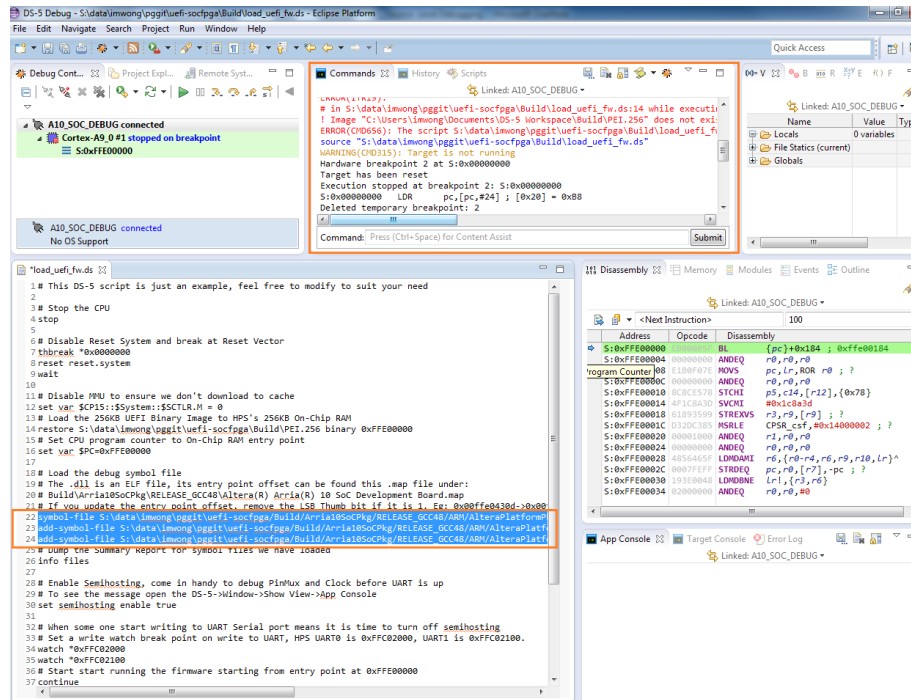
Figure 98. Modified PEI.256 Path



20. Click **Save**.

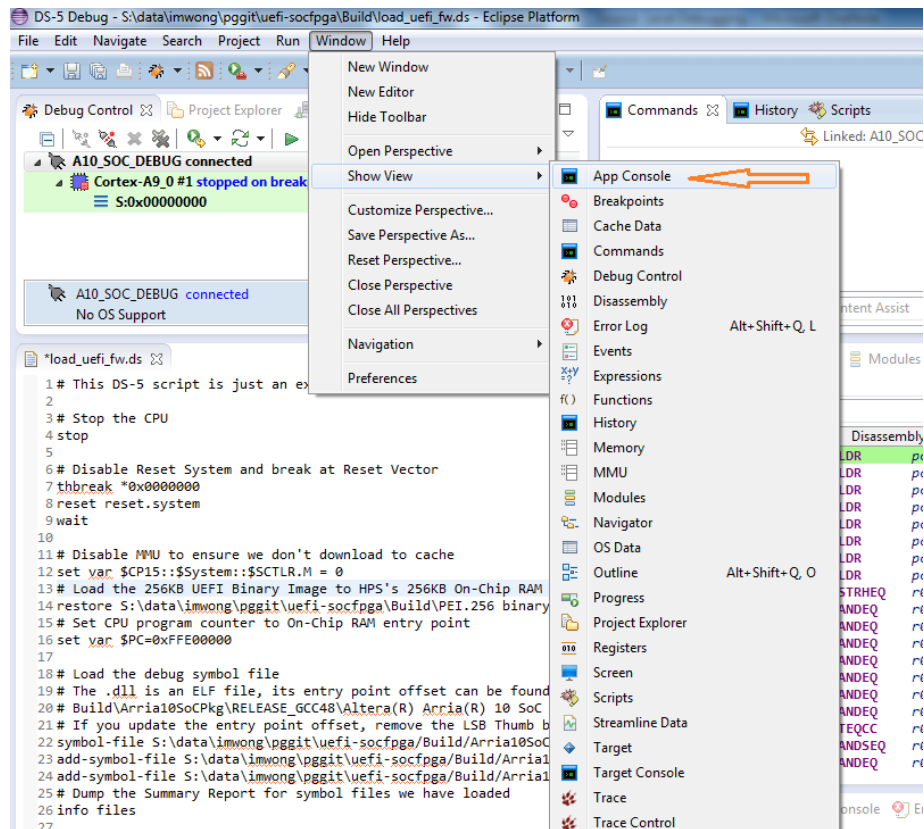
21. Double-click the **load_uefi_fw.ds** file to run the updated script.
22. If you encounter the error messages below, verify that you have edited the **load_uefi_fw.ds** to the correct path. Proceed to step 24 on page 90 if you do not see this error.

Figure 99. Error Message Window



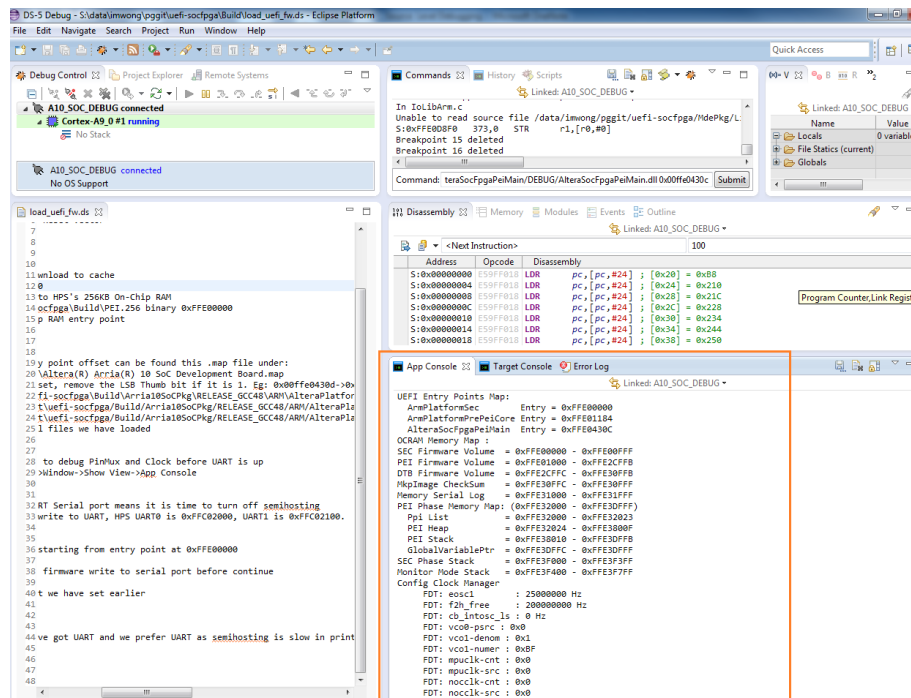
23. Click **Save** and double-click the **load_uefi_fw.ds** file to run the updated script.
24. Select **Window** ► **Show View** ► **App Console** to enable the **App Console** window.

Figure 100. Enabling the App Console Window



25. Click the continue **green arrow** to run the application.
26. The following content appears in the **App Console** window:

Figure 101. App Console Window



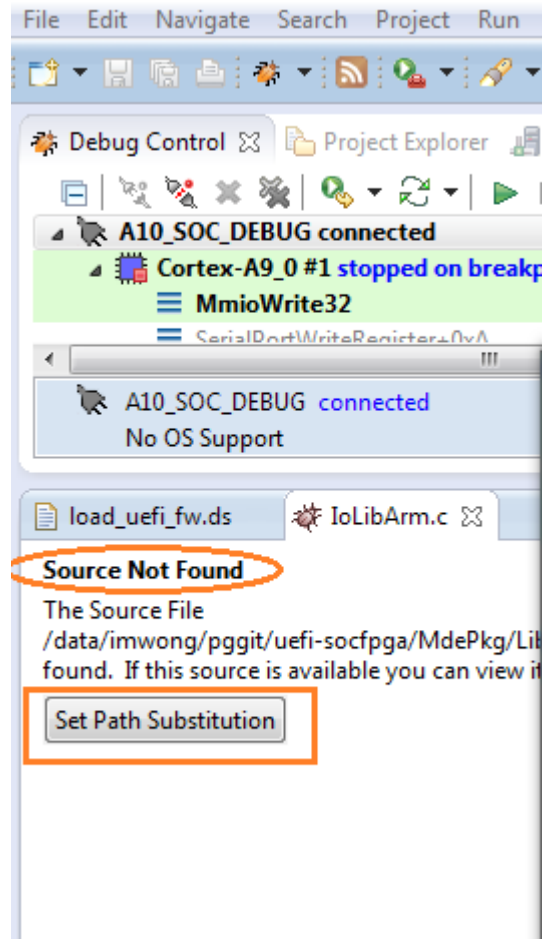
At the same time, pin multiplexing configuration output messages appear on the serial terminal program window.

Figure 102. Serial Terminal Pin Multiplexing Configuration Output Messages

```
COM10:115200baud - Tera Term VT
File Edit Setup Control Window Help
UEFI firmware version 1.0 Beta built at 16:53:17 on Jun 9 2015
Altera(R) Arria(R) 10 SoC Development Board
Hard Processor System - Silicon ID 1: 0x00010001
Hard Processor: ARM(R) Cortex(R)-A9 MPCore
ARM(R) TrustZone(R) is disabled
BootInfo reg: 0x00004410
BSL = SDMMC 1.0U
Reset Request Source(s): 0x00000100
  RST Pin Warm Reset
RST_MGR RAM Status: 0x00018000
RST_MGR MISC Status: 0x00000004
RST_MGR HdsKrn reg: 0x0000000F
RST_MGR HdsKAc reg: 0x00000108
RST_MGR Peripherals Reset State:
  PER0MODRST: 0x00000000
  PER0MODRST: 0x00000000
Security Manager current state (HPS_cursecstate reg): 0x00000000
  AUTH_DIS
  AES_DIS
  FPGA_BOOT_DIS
  HPS_FOSC1_CLK
  OCRAM_BOOT_EN
  CLR_RAM_WARM_DIS
  CLR_RAM_COLD_DIS
  CLR_RAM_ORDER_SER
SEC_MGR hps_fusesec reg: 0x00000000
SEC_MGR hps_fusesec2 reg: 0x00000000
SEC_MGR fpga_fusesec reg: 0x00000000
SEC_MGR HPS_Swoptset reg: 0x0000000F
SEC_MGR sec_fpgachk reg: 0x00000000
SEC_MGR sec_hpschk reg: 0x00000000
SEC_MGR swromcode reg: 0x00000000
Pin mux settings for shared 20 IO:
pinmux_shared_io_q1_1 (GP100_100) : usb0.clk (8)
pinmux_shared_io_q1_2 (GP100_101) : usb0.stp (8)
pinmux_shared_io_q1_3 (GP100_102) : usb0.dir (8)
pinmux_shared_io_q1_4 (GP100_103) : usb0.data0 (8)
pinmux_shared_io_q1_5 (GP100_104) : usb0.data1 (8)
pinmux_shared_io_q1_6 (GP100_105) : usb0.nxt (8)
pinmux_shared_io_q1_7 (GP100_106) : usb0.data2 (8)
pinmux_shared_io_q1_8 (GP100_107) : usb0.data3 (8)
pinmux_shared_io_q1_9 (GP100_108) : usb0.data4 (8)
pinmux_shared_io_q1_10 (GP100_109) : usb0.data5 (8)
pinmux_shared_io_q1_11 (GP100_1010) : usb0.data6 (8)
pinmux_shared_io_q1_12 (GP100_1011) : usb0.data7 (8)
pinmux_shared_io_q2_1 (GP100_1012) : enac0.tx_clk (4)
pinmux_shared_io_q2_2 (GP100_1013) : enac0.tx_ctl (4)
pinmux_shared_io_q2_3 (GP100_1014) : enac0.rx_clk (4)
pinmux_shared_io_q2_4 (GP100_1015) : enac0.rx_ctl (4)
pinmux_shared_io_q2_5 (GP100_1016) : enac0.tx0 (4)
pinmux_shared_io_q2_6 (GP100_1017) : enac0.tx1 (4)
pinmux_shared_io_q2_7 (GP100_1018) : enac0.rxd0 (4)
pinmux_shared_io_q2_8 (GP100_1019) : enac0.rxd1 (4)
pinmux_shared_io_q2_9 (GP100_1020) : enac0.tx2 (4)
pinmux_shared_io_q2_10 (GP100_1021) : enac0.tx3 (4)
pinmux_shared_io_q2_11 (GP100_1022) : enac0.rxd2 (4)
pinmux_shared_io_q2_12 (GP100_1023) : enac0.rxd3 (4)
pinmux_shared_io_q3_1 (GP101_100) : spin1.clk (3)
pinmux_shared_io_q3_2 (GP101_101) : spin1.mosi (3)
pinmux_shared_io_q3_3 (GP101_102) : spin1.miso (3)
pinmux_shared_io_q3_4 (GP101_103) : spin1.ss0_n (3)
pinmux_shared_io_q3_5 (GP101_104) : spin1.ssi_n (3)
```

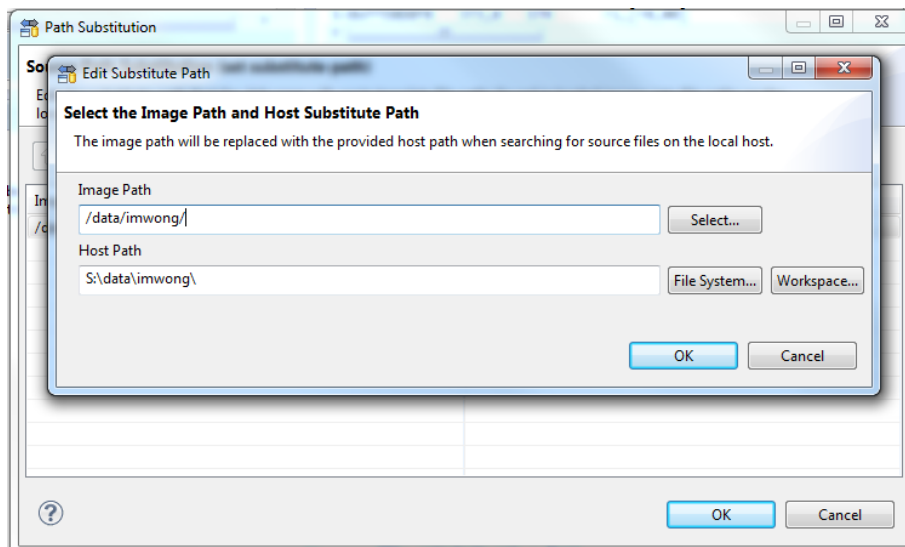
27. Use the **Outline** view to edit and view your source file. If you encounter a "Source Not Found" error, click **Set Path Substitution**.

Figure 103. Correcting a "Source Not Found" Error



28. When the **Edit Substitute Path** dialog box appears, configure the **Host Path** to the correct path and click **OK**.

Figure 104. Configuring the Host Path

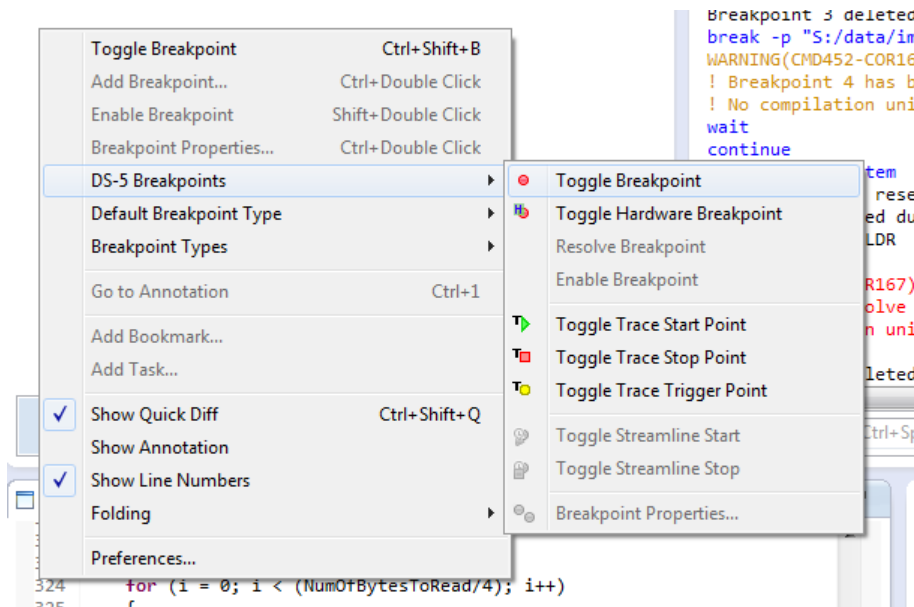


1.5.10.2. Setting a Breakpoint

You can set breakpoints in your program for debugging purposes. This task describes how to set a breakpoint on line 324 of the FpgaManager .c file.

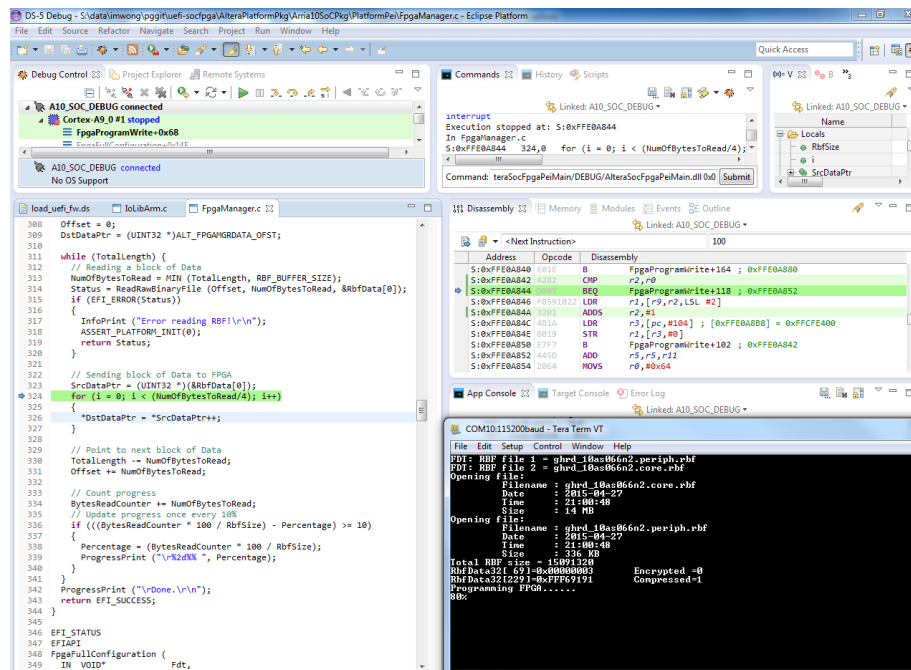
1. Open the FpgaManager .c file from the **Outline** view.
2. Go to line 324 to insert the breakpoint. Right click and select **DS-5 Breakpoints** ➤ **Toggle Breakpoint**.

Figure 105. Inserting a Breakpoint in the FpgaManager .c File



3. When you resume your application, the debugger stops on this line.

Figure 106. Breakpoint Stop



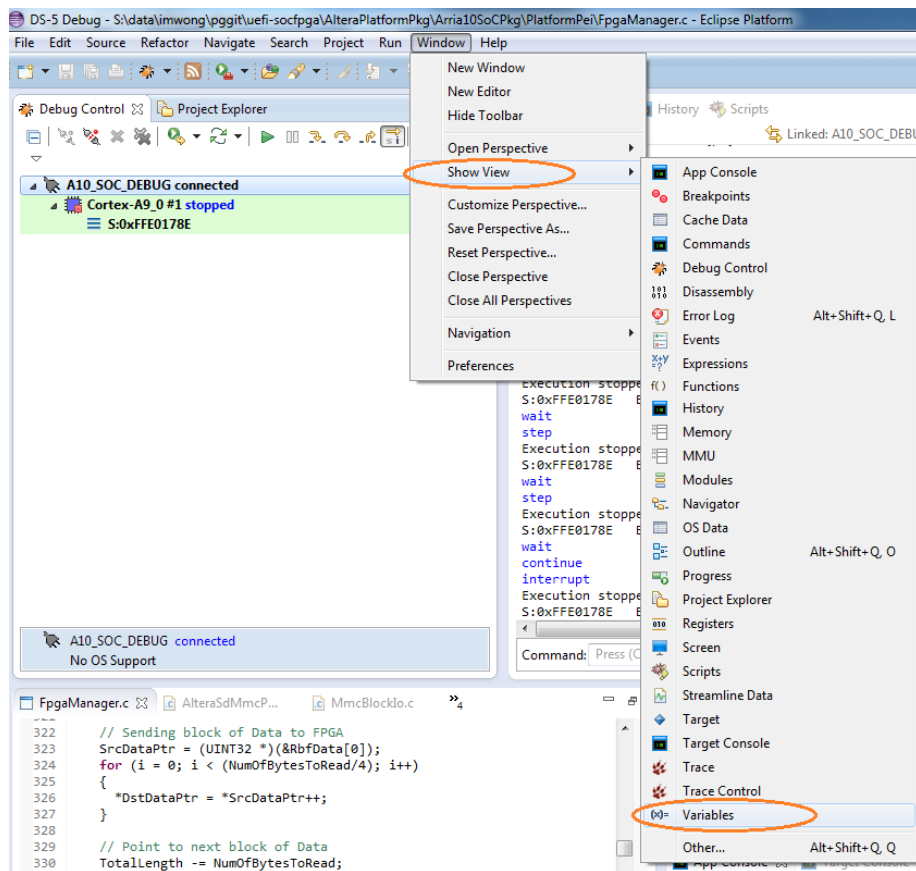
4. Use the **Step Over** or **Step Into** arrows in the Debug Window to step through your code.

1.5.10.3. Watching Variable Contents

This task describes how to check the contents of a variable.

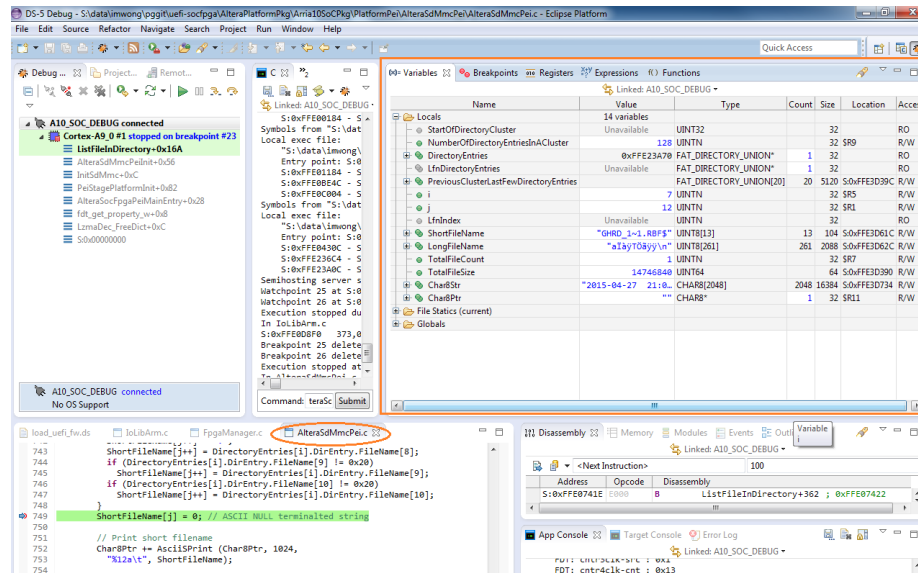
1. Select **Window** ► **Show View** ► **Variables**.

Figure 107. Entering Variable Content View



2. Observe the content of your variables in the **Variables** window. The screenshot below shows the **Locals** variable content for the AlteraSdMmcPei.c file.

Figure 108. Locals Variable Watch



1.5.11. UEFI Boot Loader Customization

This section presents a list of useful customization recipes for the UEFI boot loader to run on your custom board.

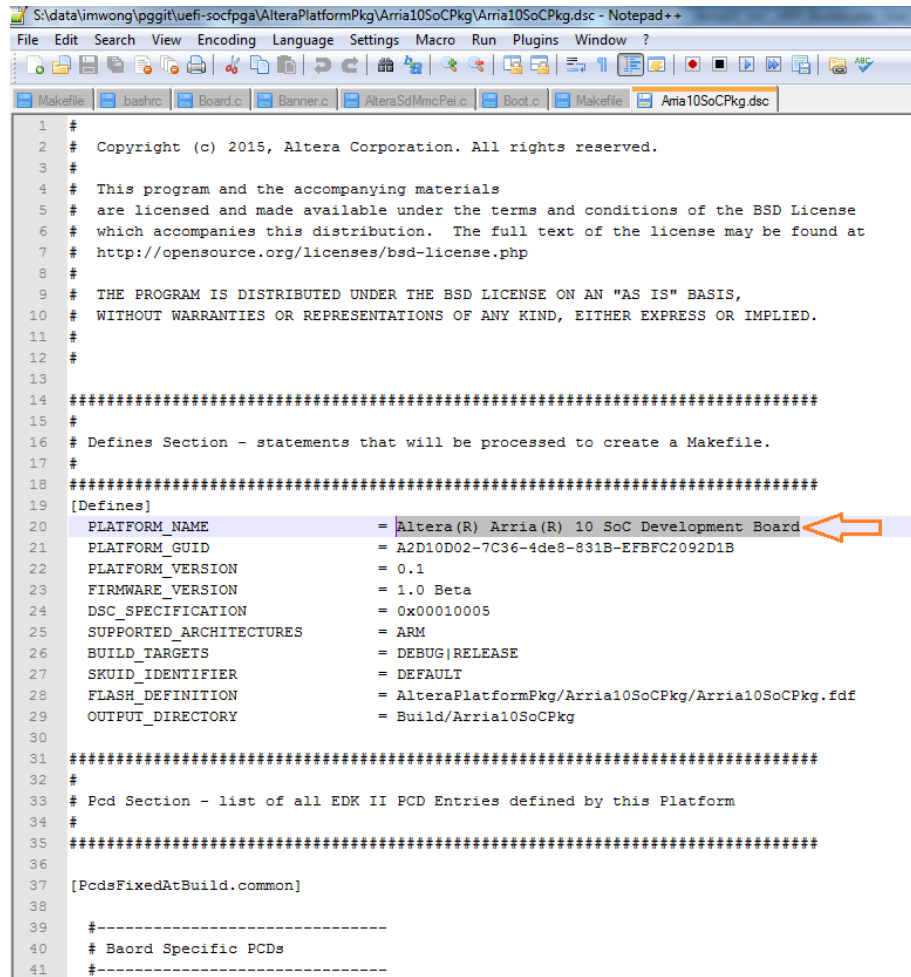
Task time: 15 minutes

1.5.11.1. Changing the Platform Name

If you are not using the Intel Arria 10 SoC FPGA development board and want to change to your platform name, you can edit parameters in the `Arria10SoCPkg.dsc` file to support this.

1. Use your preferred editor to open the `<your_uefi_socfpga>/uefi-socfpga/AlteraPlatformPkg/Arria10SoCPkg/Arria10SoCPkg.dsc` file.
2. Change the PLATFORM NAME below to your custom board name.

Figure 109. Editing the PLATFORM NAME in the Arria10soCPkg.dsc File



```
1 #
2 # Copyright (c) 2015, Altera Corporation. All rights reserved.
3 #
4 # This program and the accompanying materials
5 # are licensed and made available under the terms and conditions of the BSD License
6 # which accompanies this distribution. The full text of the license may be found at
7 # http://opensource.org/licenses/bsd-license.php
8 #
9 # THE PROGRAM IS DISTRIBUTED UNDER THE BSD LICENSE ON AN "AS IS" BASIS,
10 # WITHOUT WARRANTIES OR REPRESENTATIONS OF ANY KIND, EITHER EXPRESS OR IMPLIED.
11 #
12 #
13 #
14 #####
15 #
16 # Defines Section - statements that will be processed to create a Makefile.
17 #
18 #####
19 [Defines]
20 PLATFORM_NAME = Altera(R) Arria(R) 10 SoC Development Board
21 PLATFORM_GUID = A2D10D02-7C36-4de8-831B-EFBFC2092D1B
22 PLATFORM_VERSION = 0.1
23 FIRMWARE_VERSION = 1.0 Beta
24 DSC_SPECIFICATION = 0x00010005
25 SUPPORTED_ARCHITECTURES = ARM
26 BUILD_TARGETS = DEBUG|RELEASE
27 SKUID_IDENTIFIER = DEFAULT
28 FLASH_DEFINITION = AlteraPlatformPkg/Arria10SoCPkg/Arria10SoCPkg.fdf
29 OUTPUT_DIRECTORY = Build/Arria10SoCPkg
30
31 #####
32 #
33 # Pcd Section - list of all EDK II PCD Entries defined by this Platform
34 #
35 #####
36
37 [PcdsFixedAtBuild.common]
38
39 #-----
40 # Board Specific PCDs
41 #-----
```

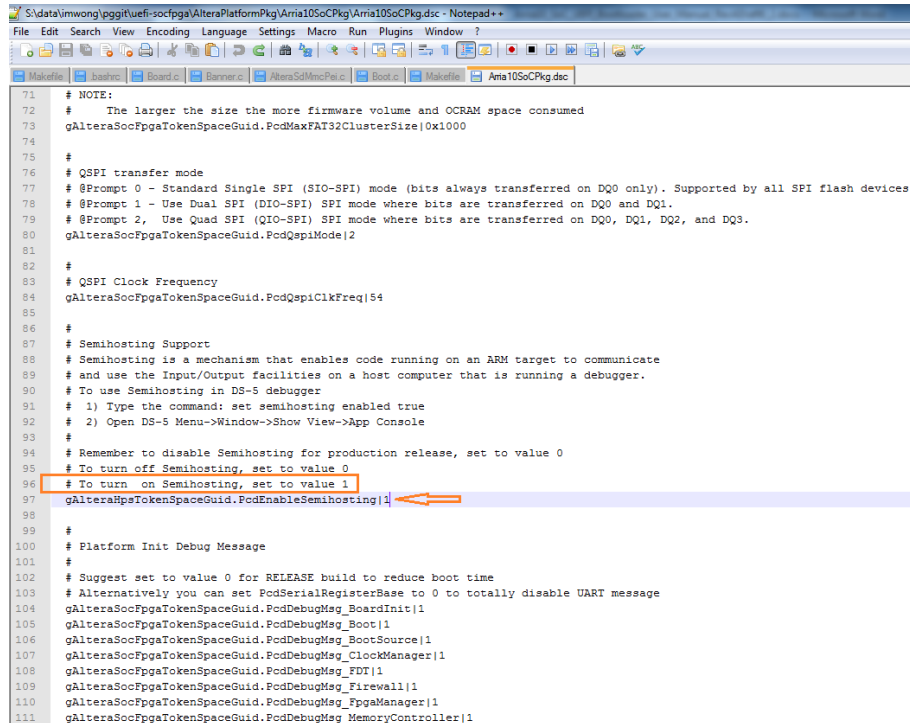
3. Configure the `gAlteraSoCFpgaTokenSpaceGuid.PcdIsAlteraSoCFPGADevelopmentBoards` value from 1 to 0.

```
File Edit Search View Encoding Language Settings Macro Run Plugins Window ?
Makefile bashrc Board.c Banner.c AlteraSdMmcPci.c Boot.c Makefile Arria10SoCPkg.dsc

19 [Defines]
20 PLATFORM_NAME = Altera(R) Arria(R) 10 SoC Development Board
21 PLATFORM_GUID = A2D10D02-7C36-4de8-831B-EFBFC2092D1B
22 PLATFORM_VERSION = 0.1
23 FIRMWARE_VERSION = 1.0 Beta
24 DSC_SPECIFICATION = 0x00010005
25 SUPPORTED_ARCHITECTURES = ARM
26 BUILD_TARGETS = DEBUG|RELEASE
27 SKUID_IDENTIFIER = DEFAULT
28 FLASH_DEFINITION = AlteraPlatformPkg/Arria10SoCPkg/Arria10SoCPkg.fdf
29 OUTPUT_DIRECTORY = Build/Arria10SoCPkg
30
31 #####
32 #
33 # Pcd Section - list of all EDK II PCD Entries defined by this Platform
34 #
35 #####
36
37 [PcdsFixedAtBuild.common]
38
39 #-----
40 # Board Specific PCDs
41 #-----
42 gAlteraHpsTokenSpaceGuid.PcdPlatformNameString|L"$ (PLATFORM_NAME) "
43 gAlteraSocFpgaTokenSpaceGuid.PcdIsAlteraSoCFPGADevelopmentBoards|1
44
45 ##*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*
46 ## Altera SoC HPS modules PCD
47 ##*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*
48
49 #-----
50 # gAlteraSocFpgaTokenSpaceGuid
51 #-----
52
53 #
54 # Platform Init Features Enable / Disable
55 #
56 gAlteraSocFpgaTokenSpaceGuid.PcdSdmmcSweepAllDrvselAndSmplselValues|0
57 gAlteraSocFpgaTokenSpaceGuid.PcdSdmmcDrvSel|3
58 gAlteraSocFpgaTokenSpaceGuid.PcdSdmmcSmplSel|2
59 gAlteraSocFpgaTokenSpaceGuid.PcdSdmmcSupport4BitMode|1
```

1. Use your preferred editor to open the `<your_uefi_socfpga>/uefi-socfpga/AlteraPlatformPkg/Arria10SoCPkg/Arria10SoCPkg.dsc` file.
2. Configure the `gAlteraHpsTokenSpaceGuid.PcdenableSemihosting` value from 1 to 0.

Figure 111. Modifying Semihosting Support Parameter



```
71 # NOTE:
72 # The larger the size the more firmware volume and OCRAM space consumed
73 gAlteraSocFpgaTokenSpaceGuid.PcdMaxFAT32ClusterSize|0x1000
74
75 #
76 # QSPI transfer mode
77 # @Prompt 0 - Standard Single SPI (SIO-SPI) mode (bits always transferred on DQ0 only). Supported by all SPI flash devices.
78 # @Prompt 1 - Use Dual SPI (DIO-SPI) SPI mode where bits are transferred on DQ0 and DQ1.
79 # @Prompt 2, Use Quad SPI (QIO-SPI) SPI mode where bits are transferred on DQ0, DQ1, DQ2, and DQ3.
80 gAlteraSocFpgaTokenSpaceGuid.PcdQspiMode|2
81
82 #
83 # QSPI Clock Frequency
84 gAlteraSocFpgaTokenSpaceGuid.PcdQspiClkFreq|54
85
86 #
87 # Semihosting Support
88 # Semihosting is a mechanism that enables code running on an ARM target to communicate
89 # and use the Input/Output facilities on a host computer that is running a debugger.
90 # To use Semihosting in DS-5 debugger
91 # 1) Type the command: set semihosting enabled true
92 # 2) Open DS-5 Menu->Window->Show View->App Console
93 #
94 # Remember to disable Semihosting for production release, set to value 0
95 # To turn off Semihosting, set to value 0
96 # To turn on Semihosting, set to value 1
97 gAlteraHpsTokenSpaceGuid.PcdEnableSemihosting|1
98
99 #
100 # Platform Init Debug Message
101 #
102 # Suggest set to value 0 for RELEASE build to reduce boot time
103 # Alternatively you can set PcdSerialRegisterBase to 0 to totally disable UART message
104 gAlteraSocFpgaTokenSpaceGuid.PcdDebugMsg_BoardInit|1
105 gAlteraSocFpgaTokenSpaceGuid.PcdDebugMsg_Boot|1
106 gAlteraSocFpgaTokenSpaceGuid.PcdDebugMsg_BootSource|1
107 gAlteraSocFpgaTokenSpaceGuid.PcdDebugMsg_ClockManager|1
108 gAlteraSocFpgaTokenSpaceGuid.PcdDebugMsg_FDT|1
109 gAlteraSocFpgaTokenSpaceGuid.PcdDebugMsg_Firewall|1
110 gAlteraSocFpgaTokenSpaceGuid.PcdDebugMsg_FpgaManager|1
111 gAlteraSocFpgaTokenSpaceGuid.PcdDebugMsg_MemoryController|1
```

3. In addition, you can also disable the following three suggested sections for your production release build in order to reduce the boot time.

Figure 112. Disabling Suggested Parameters for Boot Time Reduction

```

98
99 #
100 # Platform Init Debug Message
101 #
102 # Suggest set to value 0 for RELEASE build to reduce boot time
103 # Alternatively you can set PcdSerialRegisterBase to 0 to totally disable UART message
104 gAlteraSocFpgaTokenSpaceGuid.PcdDebugMsg_BoardInit|1
105 gAlteraSocFpgaTokenSpaceGuid.PcdDebugMsg_Boot|1
106 gAlteraSocFpgaTokenSpaceGuid.PcdDebugMsg_BootSource|1
107 gAlteraSocFpgaTokenSpaceGuid.PcdDebugMsg_ClockManager|1
108 gAlteraSocFpgaTokenSpaceGuid.PcdDebugMsg_FDT|1
109 gAlteraSocFpgaTokenSpaceGuid.PcdDebugMsg_Firewall|1
110 gAlteraSocFpgaTokenSpaceGuid.PcdDebugMsg_FpgaManager|1
111 gAlteraSocFpgaTokenSpaceGuid.PcdDebugMsg_MemoryController|1
112 gAlteraSocFpgaTokenSpaceGuid.PcdDebugMsg_Pinctmux|1
113 gAlteraSocFpgaTokenSpaceGuid.PcdDebugMsg_ResetManager|1
114 gAlteraSocFpgaTokenSpaceGuid.PcdDebugMsg_SdMmc|1
115 gAlteraSocFpgaTokenSpaceGuid.PcdDebugMsg_SecurityManager|1
116 gAlteraSocFpgaTokenSpaceGuid.PcdDebugMsg_SystemManager|1
117 gAlteraSocFpgaTokenSpaceGuid.PcdDebugMsg_Rbf|1
118 gAlteraSocFpgaTokenSpaceGuid.PcdDebugMsg_Nand|1
119 gAlteraSocFpgaTokenSpaceGuid.PcdDebugMsg_Qspi|1
120
121 #
122 # Platform Init Assertion
123 #
124 # Suggest set to value 0 for RELEASE build
125 # Suggest set to value 1 when debugging
126 gAlteraHpsTokenSpaceGuid.PcdPlatformInitAllowAssertion|1
127 gAlteraHpsTokenSpaceGuid.PcdPlatformInitStopWhenAssert|1
128
129 #
130 # Pit Stop utility
131 # Suggest set to value 0 for RELEASE build
132 # Suggest set to value 1 to always enable
133 # Suggest set to value 2+ for timeout if no key pressed
134 gAlteraSocFpgaTokenSpaceGuid.PcdEnablePitStopUtility|5
135
136 # On-Chip-Memory Serial Log Base address and Size
137 # Come in handy for PlatformInit debug as alternative
138 # when UART fail and Semihosting does not fit the situation

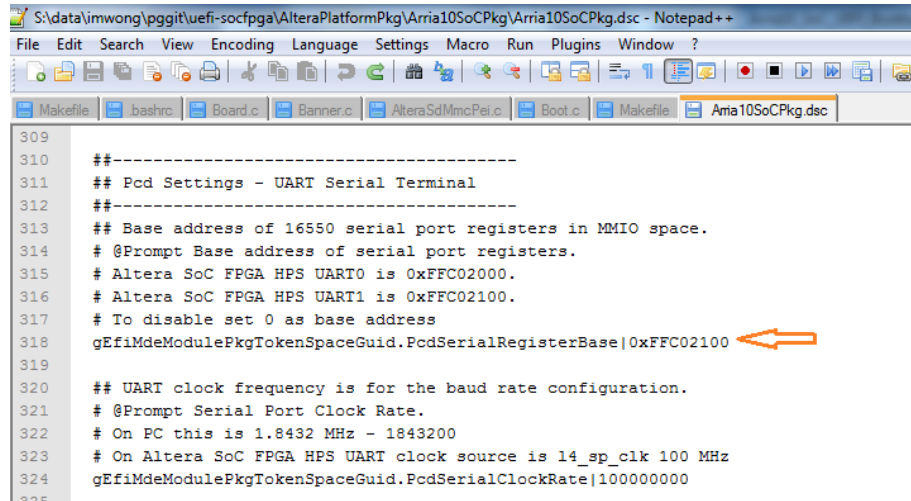
```

1.5.11.3. Modifying the UART Port Number for Serial Terminal Messages

The Intel Arria 10 SoC FPGA development board is uses UART1 for the serial terminal messages. If your custom board uses UART0 instead of UART1, you must change the UART base address so that your output messages display on the correct UART port.

1. Use your preferred editor to open the <your_uefi_socfpga>/uefi-socfpga/AlteraPlatformPkg/Arria10SoCPkg/Arria10SoCPkg.dsc file.
2. Configure the gEfiMdeModulePkgTokenSpaceGuid.PcdSerialRegisterBase value from 0xFFC02100 to 0xFFC02000.

Figure 113. Modifying the UART Serial Terminal Parameter



```
309
310  ##-----
311  ## Pod Settings - UART Serial Terminal
312  ##-----
313  ## Base address of 16550 serial port registers in MMIO space.
314  # @Prompt Base address of serial port registers.
315  # Altera SoC FPGA HPS UART0 is 0xFFC02000.
316  # Altera SoC FPGA HPS UART1 is 0xFFC02100.
317  # To disable set 0 as base address
318  gEfiMdeModulePkgTokenSpaceGuid.PcdSerialRegisterBase|0xFFC02100
319
320  ## UART clock frequency is for the baud rate configuration.
321  # @Prompt Serial Port Clock Rate.
322  # On PC this is 1.8432 MHz - 1843200
323  # On Altera SoC FPGA HPS UART clock source is l4_sp_clk 100 MHz
324  gEfiMdeModulePkgTokenSpaceGuid.PcdSerialClockRate|100000000
325
```

3. If you want to boot faster and do not want to see any output messages in the serial terminal, configure the `gEfiMdeModulePkgTokenSpaceGuid.PcdSerialRegisterBase` value to 0x0.

1.5.11.4. Adding Custom Board Initialization Code

You can add your custom board initialization code to the `<your_uefi_socfpga>/uefi-socfpga/AlteraPlatformPkg/Arria10SoCPkg/PlatformPei/Board.c` file.

Add your code in the function named `BoardSpecificInitialization` in the `Board.c` file. For the initial release of UEFI boot loader, the function call is at line 136, but the line number may change in the future.

Figure 114. Adding Custom Board Initialization Code

```

101 " 3 - 2GB DDR3 (256Mb x 40 x 2 Ranks)\r\n"
102 "\r\n";
103 InfoPrint ("SW1 DIP switch\r\n"
104 " 1 - I2C flag\r\n"
105 " 2 - FACTORY_LOAD\r\n"
106 " 3 - MSEL pins 1\r\n"
107 " 4 - MSEL pins 0\r\n"
108 "\r\n");
109 InfoPrint ("SW2 DIP switch\r\n"
110 " 1 - USER_DIPSW_HPS0\r\n"
111 " 2 - USER_DIPSW_HPS1\r\n"
112 " 3 - USER_DIPSW_HPS2\r\n"
113 " 4 - USER_DIPSW_HPS3\r\n"
114 " 5 - USER_DIPSW_FPGA0\r\n"
115 " 6 - USER_DIPSW_FPGA1\r\n"
116 " 7 - USER_DIPSW_FPGA2\r\n"
117 " 8 - USER_DIPSW_FPGA3\r\n"
118 "\r\n");
119 InfoPrint ("SW3 DIP switch\r\n"
120 " 1 - Board:\r\n"
121 " REV A0 - MAXV-A System Controller JTAG by pass\r\n"
122 " REV A1 - SOCHPS JTAG by pass\r\n"
123 " REV A1 - MAXV-A System Controller = Short J58, set SW3 DIP switch (on on on on off off on)\r\n"
124 " 2 - MAXV-B IO MUX Controller JTAG by pass\r\n"
125 " 3 - FMC-A JTAG by pass\r\n"
126 " 4 - FMC-B JTAG by pass\r\n"
127 " 5 - PCIe JTAG by pass\r\n"
128 " 6 - External JTAG by pass\r\n"
129 " 7 - CNTR0\r\n"
130 " 8 - CNTR1\r\n"
131 " Default: \r\n"
132 " USB Blaster: set SW3 DIP switch (off on on on on off off off)\r\n"
133 " Use DSTREAM: set SW3 DIP switch (off on on on on on on on )\r\n"
134 "\r\n");
135 #else
136 InfoPrint ("User's Code: Board Init...\r\n");
137 #endif
138 }
139

```

1.5.12. Enabling Checksum for the FPGA Image

Task time: 15 minutes

To prevent a system down caused by using a corrupted FPGA image, you can wrap the FPGA image with mkimage, and the mkimage header checksum can be used for verification to ensure the integrity of FPGA image is programmed into FPGA fabric.

Before programming the FPGA fabric, a section of FPGA image or full FPGA image is loaded to an on-chip RAM and SDRAM for computing. The checksum is verified as correct before the FPGA fabric is programmed.

1.5.12.1. Configuration in UEFI

1. To enable checksum for the FPGA image, use your preferred editor to open the <your_uefi_socfpga>/uefi-socfpga/AlteraPlatformPkg/Arria10SoCPkg/Arria10SoCPkg.dsc file.
2. Configure the gAlteraSocFpgaTokenSpaceGuid.PcdCheckFpgaImage value to 1.

Figure 115. Enable the FPGA Image Checksum Parameter

```

210 # Extra Boot Option for PEI Phase
211 gAlteraSocFpgaTokenSpaceGuid.PodBoot_LOAD_ZIMAGE_AT_PFI_PHASE 10
212 gAlteraSocFpgaTokenSpaceGuid.PodBoot_LOAD_UEFI_DXE_PHASE 10
213
214 # FPGA Image Checksum
215 gAlteraSocFpgaTokenSpaceGuid.PodCheckFpgaImage 1 ←
216 gAlteraSocFpgaTokenSpaceGuid.PodFileName_CORE_RBF_MKIMAGE "ghrd_10as066n2.core.rbf.mkimage"
217

```

3. Use your preferred editor to open the <your_uefi_socfpga>/uefi-socfpga/AlteraPlatformPkg/Arria10SoCPkg/Arria10SoCPkg.dts file.
4. Replace cff-file = "ghrd_10as066n2.periph.rbf" within the chosen node with cff-file = "ghrd_10as066n2.periph.rbf.mkimage".

Figure 116. Original cff-file Variable Value

```

31 /dts-v1/;
32
33 /*
34 * Bootloader: uboot
35 * Version: 1.0
36 */
37 / {
38     #address-cells = <1>;
39     #size-cells = <1>;
40     model = "SOCFPGA Arria10 Dev Kit"; /* Bootloader setting: uboot.model */
41
42     chosen {
43         cff-file = "ghrd_10as066n2.periph.rbf"; /* Bootloader setting: uboot.rbf_filename */
44         early-release-fpga-config;
45     };
46

```

Figure 117. cff-file Variable Value After Modification

```

31 /dts-v1/;
32
33 /*
34 * Bootloader: uboot
35 * Version: 1.0
36 */
37 / {
38     #address-cells = <1>;
39     #size-cells = <1>;
40     model = "SOCFPGA Arria10 Dev Kit"; /* Bootloader setting: uboot.model */
41
42     chosen {
43         cff-file = "ghrd_10as066n2.periph.rbf.mkimage"; /* Bootloader setting: uboot.rbf_filename */
44         early-release-fpga-config;
45     };
46

```

5. Type the following command to generate the Arria10SoCPkg.dtb file.

```
$ dtc -O dtb -o Arria10SoCPkg.dtb -I dts Arria10SoCPkg.dts
```

6. Run make to compile UEFI. This process may take some time to complete.

```
$ make
```

1.5.12.2. Generate mkimage

1. Type the following command to convert the file.

```
$ quartus/bin/quartus_cpf -c --hps -o bitstream_compression=on
<your_working_directory>/a10_soc_devkit_ghrd/output_files/ghrd_10as066n2.sof
<your_working_directory>/a10_soc_devkit_ghrd/output_files/ghrd_10as066n2.rbf
```

This command creates the following files in the <your_working_directory>/a10_soc_devkit_ghrd/output_files/ folder:

Table 14. Resultant RBF File Output

File	Description
~ /a10_soc_devkit_ghrd/output_files/ghrd_10as066n2.periph.rbf	I/O ring configuration file
~ /a10_soc_devkit_ghrd/output_files/ghrd_10as066n2.core.rbf	FPGA fabric configuration file

2. Type the following command to add the mkimage header to ghrd_10as066n2.periph.rbf:

```
$ mkimage -A arm -T firmware -C none -O u-boot -a 0 -e 0 -n "RBF" -d
ghrd_10as066n2.periph.rbf ghrd_10as066n2.periph.rbf.mkimage
```

3. Type the following command to add the mkimage header to the ghrd_10as066n2.core.rbf:

```
$ mkimage -A arm -T firmware -C none -O u-boot -a 0 -e 0 -n "RBF" -d
ghrd_10as066n2.core.rbf ghrd_10as066n2.core.rbf.mkimage
```

The following files are now in your <your_working_directory>/a10_soc_devkit_ghrd/output_files/ folder:

Figure 118. Resultant mkimage Files

```
[25/10/2016 - 13:50.55] [imwong.pg-nx8]
/data/imwong/GSRD/a10_soc_devkit_ghrd/output_files
$ ls -la
total 55828
drwxr-xr-x 2 imwong sofftrust 4096 Oct 25 13:50 .
drwxr-xr-x 8 imwong sofftrust 4096 Oct 24 15:01 ..
-rwxr-xr-x 1 imwong sofftrust 14654996 Oct 24 15:08 ghrd_10as066n2.core.rbf
-rw-r--r-- 1 imwong sofftrust 14655060 Oct 25 13:50 ghrd_10as066n2.core.rbf.mkimage
-rwxr-xr-x 1 imwong sofftrust 25 Oct 20 23:20 ghrd_10as066n2.done
-rwxr-xr-x 1 imwong sofftrust 476 Oct 20 23:18 ghrd_10as066n2.fit.msg
-rwxr-xr-x 1 imwong sofftrust 636 Oct 20 23:18 ghrd_10as066n2.fit.summary
-rwxr-xr-x 1 imwong sofftrust 54237 Oct 20 23:18 ghrd_10as066n2.jdi
-rwxr-xr-x 1 imwong sofftrust 7344 Oct 20 21:50 ghrd_10as066n2.map.msg
-rwxr-xr-x 1 imwong sofftrust 450 Oct 20 21:51 ghrd_10as066n2.map.summary
-rwxr-xr-x 1 imwong sofftrust 360456 Oct 24 15:08 ghrd_10as066n2.periph.rbf
-rw-r--r-- 1 imwong sofftrust 360520 Oct 25 13:39 ghrd_10as066n2.periph.rbf.mkimage
-rwxr-xr-x 1 imwong sofftrust 197758 Oct 20 23:17 ghrd_10as066n2.pin
-rwxr-xr-x 1 imwong sofftrust 606 Oct 20 23:18 ghrd_10as066n2.sld
-rwxr-xr-x 1 imwong sofftrust 26564094 Oct 20 23:19 ghrd_10as066n2.sof
-rwxr-xr-x 1 imwong sofftrust 11854 Oct 20 23:20 ghrd_10as066n2.sta.summary

[25/10/2016 - 13:52.33] [imwong.pg-nx8]
/data/imwong/GSRD/a10_soc_devkit_ghrd/output_files
$
```

4. Create the SD Card image with the PEI.ROM, ghrd_10as066n2.periph.rbf.mkimage, and ghrd_10as066n2.core.rbf.mkimage.

- a. Download the **make_sdimage.py** tool from [here](#).

Note: To obtain more information about the make_sdimage.py tool, please run the tool with the -h option:

```
$ ./make_sdimage.py -h
```

- b. Create a new folder and cd to it. Copy the required files that are listed in the table below into the current folder.

Table 15. Required Files for the make_sdimage.py Tool

File	Description
~/uefi-socfpga/Build/PEI.ROM	UEFI boot loader image
~/uefi-socfpga/Build/DXE.ROM	Optional second phase of UEFI boot loader image when you want to boot to UEFI Shell
~/a10_soc_devkit_ghrd/output_files/ghrd_10as066n2.core.rbf.mkimage	Compressed FPGA configuration file
~/a10_soc_devkit_ghrd/output_files/ghrd_10as066n2.periph.rbf.mkimage	Compressed FPGA I/O configuration file
<SoCEDs_installation_directory>/embeddedsocfpga/prebuilt_images/socfpga_arria10.dtb	Linux Device Tree Blob
<SoCEDs_installation_directory>/embeddedsocfpga/prebuilt_images/zImage	Compressed Linux kernel image
<SoCEDs_installation_directory>/embeddedsocfpga/prebuilt_images/angstrom-rootfs.tar.gz	Root filesystem as a compressed tarball

5. Create the root filesystem for the SD card image.

```
$ mkdir rootfs
$ cd rootfs
$ sudo tar xzf ../angstrom-rootfs.tar.gz .
```

6. Call the make_sdimage.py script by typing the following:

```
$ cd ../
$ sudo ./make_sdimage.py -f -P PEI.ROM,num=3,format=raw,size=10M,type=A2 -P
rootfs/*,num=2,format=ext3,size=1500M -P
zImage,DXE.ROM,ghrd_10as066n2.core.rbf.mkimage,ghrd_10as066n2.periph.rbf.mkim
age,socfpga_arria10.dtb,num=1,format=fat32,size=500M -s 2G -n
sd_card_image_a10_uefi.img
```

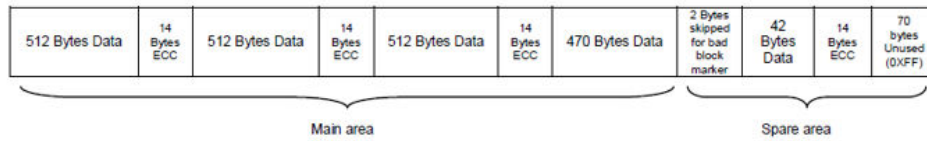
7. Power on the board with the above SD image. If the image is corrupt, the error message "FPGA: Bad Data Checksum" appears.

1.5.13. NAND Bad Block Management

NAND is unmanaged flash and requires software to manage it. NAND is not reliable compared to other flash types such as NOR. Therefore, new NAND flash may contain some bad blocks. This issue can also apply throughout the life cycle of a NAND flash where certain blocks may turn bad.

1.5.13.1. Bad Block Marker

A block is the smallest entity for erase. Although the bad area might happen at certain pages, the whole block is marked as a bad block. The bad block marker is in the spare area (OOB) within the first page of each block. A block is considered bad if the value of a bad block marker is not 0xFF.

Figure 119. NAND Layout


Note: The bad block marker location at the spare area varies according to page size.

Table 16. NAND Bad Block Location by Manufacturer

Manufacturer	Chip Type	Page Size	Device Size	Bad Block Marker Location
Micron & Others ⁽¹⁾	SLC	512 bytes	x8	6th byte at 1st page OOB
Micron & Others ⁽¹⁾	SLC	512 bytes	x16	1st byte at 1st page OOB
Micron & Others ⁽¹⁾	SLC	>512 bytes	Any	1st byte at 1st page OOB
Micron	MLC	Any	Any	1st byte at 1st page OOB
Samsung	MLC	Any	Any	1st byte at last page OOB
Hynix	MLC	Any	Any	1st byte at last page OOB
Samsung	SLC	Any	Any	1st byte at 2nd page OOB
Hynix	SLC	Any	Any	1st byte at 2nd page OOB
Toshiba	SLC	Any	Any	1st byte at 2nd page OOB
AMD	SLC	Any	Any	1st byte at 2nd page OOB
Macronix	SLC	Any	Any	1st byte at 2nd page OOB

1.5.13.2. Bad Block Handling

You must specify the number of bad blocks that can be skipped through the programming of a particular image. By default, the `gAlteraSocFpgaTokenSpaceGuid.PcdNandStopIfMoreThanThisNumberBadBlocks` value is configured to 1 which means the UEFI flags a fail operation when it encounters more than one bad block.

1. Use your preferred editor to open the `<your_uefi_socfpga>/uefi-socfpga/AlteraPlatformPkg/Arria10SoCPkg/Arria10SoCPkg.dsc` file.
2. Change the `gAlteraSocFpgaTokenSpaceGuid.PcdNandStopIfMoreThanThisNumberBadBlock` value to your requirements.

⁽¹⁾ Others refers to other configurations not mentioned in the table.

Note: You can choose to skip a bad block by specifying the skip or noskip option in the NAND command through the Pit Stop utility. For detailed information please refer to the *NAND Flash* sub-section in the *List of Pit Stop Utility Commands and Use* section.

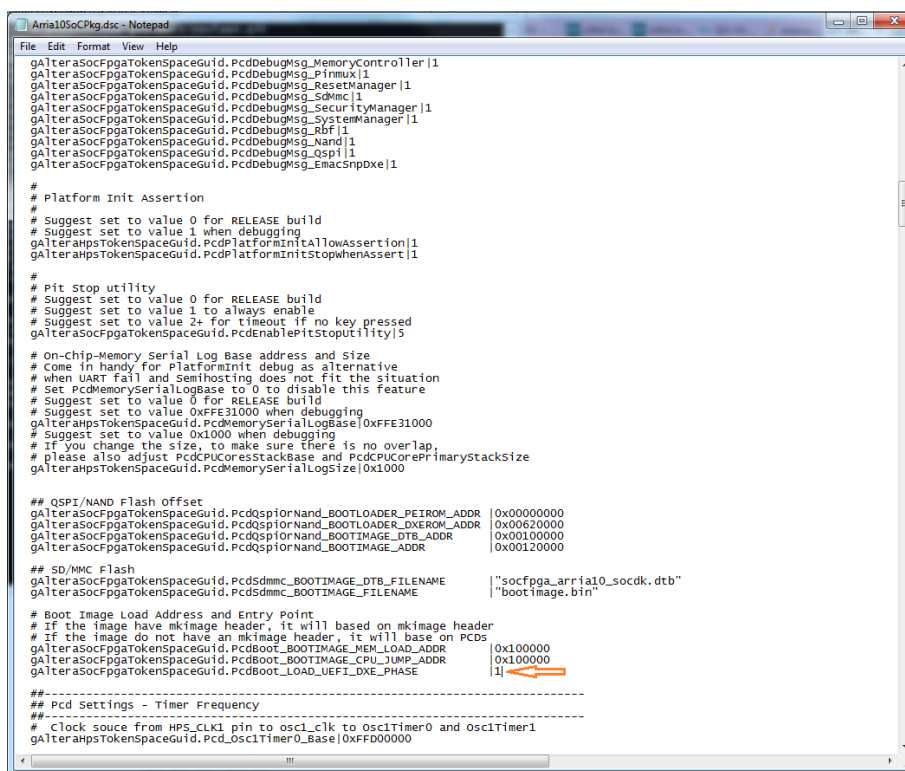
1.6. Enabling the UEFI DXE Phase and the UEFI Shell

Task time: 60 minutes

1. For a Windows build, ensure that the SoC FPGA EDS and Arm DS-5 Intel SoC FPGA Edition debugger tools are installed on your machine.
2. Complete the steps in [Obtaining the UEFI Source Code](#) on page 33.
3. To boot in the DXE phase, edit the ~:\uefisocfpga\AlteraPlatformPkg\Arria10SoCPkg\Arria10SoCPkg.dsc file to change the PcdBoot_LOAD_UEFI_DXE_PHASE token value from 0 to 1 before you run the make command. To edit the file type:

```
$ notepad AlteraPlatformPkg\Arria10SoCPkg\Arria10SoCPkg.dsc
```

Figure 120. Modifying the PcdBoot_LOAD_UEFI_DXE_PHASE Parameter



4. Verify that the SOCEDS_DEST_ROOT and DS5_ROOT is set under the Windows environment.
 - Set SOCEDS_DEST_ROOT=c:\altera\15.1\embedded
 - Set DS5_ROOT="C:\Progra~1\DS-5"

Note: If you install Arm DS-5 Intel SoC FPGA Edition in the Program Files, the path C:\Progra~1\DS-5 is to prevent a build error from occurring due to a space between the word "Program Files" in the path. For other locations, please set the path correctly.

5. Open a new Windows command prompt and execute the following commands:

```
~:\uefi-socfpga > setup
~:\uefi-socfpga> make
```

6. After the build is successful, you can copy the generated DXE.ROM file into the SD Card FAT32 partition.

1.7. Using the Network Feature Under the UEFI Shell

Task time: 20 minutes

This section describes how to use Ethernet features under the UEFI Shell after booting to the DXE Phase. The UEFI Shell network stack supports static and dynamic host configuration protocol (DHCP) IP Addressing. The basic functionality includes the standard network commands:

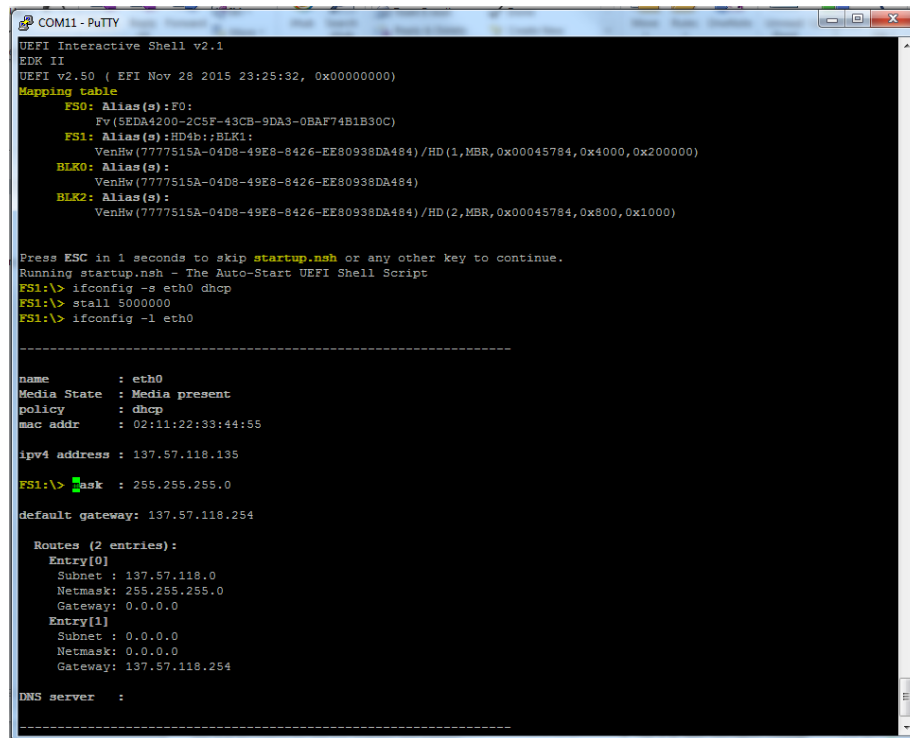
- ifconfig
- ping
- tftp

Verify that your the HPS Ethernet port on your board is connected to the network.

1.7.1. Obtain the IP Address from the DHCP Server

1. Type 1 to select the shell.
2. Observe the following messages that confirm that the DHCP works.

Figure 121. DHCP Confirmation Messages



```
COM11 - PuTTY
UEFI Interactive Shell v2.1
EDK II
UEFI v2.50 (EFI Nov 28 2015 23:25:32, 0x00000000)
Mapping table
FS0: Alias(s): F0:
    Fv(SEDA4200-2C5F-43CB-9DA3-0BAF74B1B30C)
FS1: Alias(s): HD4b::BLK1:
    VenHw(7777515A-04D8-49E8-8426-EE80938DA484)/HD(1,MBR,0x00045784,0x4000,0x200000)
BLK0: Alias(s):
    VenHw(7777515A-04D8-49E8-8426-EE80938DA484)
BLK2: Alias(s):
    VenHw(7777515A-04D8-49E8-8426-EE80938DA484)/HD(2,MSR,0x00045784,0x800,0x1000)

Press ESC in 1 seconds to skip startup.nsh or any other key to continue.
Running startup.nsh - The Auto-Start UEFI Shell Script
FS1:\> ifconfig -s eth0 dhcp
FS1:\> stall 5000000
FS1:\> ifconfig -l eth0

-----
name           : eth0
Media State    : Media present
policy         : dhcp
mac addr       : 02:11:22:33:44:55

ipv4 address   : 137.57.118.135
FS1:\> mask    : 255.255.255.0

default gateway: 137.57.118.254

Routes (2 entries):
Entry[0]
  Subnet : 137.57.118.0
  Netmask: 255.255.255.0
  Gateway: 0.0.0.0
Entry[1]
  Subnet : 0.0.0.0
  Netmask: 0.0.0.0
  Gateway: 137.57.118.254

DNS server :
```

Note: The first command, `if config -s eth0 dhcp`, that is shown in the screenshot above is to instruct the backend services to start DHCP initialization. On a typical corporate network, it may take up to 5 seconds for the DHCP server to assign the board an IP address. Thus, there is a `stall 5000000` command to delay 5 seconds.

The `ifconfig -l eth0` command lists the IP address assigned to this board. For complete help, type `help ifconfig` in the UEFI Shell.

1.7.2. Use a Static IP Address

To use the static IPv4 address configuration for the Ethernet interface, type the following command:

```
ifconfig -s eth0 static 192.168.0.5 255.255.255.0 192.168.0.1
```

where the parameters are:

```
IFCONFIG [-s <Name> dhcp | <static <IP> <Mask> <Gateway>> | <dns <IP>>]
```

Note: Type `help ifconfig` for a detailed explanation of the command.

1.7.3. Ping Test

This task describes how to pin the host name of the IP address.

1. Complete the steps in [Checking Your Machine or Laptop IPv4 Address](#) on page 143.
2. Type the following command:

```
ping 137.57.118.26
```

Note: 137.57.118.26 is the host IP address.

Figure 122. Ping Test Output

```
FS1:\> ping 137.57.118.26
Ping 137.57.118.26 16 data bytes.
16 bytes from 137.57.118.26 : icmp_seq=1 ttl=0 time<0ms
16 bytes from 137.57.118.26 : icmp_seq=2 ttl=0 time<0ms
16 bytes from 137.57.118.26 : icmp_seq=3 ttl=0 time<0ms
16 bytes from 137.57.118.26 : icmp_seq=4 ttl=0 time<0ms
16 bytes from 137.57.118.26 : icmp_seq=5 ttl=0 time<0ms
5 packets transmitted, 5 received, 0% packet loss, time 0ms
```

1.7.4. TFTP Test

This task describes how to verify that the TFTP is working correctly on your board.

1. Complete the steps in [TFTPd64 By Ph.Jounin Installation](#) on page 141.
2. Complete the steps in [Enabling the TFTP Server through the Windows Firewall](#) on page 143.
3. Complete the steps in [Using the TFTP Server in Windows](#) on page 143.
4. Type the following command to perform the tftp test:

```
tftp 137.57.118.26 1.png
```

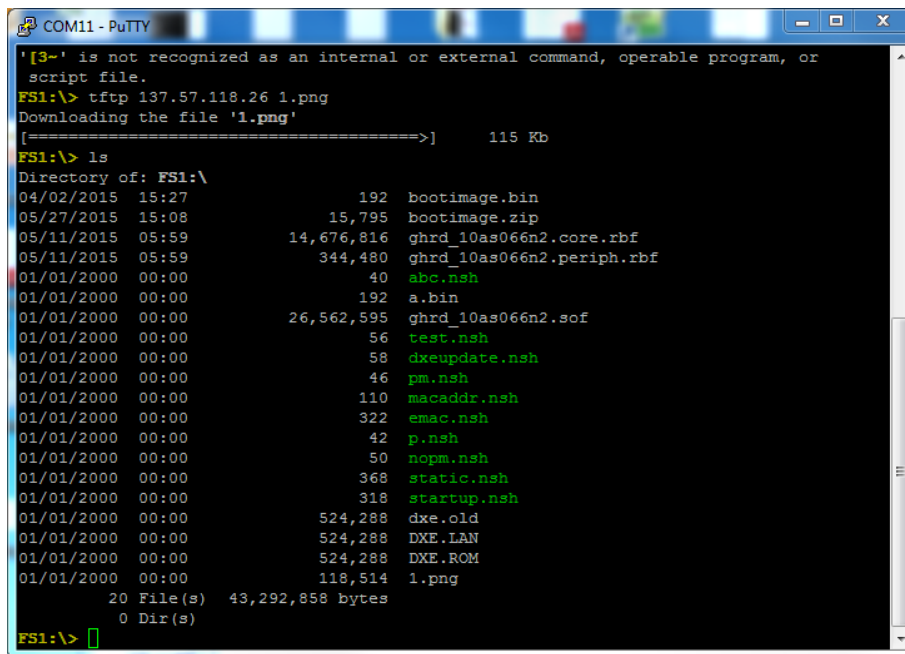
Note: 137.57.118.26 is the host IP address and 1.png is the file located in the base directory of the TFTP server.

Figure 123. tftp Command

```
FS1:\> tftp 137.57.118.26 1.png
Downloading the file '1.png'
[=====>] 115 Kb
FS1:\>
```

5. When the tftp is complete, list the directory to confirm that the 1.png file has transferred successfully.

Figure 124. List 1.png File Transfer



```
COM11 - PuTTY
'[3~' is not recognized as an internal or external command, operable program, or
script file.
FS1:\> tftp 137.57.118.26 1.png
Downloading the file '1.png'
[=====>]      115 Kb
FS1:\> ls
Directory of: FS1:\
04/02/2015  15:27                192  bootimage.bin
05/27/2015  15:08            15,795  bootimage.zip
05/11/2015  05:59      14,676,816  ghrd_10as066n2.core.rbf
05/11/2015  05:59      344,480   ghrd_10as066n2.periph.rbf
01/01/2000  00:00                40  abc.nsh
01/01/2000  00:00                192  a.bin
01/01/2000  00:00      26,562,595  ghrd_10as066n2.sof
01/01/2000  00:00                56  test.nsh
01/01/2000  00:00                58  dxupdate.nsh
01/01/2000  00:00                46  pm.nsh
01/01/2000  00:00               110  macaddr.nsh
01/01/2000  00:00               322  emac.nsh
01/01/2000  00:00                42  p.nsh
01/01/2000  00:00                50  nopm.nsh
01/01/2000  00:00               368  static.nsh
01/01/2000  00:00               318  startup.nsh
01/01/2000  00:00       524,288  dx.eold
01/01/2000  00:00       524,288  DXE.LAN
01/01/2000  00:00       524,288  DXE.ROM
01/01/2000  00:00       118,514  1.png
                20 File(s)  43,292,858 bytes
                0 Dir(s)
FS1:\> 
```

1.8. Creating your First UEFI Application

This task describes how to create and run a HelloWorldBaremetal and BlinkingLED_A1-SoCDevKit application on a Intel Arria 10 SoC FPGA RevB.1 board.

Task time: 60 minutes

1. Open a Windows command prompt and go to the ~:\uefi-socfpga directory. Type the following command to compile the HelloWorldBaremetal and BlinkingLED_A10SoCDevKit example application:

```
$ make app
```

Expect the compilation to take some time. Files are generated under the UEFI root ~:\uefi-socfpga\Build folder during this time.

Figure 125. make app File Output

```
Administrator: Git CMD
ppPkg\RELEASE_GCC48\ARM\WebServer.efi
1 file(s) copied.
copy /y u:\uefi-socfpga\Build\SocFpgaAppPkg\RELEASE_GCC48\ARM\AppPkg\Applications\Sockets\WebServer\WebServer\DEBUG\*.map u:\uefi-socfpga\Build\SocFpgaAppPkg\RELEASE_GCC48\ARM\AppPkg\Applications\Sockets\WebServer\WebServer\OUTPUT
u:\uefi-socfpga\Build\SocFpgaAppPkg\RELEASE_GCC48\ARM\AppPkg\Applications\Sockets\WebServer\WebServer\DEBUG\WebServer.map
1 file(s) copied.
make[2]: Leaving directory 'u:\uefi-socfpga\Build\SocFpgaAppPkg\RELEASE_GCC48\ARM\AppPkg\Applications\Sockets\WebServer\WebServer'
Build report can be found at U:\uefi-socfpga\report.log

- Done -
Build end time: 15:08:39, Jan.19 2016
Build total time: 00:37:19

make[1]: Leaving directory 'U:\uefi-socfpga'
Build\SocFpgaAppPkg\RELEASE_GCC48\ARM\HelloWorldBaremetal.efi
Build\SocFpgaAppPkg\RELEASE_GCC48\ARM\Main.efi
Build\SocFpgaAppPkg\RELEASE_GCC48\ARM\Hello.efi
Build\SocFpgaAppPkg\RELEASE_GCC48\ARM\BlinkingLED_A10SoCDevKit.efi
Build\SocFpgaAppPkg\RELEASE_GCC48\ARM\SocketDxe.efi
Build\SocFpgaAppPkg\RELEASE_GCC48\ARM\DataSink.efi
Build\SocFpgaAppPkg\RELEASE_GCC48\ARM\DataSource.efi
Build\SocFpgaAppPkg\RELEASE_GCC48\ARM\GetAddrInfo.efi
Build\SocFpgaAppPkg\RELEASE_GCC48\ARM\GetHostByAddr.efi
Build\SocFpgaAppPkg\RELEASE_GCC48\ARM\GetHostByDns.efi
Build\SocFpgaAppPkg\RELEASE_GCC48\ARM\GetHostByName.efi
Build\SocFpgaAppPkg\RELEASE_GCC48\ARM\GetNameInfo.efi
Build\SocFpgaAppPkg\RELEASE_GCC48\ARM\GetNetByName.efi
Build\SocFpgaAppPkg\RELEASE_GCC48\ARM\GetNetByDns.efi
Build\SocFpgaAppPkg\RELEASE_GCC48\ARM\GetServByName.efi
Build\SocFpgaAppPkg\RELEASE_GCC48\ARM\GetServByPort.efi
Build\SocFpgaAppPkg\RELEASE_GCC48\ARM\OobRx.efi
Build\SocFpgaAppPkg\RELEASE_GCC48\ARM\OobTx.efi
Build\SocFpgaAppPkg\RELEASE_GCC48\ARM\RawIp4Rx.efi
Build\SocFpgaAppPkg\RELEASE_GCC48\ARM\RawIp4Tx.efi
Build\SocFpgaAppPkg\RELEASE_GCC48\ARM\RecvDgram.efi
Build\SocFpgaAppPkg\RELEASE_GCC48\ARM\SetHostName.efi
Build\SocFpgaAppPkg\RELEASE_GCC48\ARM\SetSockOpt.efi
Build\SocFpgaAppPkg\RELEASE_GCC48\ARM\WebServer.efi
24 file(s) copied.
Build Done.
U:\uefi-socfpga>
```

2. Copy the following two UEFI application images into the SD card.

- ~:\uefi-socfpga\Build\HelloWorldBaremetal.efi
- ~:\uefi-socfpga\Build\BlinkingLED_A10SoCDevKit.efi

Figure 126. Copying UEFI Application Images

```
U:\uefi-socfpga>copy Build\HelloWorldBaremetal.efi d:\
1 file(s) copied.

U:\uefi-socfpga>copy Build\BlinkingLED_A10SoCDevKit.efi d:\
1 file(s) copied.

U:\uefi-socfpga>dir d:\
Volume in drive D is SOCFPGA
Volume Serial Number is 9EA3-E881

Directory of d:\

01/18/2016  07:22 PM             1,048,576  DXE.ROM
12/14/2015  06:53 PM      15,017,328  ghrd_10as066n2.rbf
01/19/2016  03:00 PM           67,008  HelloWorldBaremetal.efi
01/19/2016  03:01 PM          71,520  BlinkingLED_A10SoCDevKit.efi
               4 File(s)      16,204,432 bytes
               0 Dir(s)      1,053,331,456 bytes free

U:\uefi-socfpga>
```

3. Type the following commands under the UEFI shell on your board to run the UEFI application:

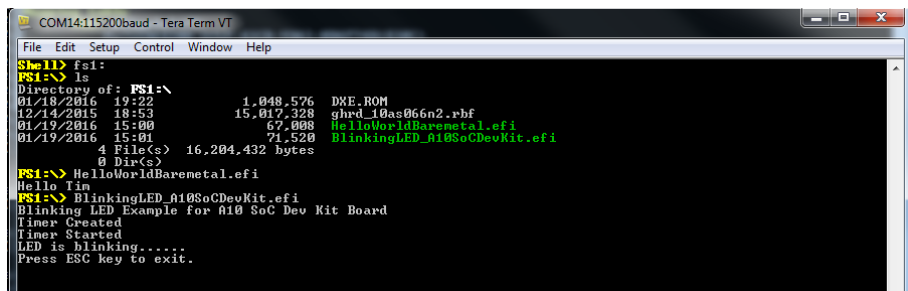
```
Shell> fs1:

FS1:\> HelloWorldBaremetal.efi

FS1:\> BlinkingLED_A10SoCDevKit.efi
```

Observe the following output messages on your serial terminal program:

Figure 127. Output Messages



```
COM14:115200baud - Tera Term VT
File Edit Setup Control Window Help
Shell> fs1:
FS1:\> ls
Directory of: FS1:\
01/18/2016  19:22             1,048,576  DXE.ROM
12/14/2015  18:53      15,017,328  ghrd_10as066n2.rbf
01/19/2016  15:00           67,008  HelloWorldBaremetal.efi
01/19/2016  15:01          71,520  BlinkingLED_A10SoCDevKit.efi
               4 File(s)      16,204,432 bytes
               0 Dir(s)
FS1:\> HelloWorldBaremetal.efi
Hello Tim
FS1:\> BlinkingLED_A10SoCDevKit.efi
Blinking LED Example for A10 SoC Dev Kit Board
Timer Created
Timer Started
LED is blinking.....
Press ESC key to exit.
```

1.9. Using Arm DS-5 Intel SoC FPGA Edition (For Windows Only)

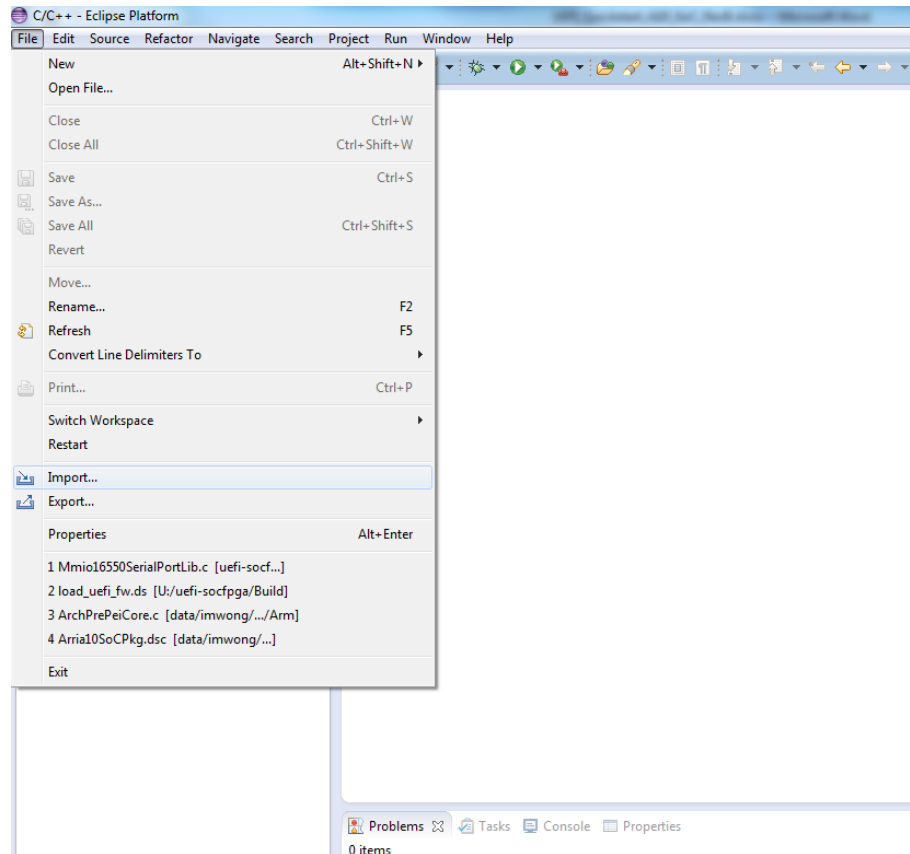
Task time: 15 minutes

1.9.1. Importing the DS-5 Project

This task describes how to import the pre-existing Arm DS-5 Intel SoC FPGA Edition project file in the UEFI Git repository's root directory into the Arm DS-5 Intel SoC FPGA Edition project.

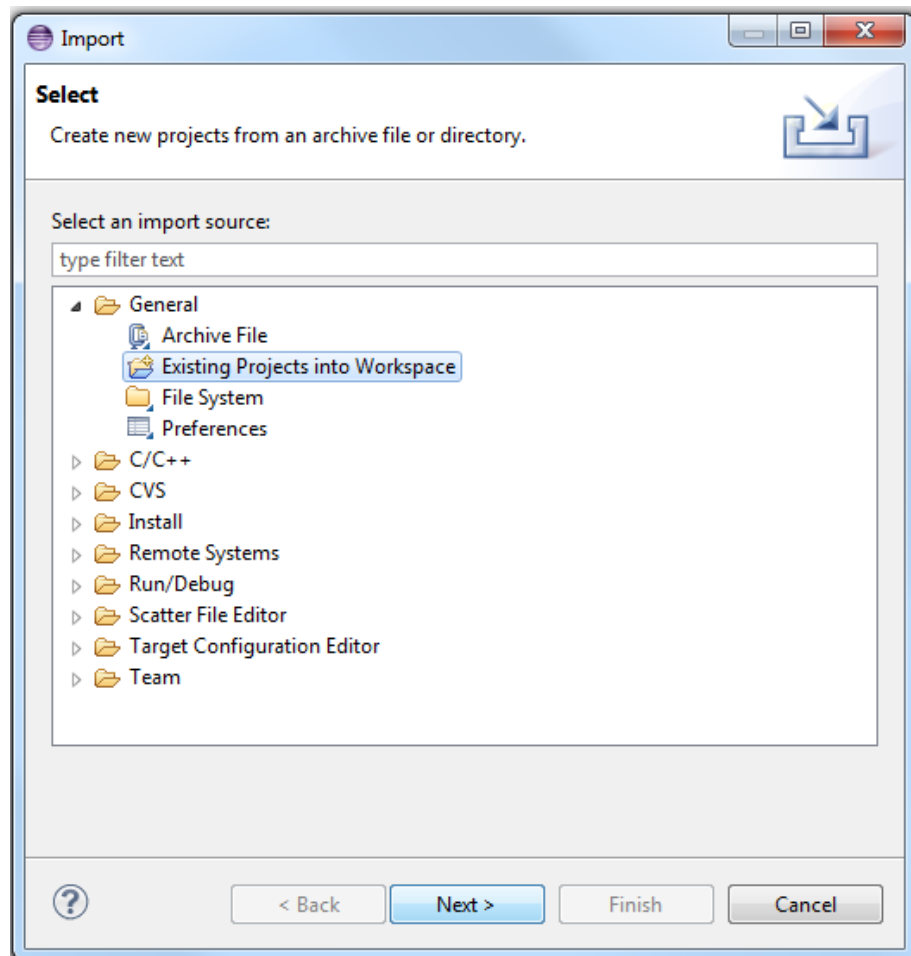
1. Launch Arm DS-5 Intel SoC FPGA Edition in Eclipse.
2. Select **File ► Import**.

Figure 128. File Import Selection in the Arm DS-5 Intel SoC FPGA Edition Eclipse Window



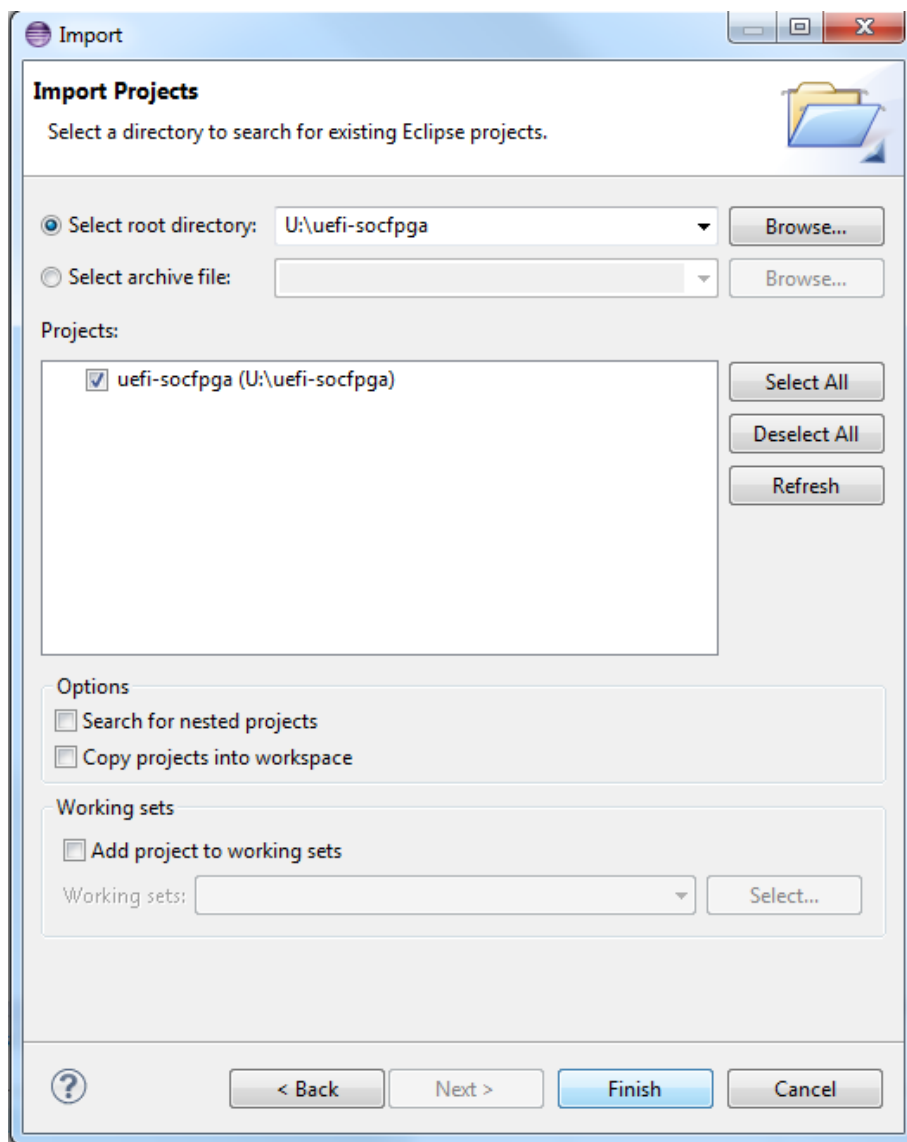
3. Select **General ► Existing Projects into Workspace**.

Figure 129. Selecting an Existing Project



4. Select the **Browse** button and browse to the root directory of the uefi-socfpga Git repository.

Figure 130. Browse to Project Location

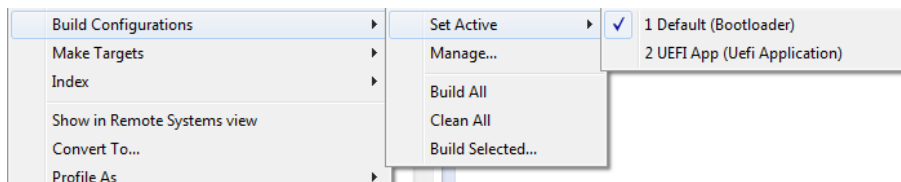


5. Click **Finish**.

1.9.2. Building UEFI in Arm DS-5 Intel SoC FPGA Edition

1. At the **Project Explorer** window in Eclipse, right click on the root node and select **Build Configurations ► Set Active**.
A list of build options that can be **Set Active** appear as shown below:

Figure 131. Build Configuration Options



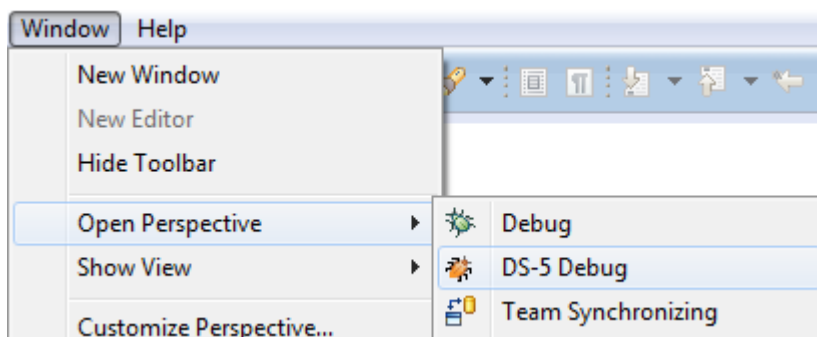
2. Select **Build All** to perform the make operation or make app operation. Alternatively, select **Clean All** to perform the make clean operation.

1.9.3. Importing the Arm DS-5 Intel SoC FPGA Edition Debug Script

This task describes how to import the UEFI debug script that is generated under the UEFI root ~\uefi-socfpga\Build folder after the UEFI build completes.

1. Switch to the Arm DS-5 Intel SoC FPGA Edition perspective by selecting **Windows** ► **Open Perspective** ► **DS-5 Debug**.

Figure 132. Opening DS-5 Debug



2. Select **Windows** ► **Show View** ► **Scripts**.
3. At the **Scripts explorer** window, click the **import script** icon.
4. Browse to the ~:\uefi-socfpga\Build folder and import the Arria10SoCPkg.ds5 file.

1.10. Pit Stop Utility Guide

Approximate time: 10 minutes

You can use the Pit Stop Utility to debug the Intel Arria 10 SoC UEFI Bootloader. This tool is available during the on-chip RAM boot phase after the HPS initialization and prior to booting to the subsequent SDRAM boot phase. When enabled, the tool

prompts you with the message “Press any key in N seconds to enter Pit Stop utility”. This light weight command-line interpreter provides a fixed list of useful debug commands, such as:

- Memory related commands: memory read, memory write, memory dump and memory compare.
- QSPI/NAND Flash commands: probe, read, write, erase and update.
- SD/MMC commands: block read and write.
- FAT32 file system commands: list files, HEX dump of file and reading an entire file into memory.
- CPU jump command: hands off to a program loaded at a specific memory offset address.

1.10.1. Enabling or Disabling the Pit Stop Utility

Intel recommends that you enable this debug feature during the development stage and disable it prior to product release.

1. Use your preferred editor to open the `<your_uefi_socfpga>/uefi-socfpga/AlteraPlatformPkg/Arria10SoCPkg/Arria10SoCPkg.dsc` file.
2. Configure the `gAlteraSocFpgaTokenSpaceGuid.PcdEnablePitStopUtility` to your requirements:
 - 0 to disable
 - 1 to enable

Figure 133. Modifying the Pit Stop Utility Parameter

1.10.2. Pit Stop Utility Commands and Use

1.10.2.1. Memory and Memory-Mapped I/O Commands

Command	Usage	Description
Read	mr addr [count]	32-bit memory read. Read a 32-bit word (4 bytes) of data from addr and display the data in HEX format. addr: memory address, must be 32-bit aligned. count: If not specified, the default value is 1. If specified, increase the address by a 32-bit word (4 bytes) for the next count.
Write	mw addr data [count]	32-bits memory write. Write a 32-bit word (4 bytes) of data to addr. If the count is more than one, the same piece of data is written to the subsequence address. addr: memory address must be 32-bit aligned.

continued...

Command	Usage	Description
		count: If not specified, the default value is 1. If specified, increase the address by a 32-bit word (4 bytes) for the next count.
Compare	mcmp addr1 addr2 [count]	32-bit memory compare. Compare a 32-bit word (4 bytes) of data between two different 32-bit memory locations for a count iteration and print out the differences in the process. addr: memory address, must be 32 bits aligned. count: If not specified, the default value is 1. If specified, increase the address by a 32-bit word (4 bytes) for the next count.
Dump	mdmp addr len	Memory Hex Dump. Each byte (8-bits) is represented as a two-digit hexadecimal number. addr: memory address, must be 32-bit aligned. len: number of bytes, must be 32-bit aligned.

1.10.2.2. QSPI Flash Commands

Command	Usage	Description
Probe	qspi probe	Initialize flash device
Read	qspi read addr offset len	Read 'len' bytes starting at 'offset' to memory at 'addr'
Write	qspi write addr offset len	Write len bytes from memory at addr to flash at offset
Erase	qspi erase offset len	Erase len bytes of data from the offset
Update	qspi update addr offset len	Erase and write len bytes from memory at addr to flash at offset

1.10.2.3. NAND Flash Commands

Command	Usage	Description
Probe	nand probe	Initialize flash device
Bad	nand bad	Report the bad blocks
Read raw	nand read addr offset len raw	Read raw data len bytes starting at flash offset to memory at addr
Read skip	nand read addr offset len skip	Read len bytes starting at flash offset to memory at addr but skip bad block
Read noskip	nand read addr offset len noskip	Read len bytes starting at flash offset to memory at addr without skip bad block
Read spare	nand read addr block page spare	Read spare data len bytes starting at flash offset to memory at addr
Write raw	nand write addr offset len raw	Write raw data len bytes from memory at addr to flash at offset
Write skip	nand write addr offset len skip	Write len bytes from memory at addr to flash at offset but skip bad block
continued...		

Command	Usage	Description
Write noskip	nand write addr offset len noskip	Write len bytes from memory at addr to flash at offset without skip bad block
Write spare	nand write addr block page spare	Write spare data len bytes from memory at addr to flash at offset
Erase skip	nand erase offset len skip	Erase len bytes from offset but skip bad blocks
Erase noskip	nand erase offset len noskip	Erase len bytes from offset without skip bad blocks
Update skip	nand update addr offset len skip	Erase, then write len bytes from memory at addr to flash at offset but skip bad blocks
Update noskip	nand update addr offset len noskip	Erase then write len bytes from memory at addr to flash at offset without skip bad blocks

1.10.2.4. SD/MMC Commands

Command	Usage	Description
Read	mmc read addr offset blockcnt	Block Read. addr: memory address to read the data. offset: LBA block number to read from. blockcnt: number of blocks of data.
Write	mmc write addr offset blockcnt	Block Write. addr: memory address to write the data. offset: LBA block number to write to. blockcnt: number of blocks of data.

1.10.2.5. FAT32 File System Commands

Command	Usage	Description
List files	ls	Display a list of files in root directory of FAT32 partition.
Read file	fatload mmc addr file	Load a file into a memory location. addr: destination memory address. file: filename of a file located in root directory of a FAT32 partition
HexDump File	hexdump file	Display a file as HEX dump. file: filename of a file located in root directory of a FAT32 partition.

1.10.2.6. Others

Command	Usage	Description
Jump	jump addr	Transfers the flow of execution to the entry addr
Boot zImage	bootz [filename.dtb]	Boot Linux: SoC FPGA zImage filename.dtb: filename of a Linux .dtb file located in root directory of a FAT32 partition. If unspecified, it is based on the PcdFileName_LINUX_DTB parameter.
Program core rbf	programrbf flashtype rbftype file/offset	Program the core .rbf file into the SD/MMC, QSPI or NAND flash memory.

continued...

Command	Usage	Description
		If you are using SD/MMC flash: • flashtype: mmc • rbftype: core or peripheral or combined • file: filename of your SD/MMC core.rbf file If you are using a QSPI flash: • flashtype: qspi • rbftype: core or peripheral or combined • offset: in this case, the offset address which is 0x820000 If you are using NAND flash: flashtype: nand rbftype: core or peripheral or combined offset: in this case, the offset address which is 0x820000

1.10.2.7. Help

Command	Usage	Description
help	help	List all available commands in the Pit Stop Utility

1.11. Porting HWLIBs to UEFI Guidelines

Task time: 10 minutes

1.11.1. Coding Standard

UEFI follows the EDK2 coding standard v2 instead of the C coding standard in HWLIBs. The following sections highlight the EDK2 coding standard v2 guidelines.

Related Information

[EDK2 Coding Standard v2](#)

For more detail about the coding guidelines

1.11.1.1. General Rules

1. Do not use tab characters. Use spaces instead.
2. Start a new line with double spacing.

1.11.1.2. Naming Conventions

Start each word or concept with a capital letter followed by lowercase letters.

- Example: UefiVariable

1.11.1.3. Type and Macro Names

1. Use all capital letters for both #define and typedef declarations. This clearly differentiates static declarations from dynamic data types.

```
UINT32 UefiVariable
```


- Each word of a concept shall be separated by an underscore character. The underscore effectively separates the words, making the names more readable.

```
typedef UINT32 THIS_IS_AN_EXAMPLE_OF_WHAT_TO_DO_FOR_UEFI
typedef struct MyStruct {
    int one;
    int two;
    int three;
} MY_STRUCT;
```

- The use of the `_t` suffix, designating a type, is not allowed.
- The names of guard macros shall end with an underscore character.

```
#ifndef FILE_NAME_H_
#define FILE_NAME_H_
```

1.11.1.4. Variables

In UEFI, modular variables and global variables replace the static and global variables found in HWLIBs. Any variable with a file scope, or better, shall be prefixed by an `m` or `g`. There are no exceptions to this rule. The `m` prefix identifies a variable with module scope, while a `g` prefix identifies a global variable.

- Example global variable: `gThisIsAGlobalVariableName`
- Example modular variable: `mThisIsaModuleVariableName`

1.11.1.5. Function

Each type specification for the argument variable must be preceded by `IN` and/or `OUT` modifiers. The modifiers are used to indicate whether the argument is an input or output variable. It is recommended that the `IN` variables are first and `OUT` variables are next. If data passes in and out through a variable, mark it as both `IN` and `OUT`. Mark a buffer that passes into a routine that modifies the contents of the buffer as `IN` and `OUT`. `EFIAPI` is an optional functional modifier.

```
EFI_STATUS
EFIAPI
FunctionName (
    IN UINTN Arg1,
    IN UINTN Arg2,
    OUT UINTN *Arg3,
    IN OUT UINTN * Arg4
);
{
    UINTN Local;
    ...
}
```

1.11.1.6. Comment

All comments must start with double slash `//` instead of `/*` and `*/`.

```
// This is my comment
```

1.11.2. Data Types Conversion

Table 17. Conversion of Data Types from HWLIBS to UEFI

HWLIB	UEFI
uint8_t	UINT8
uint16_t	UINT16
uint32_t	UINT32
uint64_t	UINT64
unsigned int	UINTN
char	CHAR8
bool	BOOLEAN
const	CONST
void	VOID
int	INTN
int8_t	INT16
int32_t	INT32
int64_t	INT64

1.11.3. Status Code Conversion

Table 18. Status Code Conversion from HWLIBs to UEFI

HWLIB	UEFI
ALT_STATUS_CODE	EFI_STATUS
ALT_E_SUCCESS	EFI_SUCCESS
ALT_E_ERROR	EFI_DEVICE_ERROR
ALT_E_FPGA_CFG	N/A
ALT_E_FPGA_CRC	EFI_CRC_ERROR
ALT_E_FPGA_CFG_STM	N/A
ALT_E_FPGA_PWR_OFF	N/A
ALT_E_FPGA_NO_SOC_CTRL	N/A
ALT_E_FPGA_NOT_USER_MODE	EFI_NOT_READY
ALT_E_ARG_RANGE	EFI_UNSUPPORTED
ALT_E_BAD_ARG	EFI_NOT_FOUND
ALT_E_BAD_OPERATION	N/A
ALT_E_INV_OPTION	EFI_INVALID_PARAMETER
ALT_E_TMO	EFI_TIMEOUT
ALT_E_RESERVED	EFI_ACCESS_DENIED
continued...	

HWLIB	UEFI
ALT_E_BAD_CLK	N/A
ALT_E_BAD_VERSION	EFI_INCOMPATIBLE_VERSION
ALT_E_BUF_OVF	EFI_VOLUME_FUL

1.11.4. Header Conversion

Header	HWLIB	UEFI
IO memory access	<stdio.h>	<Library/IoLib.h>
Memory	<stdio.h>	<Library/BaseMemoryLib.h> <Library/MemoryAllocationLib.h>
Debug	<stdio.h>	<Library/DebugLib.h> <Library/PrintLib.h> <Library/SerialPortPrintLib.h>
Timer	<time.h>	<Library/TimerLib.h>
Assert	<assert.h>	Assert.h
String/String Conversion	<string.h> <stdlib.h>	<Library/BaseLib.h>
Device Tree	N/A	<libfdt.h>
Math	<math.h>	<Library/BaseLib.h>

1.11.5. Basic I/O Functions Conversion

Table 19. Basic I/O Function Conversion from HWLIBs to UEFI

HWLIB	UEFI
alt_write_word	MmioWrite32
alt_read_word	MmioRead32
alt_setbits_word	MmioOr32
alt_clrbits_word	MmioAnd32

As an example, in HWLIBs the function:

```
alt_write_word (data, address);
```

must be written in the following way for UEFI:

```
MmioWrite32 (address, data);
```

1.12. Tera Term Installation

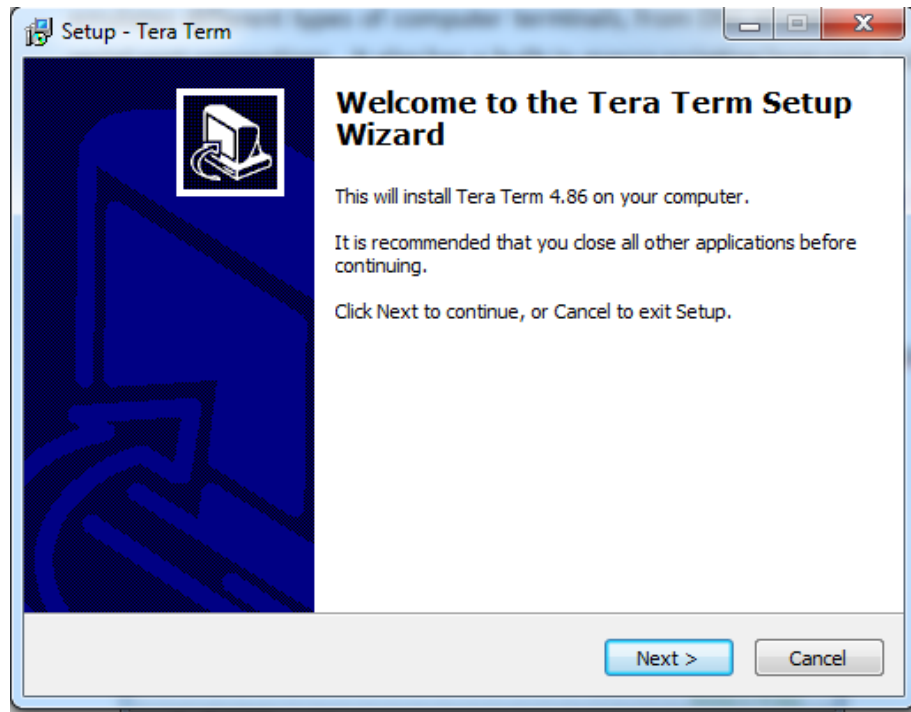
Task time: 20 minutes

Tera Term is an open-source, free, software implemented, terminal emulator (communications) program. It emulates different types of computer terminals, from DEC VT100 to DEC VT382. It supports telnet, SSH 1 and SSH 2 and serial port connections. It also has a built-in macro scripting language and useful plugins.

This task describes how to install Tera Term into your machine.

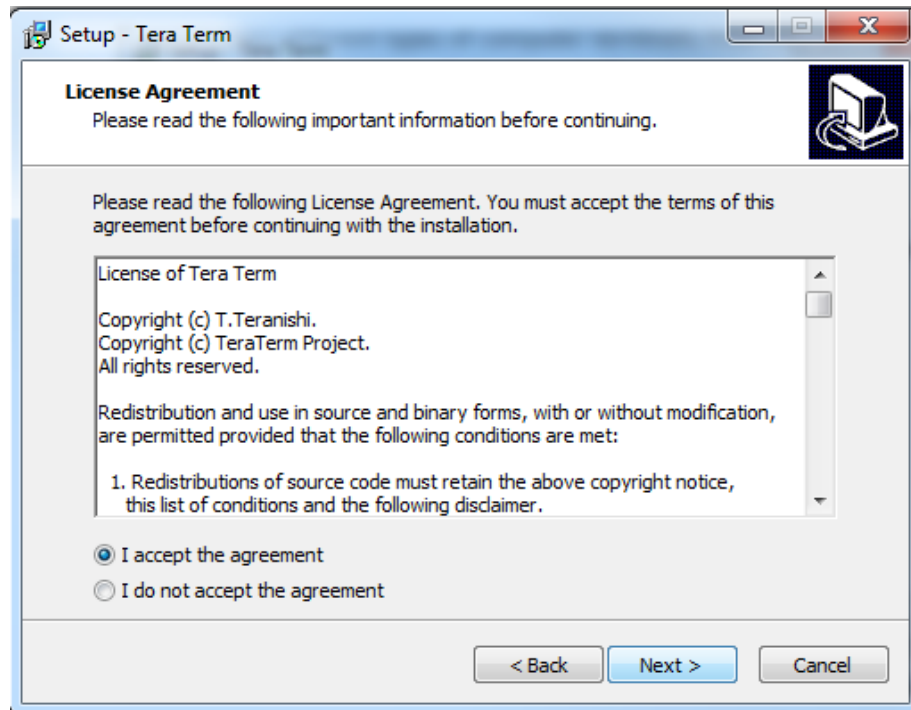
1. Download the tool from [here](#).
2. Download the installer teraterm486.exe into your local drive.
3. Double-click on the Tera Term installer **teraterm486.exe**.
4. The **Setup – Tera Term** window appears. Click **Next** to continue the setup.

Figure 134. Tera Term Setup Wizard



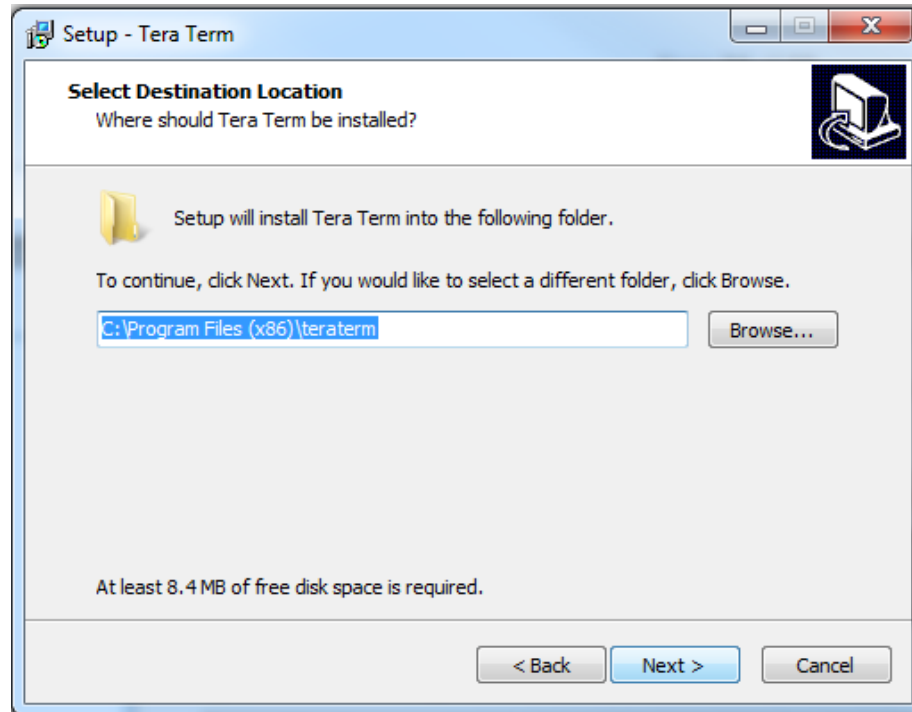
5. Check the radio button for **I accept the agreement** and click **Next**.

Figure 135. Accepting the Tera Term Agreement



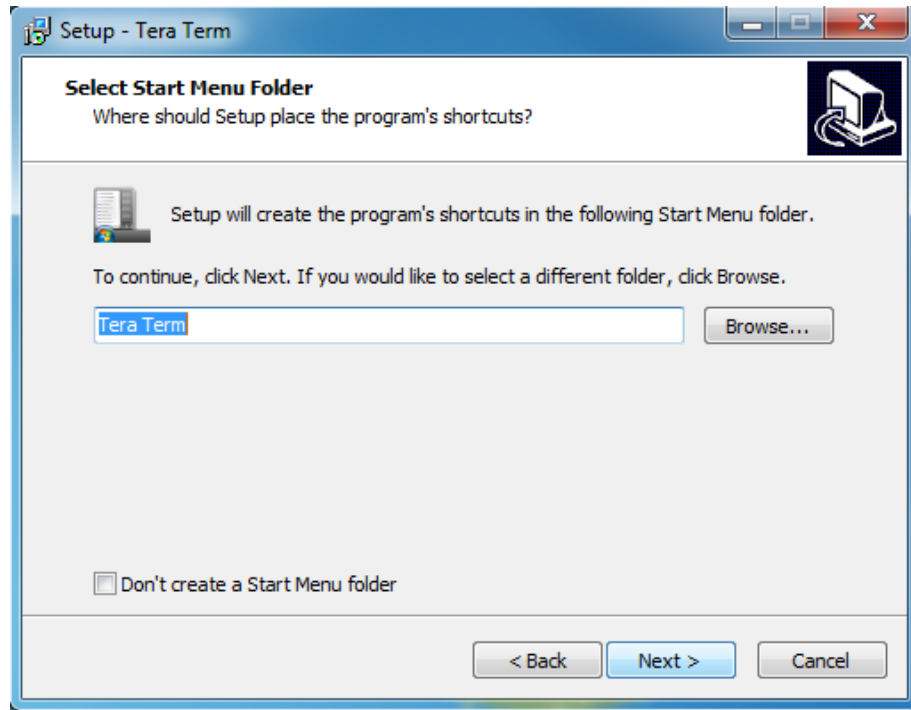
6. Leave the installation folder as default and click **Next** to proceed with the installation.

Figure 136. Selecting the Installation Folder



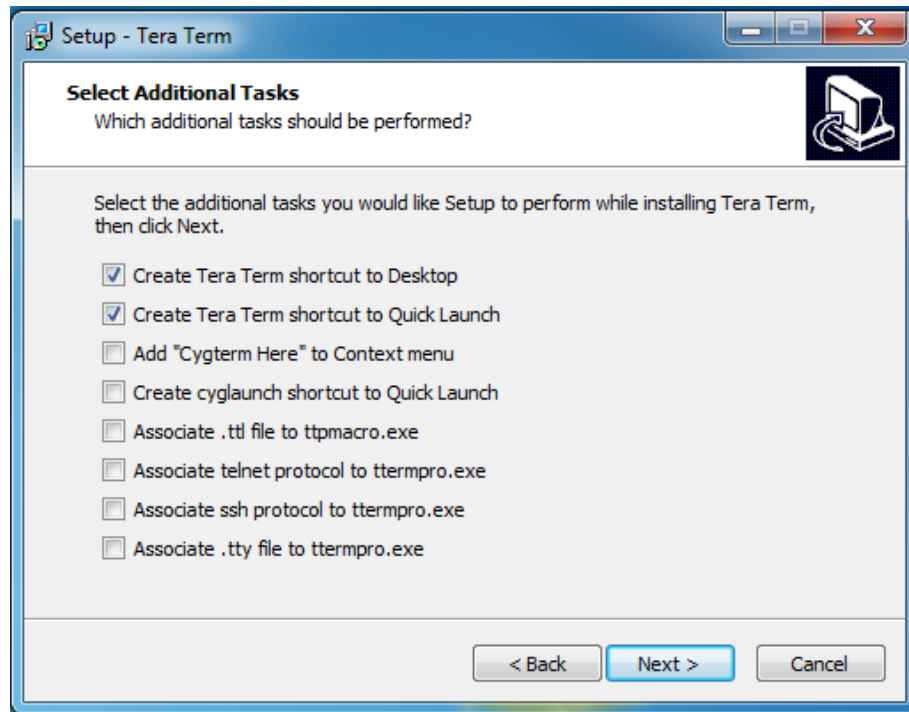
7. When the **Select Components** topic appears, click **Next**.
8. Select **English** (default) as the language to be used and click **Next**.
9. Leave the **Start Menu Folder** path as the default of **Tera Term** and click **Next**.

Figure 137. Tera Term Start Menu Folder



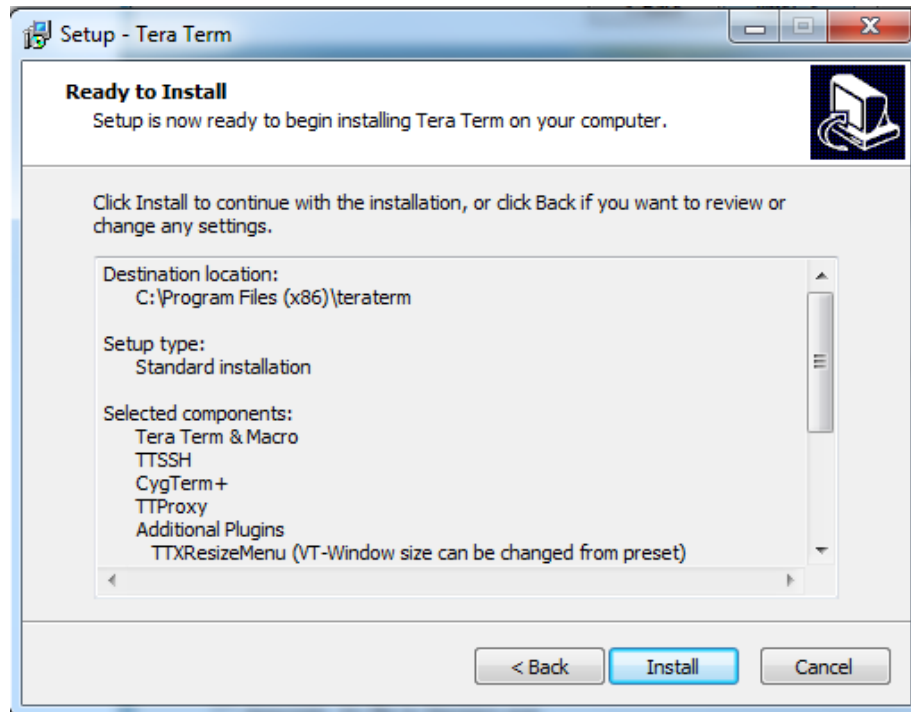
10. Leave the **Additional Tasks** selection as default and Click **Next** to continue.

Figure 138. Tera Term Additional Tasks Selection



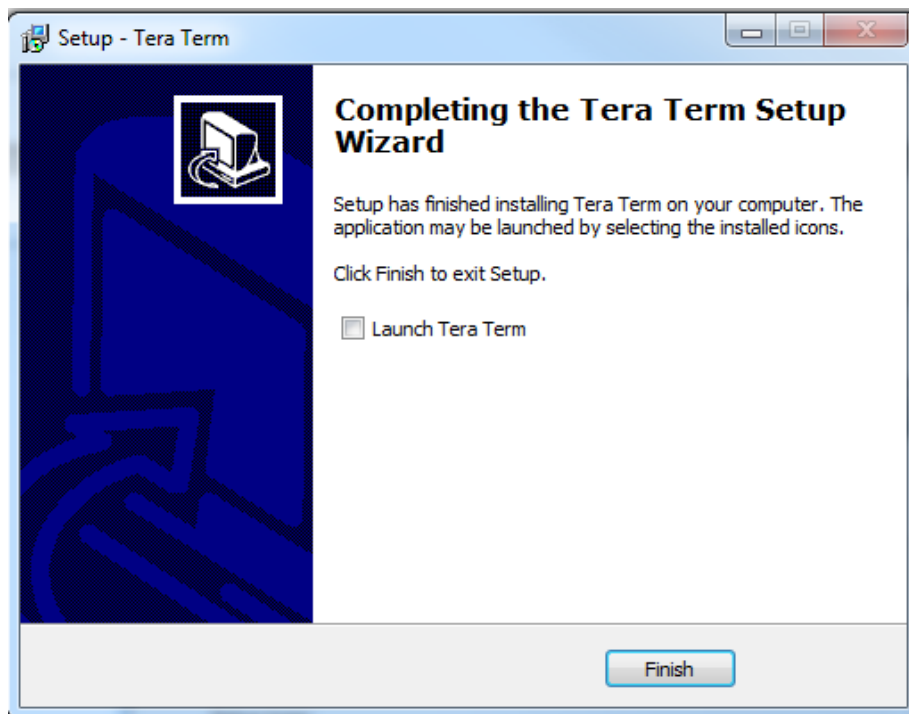
11. When the **Ready to Install** window appears, click **Install**.

Figure 139. Ready to Install Selection



12. When the installation wizard finishes, the **Completing the Tera Term Setup Wizard** window appears. Click **Finish** to exit the Setup. Tera Term is installed.

Figure 140. Completing the Tera Term Setup Wizard

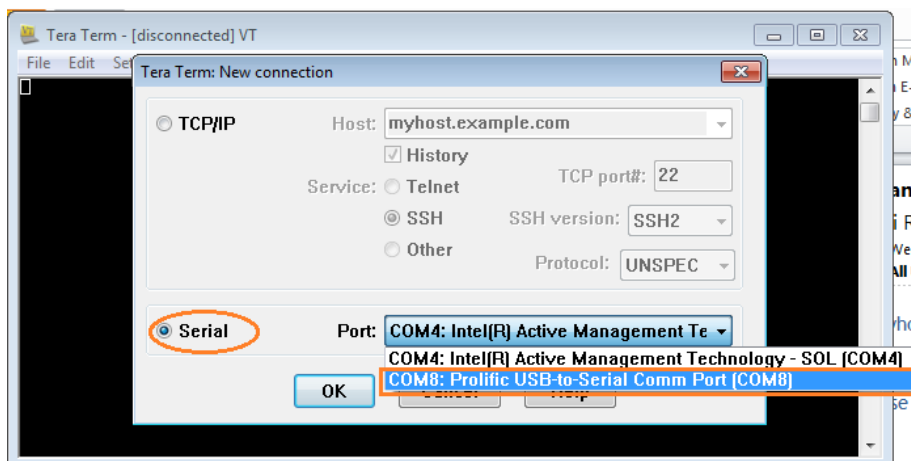


1.12.1. Making a Serial Connection

1. Start Tera Term by selecting **Start > All Programs > Tera Term > Tera Term** in the Program Files menu selection or by double-clicking on the **Tera Term** icon on your desktop.
2. When the **Tera Term: New connection** pop-up window appears within the program, select which serial port you would like to enable.
 - a. Select the **Serial** radio button.
 - b. Select your port from the **Port** drop down menu.
 - c. Press OK

Note: If the **New connection** window does not open when you start Tera Term, you can get there by navigating to **File > New Connection....**

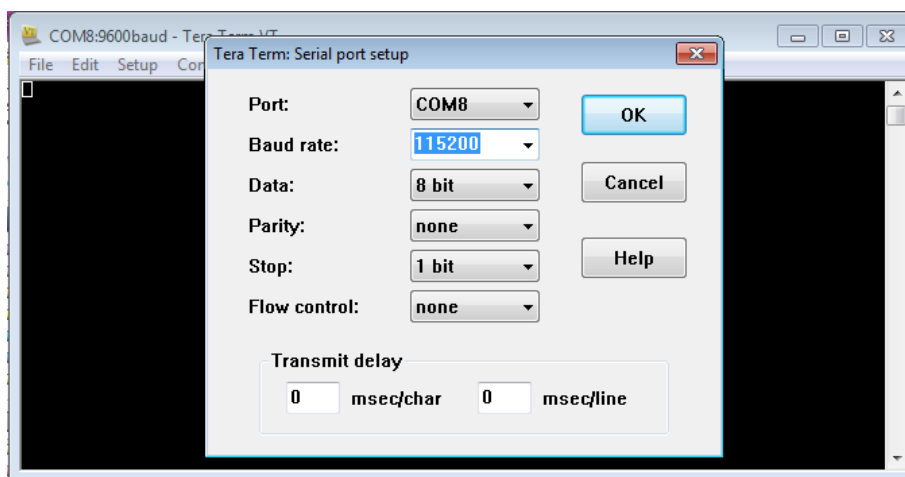
Figure 141. Tera Term Serial Port Selection



The serial port connection opens. Tera Term defaults to a baud rate setting of 9600 bps (8 bit data-no parity-1 stop bit).

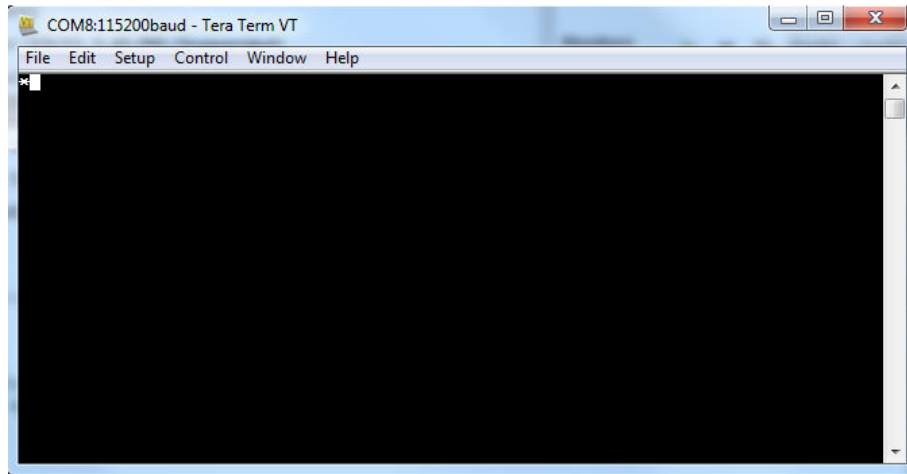
3. If you need to adjust the serial settings, go to **Setup > Serial port....**
4. In the **Tera Term: Serial port setup** window, adjust the settings in each of the pull down selections and click **OK**. In the example below, the serial port has a configuration of 115200 bps, 8-bit data, no parity and 1 stop bit.

Figure 142. Serial Port Setup Selection



The title of the Tera Term window changes to **COM8:115200baud – Tera Term VT**. The blank window with the blinking cursor is where data is both sent (by typing) and received.

Figure 143. New Tera Term Window



5. Configure your **Tera Term: Keyboard setup** as shown below:

Figure 144. Tera Term Keyboard Setup



1.13. Minicom Installation

Task time: 15 minutes

Minicom is a text-based modem control and terminal emulation program for Unix-like operating systems. A common use for Minicom is to setup a remote serial console.

1. The virtual serial port is usually named **/dev/ttyUSB0**. In order to determine the device name associated with the virtual serial port on your host PC, please perform the following steps:
 - a. Type **ls /dev/ttyUSB*** to determine which USB serial device are already installed
 - b. Connect mini USB cable from J10 to the PC. This will enable the PC to communicate with the board, even if the board is not powered yet.

- c. Use the **ls /dev/ttyUSB*** command again to determine which new USB serial device appeared.
2. On CentOS, type:

```
sudo yum install minicom
```
3. On Ubuntu, type:

```
sudo apt-get install minicom
```

1.13.1. Making a Serial Connection

1. Configure Minicom.

```
$ sudo minicom -s
```

A **configuration** menu appears.

Figure 145. Configuration Menu

```
+-----[configuration]-----+
| Filenames and paths          |
| File transfer protocols      |
| Serial port setup           |
| Modem and dialing            |
| Screen and keyboard          |
| Save setup as dfl             |
| Save setup as..              |
| Exit                         |
| Exit from Minicom            |
+-----+-----+-----+-----+
```

2. Under the **Serial port setup**, choose the following:
 - Serial device: **/dev/ttyUSB0** (edit to match the system as necessary)
 - Bps/Par/Bits: **115200 8N1**
 - Hardware Flow Control: **No**
 - Software Flow Control: **No**

Figure 146. Serial Port Setup

```

+-----+
| A -   Serial Device       : /dev/modem |
| B - Lockfile Location    : /var/lock   |
| C - Callin Program       :             |
| D - Callout Program      :             |
| E -   Bps/Par/Bits       : 115200 8N1  |
| F - Hardware Flow Control : Yes        |
| G - Software Flow Control : No         |
|                                     |
|      Change which setting?           |
+-----+
| Screen and keyboard |
| Save setup as dfl   |
| Save setup as..    |
| Exit                |
| Exit from Minicom   |
+-----+

```

3. Hit **[ESC]** to return to the main configuration menu.
4. Select **Save setup as dfl** to save the default setup.
5. Select **Exit** to exit the menu.

1.14. Win32DiskImager Tool Installation

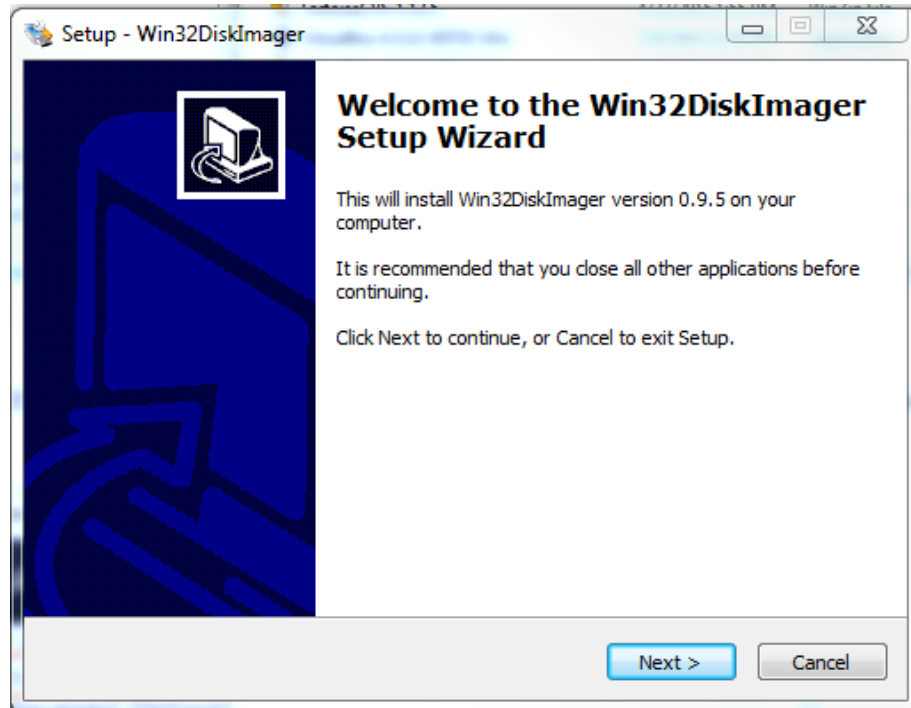
Task time: 10 minutes

Win32DiskImager writes a raw disk image to a removable device or can backup a removable device to a raw image file. It is very useful for embedded development, namely Arm development projects.

To install Win32DiskImager into your machine, execute the following steps:

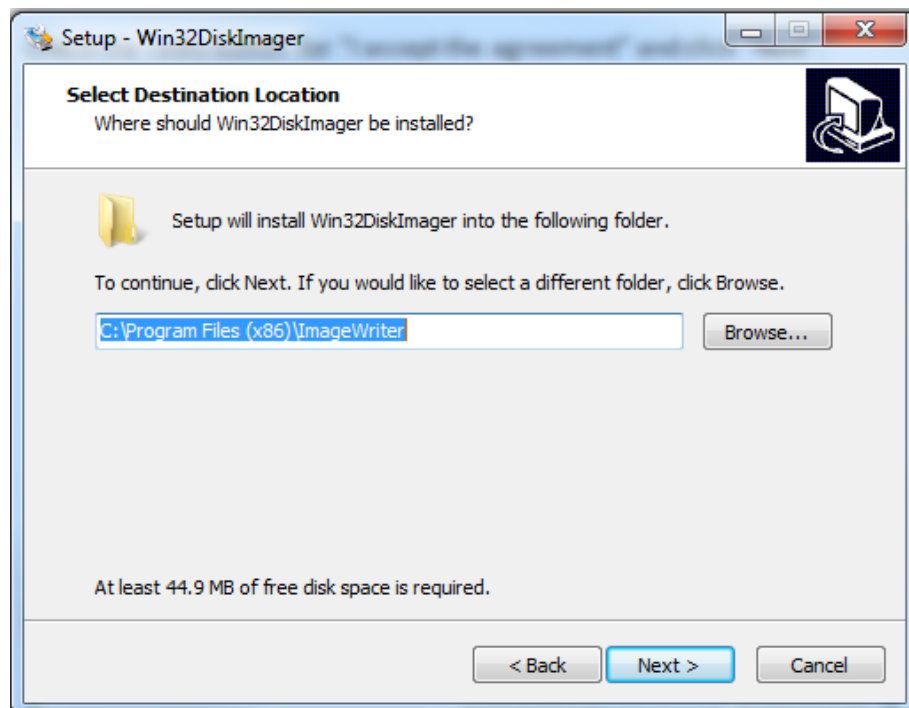
1. Download the tool from [here](#).
2. Download the installer **Win32DiskImager-0.9.5-install.exe** into your local drive.
3. Double-click the installer **Win32DiskImager-0.9.5-install.exe** to start the installation for Win32DiskImager into your local machine.
4. When the **Setup – Win32DiskImager** window appears, click **Next** to proceed.

Figure 147. Win32DiskImager Setup Window



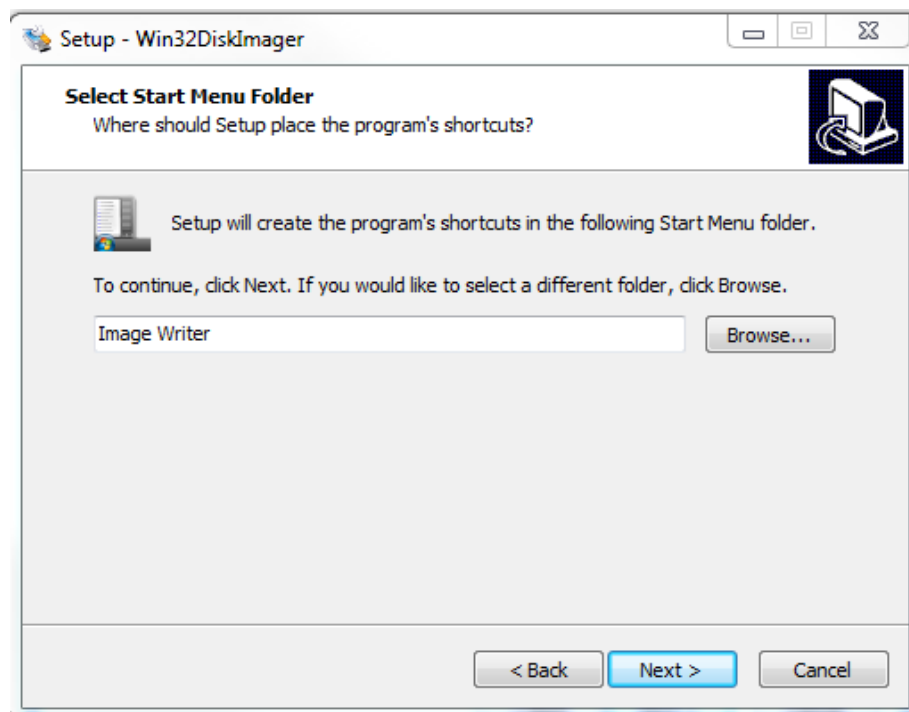
5. Check the radio button for **I accept the agreement** and click **Next**.
6. Configure the installation path as below and click **Next**.

Figure 148. Installation Path



7. Leave the Select Start Menu Folder as the default **Image Writer** and click **Next**.

Figure 149. Select Start Menu Folder



8. Check the box for **Create a desktop icon** and click **Next**. The **Ready to Install** window appears.
9. Click **Install** to proceed with the installation.
10. When the installation wizard has finished, the **Completing the Win32DiskImager Setup Wizard** window appears. Click **Finish** to exit the setup. Win32DiskImager is now installed.

When the installation process completes, you can launch Win32DiskImager and view the README file. The README describes the functions and limitations of the Win32DiskImager and indicates that Win32DiskImager is a free tool released under a GNU General Public License (GPL).

1.14.1. Using Win32Disk Imager

Using Win32DiskImager is a very simple three-step process:

1. Select the IMG file you want to write.
2. Select the removable device onto which to write the IMG file.
3. Hit the **Write** button and wait for the process to complete.

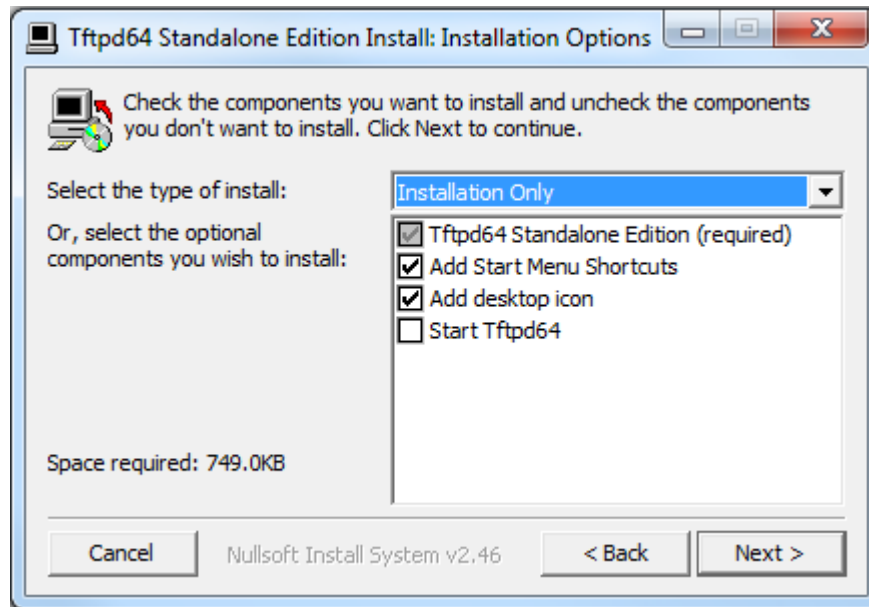
1.15. TFTPd64 By Ph.Jounin Installation

Task time: 30 minutes

Tftp64 is an IPv6 ready application which includes DHCP, TFTP, DNS, SNTP and Syslog servers as well as a TFTP client. The program allows you to maximize performance when transferring the data from a TFTP server. The included DHCP server provides unlimited automatic or static IP address assignment.

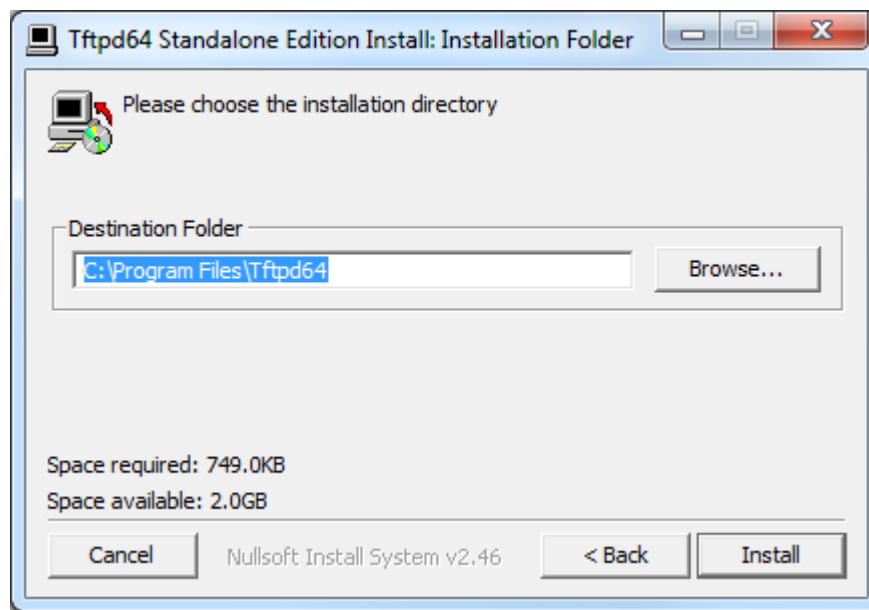
1. Download the tool from [here](#)
2. Download the installer, **tftpd64-4.51-setup.exe**, into your local drive.
3. Double-click on the Tftpd64 installer, **tftpd64-4.51-setup.exe**.
4. Click **I Agree**.
5. Leave the installation components as default selection and click **Next** to proceed with the installation.

Figure 150. Tftpd64 Installation Options



6. Leave the installation directory as default and click **Install** to proceed.

Figure 151. Tftpd64 Installation Directory



7. When the installation completes, click **Close** to exit the installation setup.

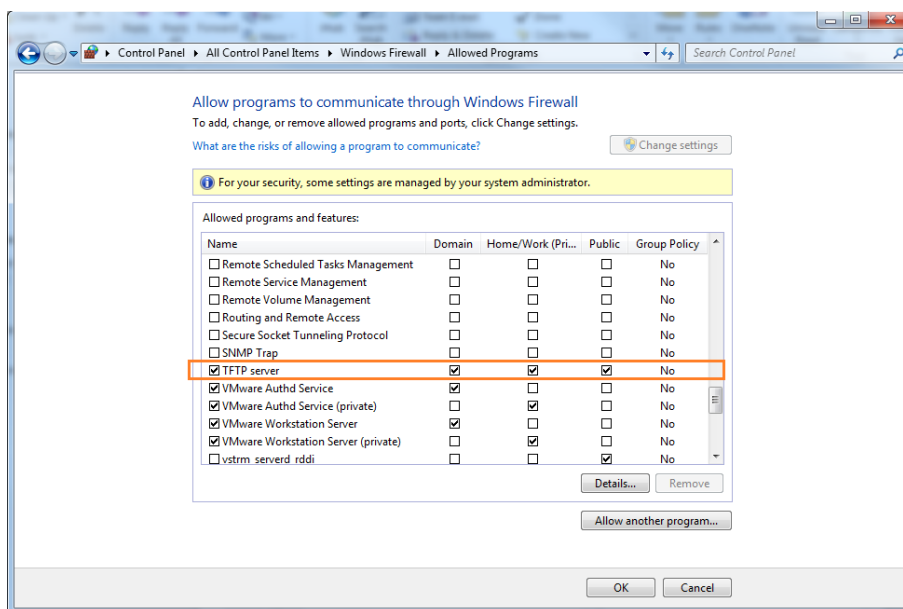
1.15.1. Checking Your Machine or Laptop IPv4 Address

1. Click on your machine Internet connection access icon by selecting **Open Network and Sharing Center**.
2. Select **Local Area Connection 3**. The **Local Area Connection 3 Status** pop-up window appears.
3. Click on **Details...**
4. Read the IPv4 Address.

1.15.2. Enabling the TFTP Server through the Windows Firewall

1. Click on your machine Internet connection access icon follow by selecting **Open Network and Sharing Center**.
2. Select **Windows Firewall**.
3. Select **Allow a program or feature through Windows Firewall**.
4. Scroll down and check the **TFTP Server** checkbox to allow this program to communicate through Windows Firewall

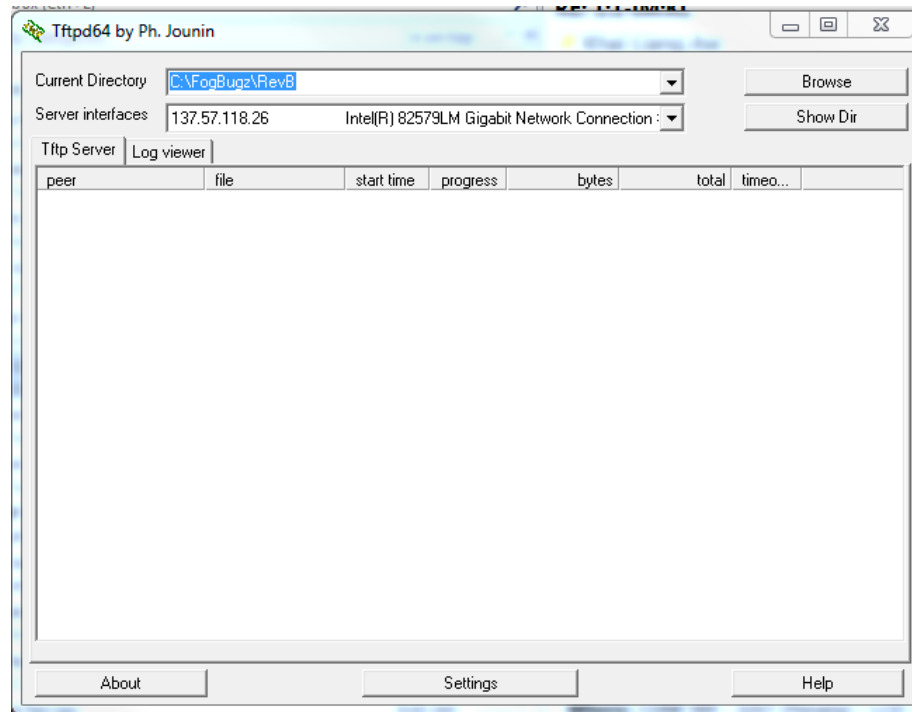
Figure 152. Enabling TFTP Server to Communication Through Firewall



1.15.3. Using the TFTP Server in Windows

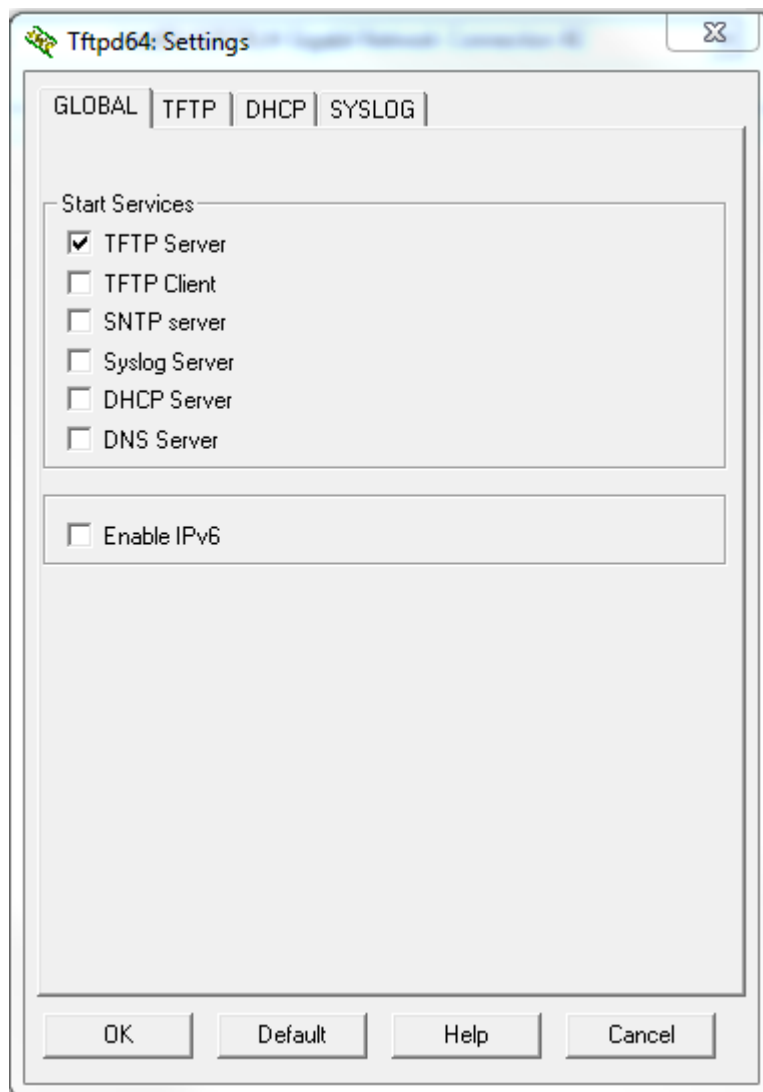
1. Start Tftpd64 by selecting **Start > All > Tftpd64 > Tftpd64**. Alternatively, you can double-click the **Tftpd64** icon on your desktop.
2. When the Tftpd64 program opens, click on the **Settings** button.

Figure 153. Selecting the Settings Button



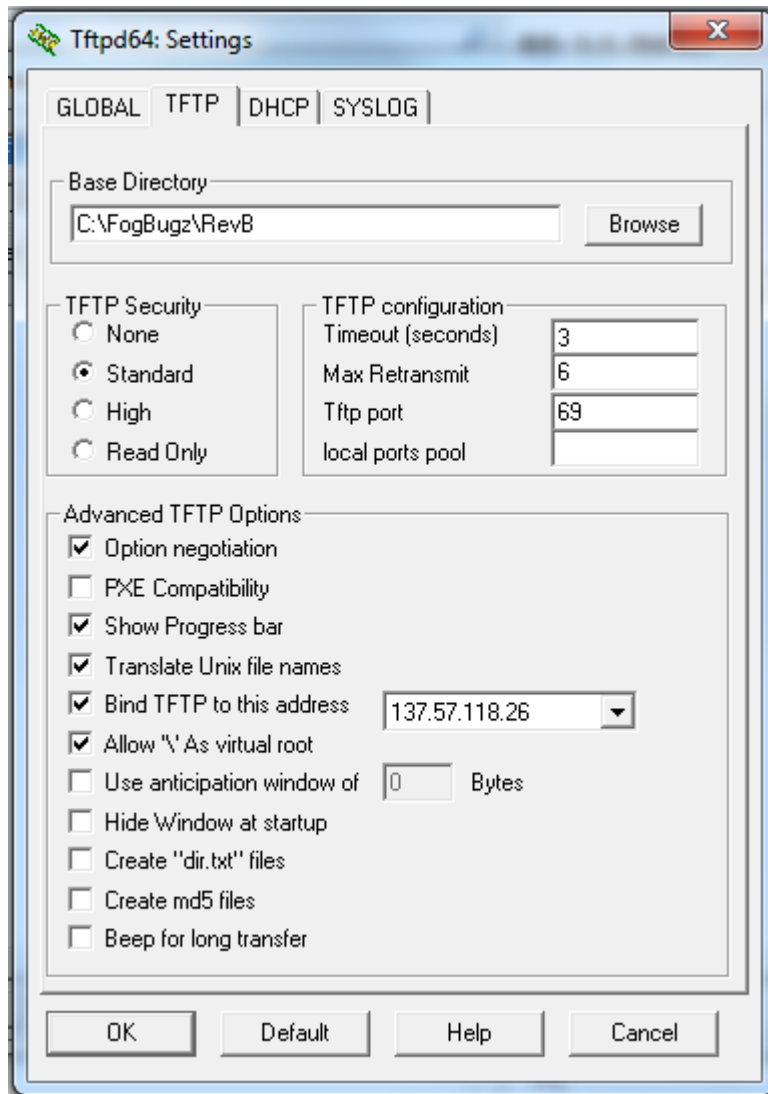
3. Click on the **Global** tab and put a check mark only to the **TFTP Server** option. Remove checkmarks from all other options.

Figure 154. Tftpd64 Settings



4. Click on the **TFTP** tab. The tab appears as shown below.

Figure 155. Tftpd64 Global Settings: TFTP Tab



5. Click on the **Browse** button to specify the **Base Directory**. You must specify the Base Directory of the TFTP Server. Select the TFTP folder in your computer as the Base Directory.
Note: All the files transferred using TFTP Server are placed in the Base Directory.
6. Under the **TFTP Security** option, select the **Standard** option. Ensure that you select the option **Bind TFTP to this address**, which is your machine IPv4 address. Use the address that is available in the drop down box.
7. Once you have performed all the above steps, click **OK**. A window appears asking to restart Tftpd64 to apply the new settings.
8. Click **OK**.

1.16. Revision History of Intel Arria 10 SoC UEFI Boot Loader User Guide

Version	Changes
2017.12.15	Initial release